Steps in Interfacing an I/O Device

- Connect the data bus of the microprocessor system with the data bus of the I/O port.
- Derive a device address pulse by decoding the required address of the device and use it as the chip select of the device.
- Use a suitable control signal i.e. IORD or IOWR to carry out device operations

I/O Structure of a Typical Microcomputer

- The IO devices connected to a microprocessor system provides an efficient means of communication between the microprocessor system and outside world.
- The characteristics of the IO devices are normally different from the characteristics of the microprocessor.
- Since the characteristics of the IO devices are not compatible with that of microprocessor, an interface hardware circuitry between microprocessor and the IO device is necessary.

I/O Structure of a Typical Microcomputer Cont..

- There are three major types of data transfer between the microprocessor and an IO device. They are:

1. Programmed IO
2. Interrupt driven IO
3. Direct memory access (DMA)

- In programmed IO, the data transfer is accomplished through IO port and controlled by a software.
- In Interrupt driven IO, the IO device will interrupt the processor and initiate data transfer.
- In DMA, the data transfer between the memory and IO can be performed by bypassing the microprocessor.

Interfacing IO and Peripheral devices

- IO devices are generally slow devices and so they are connected to system bus through ports.
- The ports are buffered IC's which are used to temporarily hold the data transmitted from the microprocessor to the IO device.

For data transfer from the input device to the processor, the following operations are performed:

1. The input device will load the data to port.
2. When the port receives a data, it sends a message to the processor to read the data.
3. The processor will read the data from the port.
4. After a data has been read by the processor, the input device will load the next data into the port.

Interfacing IO and Peripheral devices Cont..

For data transfer from processor to the output device , the following operations are performed:

1. The processor will load the data to port.
2. The port will send a message to the output device to read the data.
3. The output device will read the data from the port.
4. After a data has been read by the output device , the processor can load the next data to the port.

- Latch acts as a good output port.
- Buffer acts as a good input port.

Interfacing IO and Peripheral devices Cont..

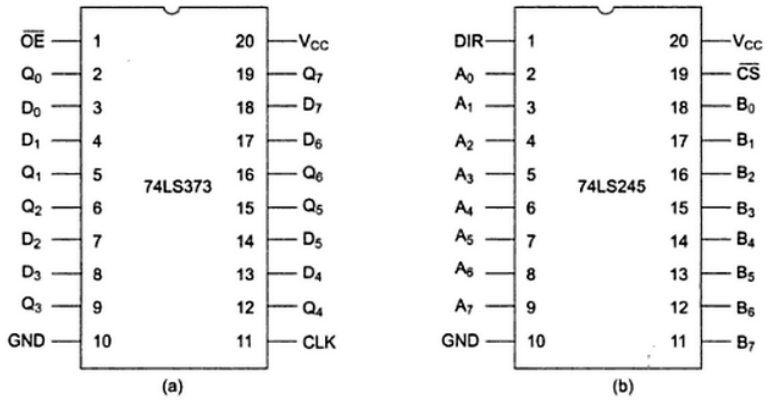


Fig. a) Latch (O/P Port) b) Buffer (I/P Port)

26

Methods of Interfacing IO devices

- There are two ways of interfacing IO devices in a microprocessor/microcontroller based system:
  - Memory Mapped IO Device
  - Standard IO Mapped IO Device or Isolated IO Mapping

Comparison of Memory mapping and IO mapping

Memory Mapping	IO mapping
1. 20-bit addresses are provided for IO devices.	1. 8-bit or 16-bit address are provided for IO devices
2. The IO ports or peripherals can be treated like memory locations and so all instructions related to memory can be used for data transfer.	2. Only IN and OUT instructions can be used for data transfer between IO device and the processor.
3. In memory mapped ports, the data can be moved from any register to port and vice versa	3. In IO mapped ports, the data transfer can take only between the accumulator and the ports
4. When memory mapping is used for IO devices, the full memory address space cannot be used for addressing memory.	4. When IO mapping is used for IO devices, then the full address space can be used for addressing memory.
5. For accessing memory mapped devices, the processor executes the memory read or write cycle. During this cycle, *M / IO* is asserted high.	5. For accessing IO mapped devices, the processor executes the IO read or write cycle. During this cycle, *M / IO* is asserted low.

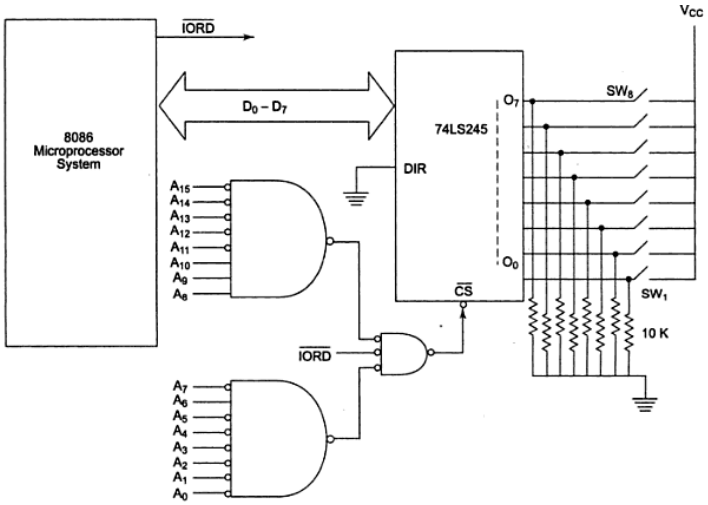
28

Problem

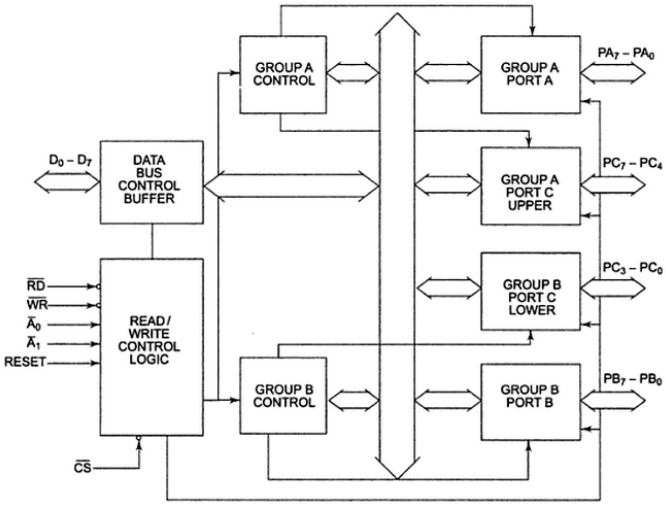
- Interface an input port 74LS245 to read the status of switches SW1 to SW8. The switches, when shorted, input a 1 else input a 0 to the microprocessor system. Store the status in register BL. The address of the port is 0740H.
- The ALP is given as follows:

```
MOV BL,00
        MOV DX,0740H
        IN AL,DX
        MOV BL,AL
        HLT
```

Problem



Programmable Peripheral Interface (PPI) 8255 or Programmable Peripheral Input Output Port or Parallel Input Output Port (PIO)



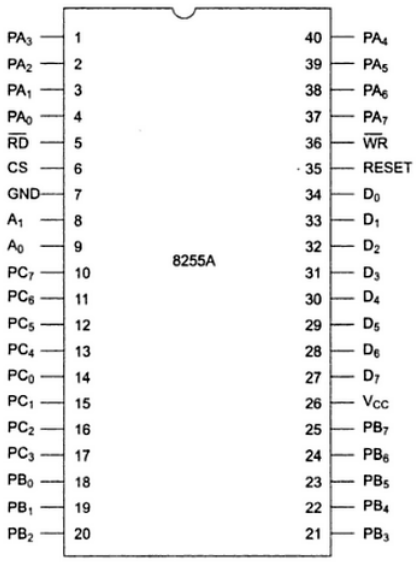
PROGRAMMABLE INPUT-OUTPUT PORT-8255PIO

- The parallel input-output port chip 8255 is also known as programmable peripheral input-output port.
- It has 24 input/output lines which may be individually programmed in two groups of twelve lines each, or three groups of eight lines.
- The two groups of I/O pins are named as Group A and Group B.
- Each of these two group contains a subgroup of eight I/O lines called as 8-bit port and another subgroup of four I/O lines or a 4-bit port.
- Thus Group A contains an 8-bit port A along with a 4-bit port, C upper. The port A lines are identified by symbols PA_0 –PA_7 while the port C lines are identified as PC_4-PC_7.

PROGRAMMABLE INPUT-OUTPUT PORT-8255Cont..

- Similarly Group B contains an 8-bit port and a 4-bit port C with lower bits.
- The port C upper and port C lower can be used in combination as an 8-bit port C.
- All of these ports function independently either as input or as output ports.
- This can be achieved by programming the bits of internal register of 8255 called as Control WordRegister.
- The 8-bit data bus buffer is controlled by read/write control logic.
- The read/write control logic manages all of the internal and external transfer of both data and control words.

Pin Diagram of Programmable Peripheral Interface (PPI) 8255



Pin Diagram of Programmable Peripheral Interface (PPI) 8255

PA_7-PA_0 These are eight port A lines that act as either latched output or buffered input lines depending upon the control word loaded into the control word register.

PC_7-PC_4 Upper nibble of port C lines. They may act as either output latches or input buffers lines. This port also can be used for generation of handshake lines in mode1 or mode 2.

PC_3-PC_0 These are the lower port C lines, other details are the same as PC7-PC4 lines.

PB_0-PB_7 These are the eight port B lines which are used as latched output lines or buffered input lines in the same way as port A.

$\overline{RD}$ This is the input line driven by the microprocessor and should be low to indicate read operation to 8255.

$\overline{WR}$ This is an input line driven by the microprocessor. A low on this line indicates write operation.

$\overline{CS}$ This is a chip select line. If this line goes low, it enables the 8255 to respond to $\overline{RD}$ and $\overline{WR}$ signals, otherwise $\overline{RD}$ and $\overline{WR}$ signals are neglected.

A_1-A_0 These are the address input lines and are driven by the microprocessor. These lines ($A_1 - A_0$) with $\overline{RD}$, $\overline{WR}$ and $\overline{CS}$ form the following operations for 8255. These address lines are used for addressing any one of the four registers, i.e. three ports and a control word register

Pin Diagram of Programmable Peripheral Interface (PPI) 8255

In case of 8086 systems, if the 8255 is to be interfaced with lower order data bus, the A_0 and A_1 pins of 8255 are connected with A_1 and A_2 respectively.

Table (a)

$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	A_1	A_0	*Input (Read) cycle*
0	1	0	0	0	Port A to data bus
0	1	0	0	1	Port B to data bus
0	1	0	1	0	Port C to data bus
0	1	0	1	1	CWR to data bus

Table (b)

$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	A_1	A_0	*Output (Write) cycle*
1	0	0	0	0	Data bus to Port A
1	0	0	0	1	Data bus to Port B
1	0	0	1	0	Data bus to Port C
1	0	0	1	1	Data bus to CWR

Table (c)

$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	A_1	A_0	*Function*
X	X	1	X	X	Data bus tristated
1	1	0	X	X	Data bus tristated

Pin Diagram of Programmable Peripheral Interface (PPI) 8255

D_0-D_7 These are the data bus lines those carry data or control word to/from the micro-processor.

RESET A logic high on this line clears the control word register of 8255. All ports are set as input ports by default after reset.

Modes of Operation of 8255

- There are two basic modes of operation of 8255 - I/O mode and BSR (Bit-Set-Reset) mode.
- In the I/O mode, the 8255 ports work as programmable I/O ports, while in BSR mode only port C (PC_0 – PC_7) can be used to set or reset its individual port bits.
- Under the I/O mode of operation, further there are three modes of operation of 8255,so as to support different types of applications, viz. *mode 0*, *mode 1* and *mode 2*.

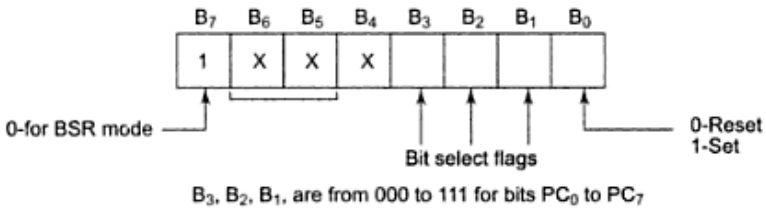
BSR Mode

- In this mode, any of the 8-bits of port C can be set or reset depending on B_0 of the control word.
- The bit to be set or reset is selected by bit select flags B_3, B_2 and B_1 of the CWR as given in the table below.

B_3	B_2	B_1	*Selected Bits of port C*
0	0	0	B_0
0	0	1	B_1
0	1	0	B_2
0	1	1	B_3
1	0	0	B_4
1	0	1	B_5
1	1	0	B_6
1	1	1	B_7

BSR Mode

- The CWR format is as shown below:



B_3, B_2, B_1, are from 000 to 111 for bits PC_0 to PC_7

I/O Modes – MODE 0 (Basic I/O Mode)

- This mode is also known as basic input/output mode.
- This mode provides input and output capability using each of the three ports.
- Data can be simply read from or written to the input and output ports respectively, after appropriate initialization.

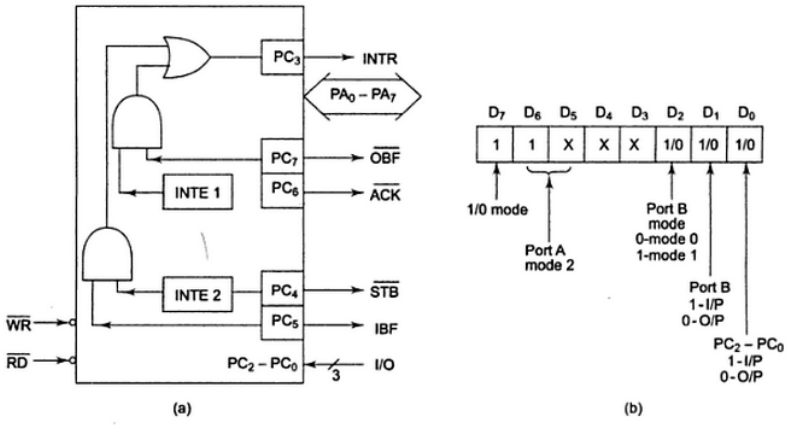
I/O Modes – MODE 0 (Basic I/O Mode)

- The salient features of this mode are

1. Two 8 bit ports(port A and port B) and two 4-bits ports(port C upper and lower) are available. The two 4-bit ports can be combinedly used as a third 8-bit port.
2. Any port can be used as an input and output port
3. Output ports are latched. Input ports are not latched.
4. A maximum of four ports are available so that overall 16 I/O configurations are possible.

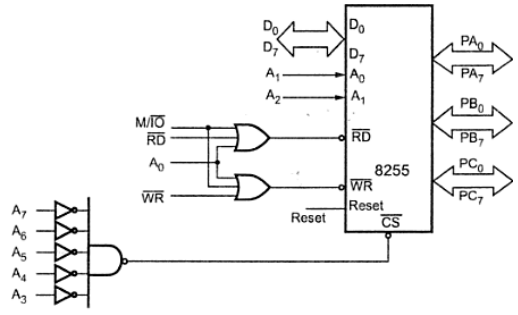
- All these modes can be selected by programming a register internal to 8255, known as Control Word Register (CWR) which has two formats. The first format is valid for I/O modes of operation i.e, mode 0, mode 1 and mode 2 while the second format is valid for bit set/reset (BSR) mode of operation.

I/O Modes – Mode 2 (Strobed Bidirectional I/O Mode)



(a) *Mode 2 Pins* (b) *Mode 2 Control Word*

Interfacing 8255 to 8086



I/O mapped I/O

I/O Map :

Port / control Register	Address lines								Address
	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	
Port A	0	0	0	0	0	0	0	0	00H
Port B	0	0	0	0	0	0	1	0	02H
Port C	0	0	0	0	0	1	0	0	04H
Control register	0	0	0	0	0	1	1	0	06H

Interfacing 8255 to 8086

Problem: Interface an 8255 with 8086 to work as an I/O port. Initialize port A as output port, port B as input port and port C as output port. Port A address should be 0740H. Write a program to sense switch positions SW0-SW7 connected to port B. the sensed pattern is to display on port A, to which 8 LEDs are connected, while the port C lower displays number of on switches out of the total eight switches.

Solution:

B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0	Control word
1	0	0	0	0	0	1	0	= 82H
I/O mode	Port A in mode 0		Port A,o/p	Port C,o/p	Port B,mode 0	Port B,i/p	Port C,o/p	

Interfacing 8255 to 8086 Cont..

Solution: Thus 82H is the control word for the requirements in the problem. The port address can be done as given below.

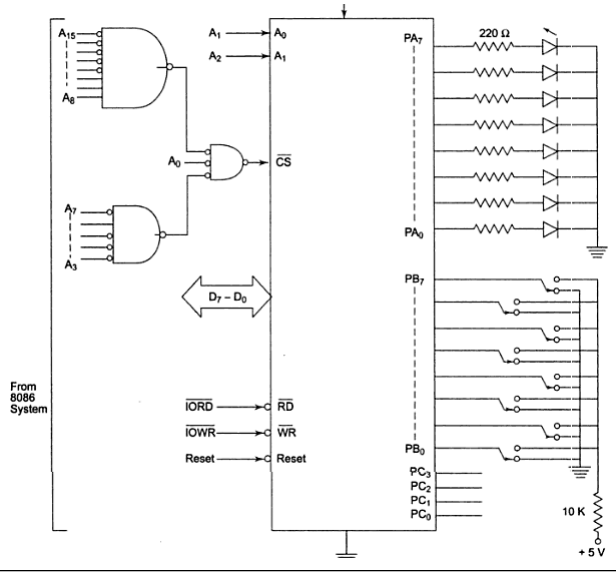
8255 Ports	I/O Address lines A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_{09}	A_{08}	A_{07}	A_{06}	A_{05}	A_{04}	A_{03}	A_{02}	A_{01}	A_{00}	Hex. Port Addresses
PortA	0	0	0	0	0	1	1	1	0	1	0	0	0	0	0	0	0740H
Port B	0	0	0	0	0	1	1	1	0	1	0	0	0	0	1	0	0742H
8255 Ports	I/O Address lines A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_{09}	A_{08}	A_{07}	A_{06}	A_{05}	A_{04}	A_{03}	A_{02}	A_{01}	A_{00}	Hex. Port Addresses
Port C	0	0	0	0	0	1	1	1	0	1	0	0	0	1	0	0	0744H
CWR	0	0	0	0	0	1	1	1	0	1	0	0	0	1	1	0	0746H

Interfacing 8255 to 8086 Cont..

Solution: The 8255 is to be interfaced with lower order data bus, i.e. D_0-D_7

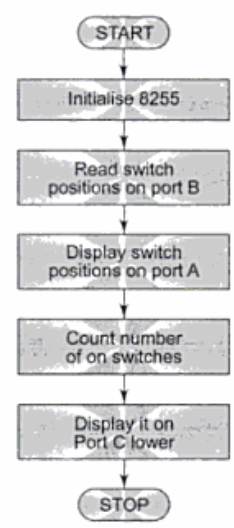
- The A_0 and A_1 pins of 8255 are connected to A_{01} and A_{02} pins of microprocessor respectively.
- The A_{00} pin of the microprocessor is used for selecting lower byte of data bus.
- Hence any change in the status of A_{00} does not affect the port to be selected.
- Let us use absolute decoding scheme that uses all the 16 address lines for deriving the device address pulse.
- Out of A_0-A_{15} lines, two address lines A_{02} and A_{01} are directly required by 8255 for three port and CWR address decoding.
- Hence only A_3 to A_{15} are used for decodingaddress.

Interfacing 8255 to 8086 Cont..



Interfacing 8255 to 8086 Cont..

Flow Chart



Interfacing 8255 to 8086 Cont..

Program

```
    MOV DX,0746H        ; Initialize CWR with
    MOV AL,82H          ; control word 82H
    OUT DX,AL           ;
    SUB DX,04           ;Get address of port B in DX
    IN AL,DX            ;Read port B for switch
    SUB DX,02           ; positions in to AL and get port A
                        address in DX
    OUT DX,AL           ; Display switch positions on port A
    MOV BL,00H          ; Initialize BL for switch count
    MOV CH,08H
YY: ROL AL              ; Rotate AL through carry to check,
    JNC XX                      ; whether the switches are on or
    INC BL                      ;off, i.e. either 1 or 0
XX: DEC CH              ; Check for next switch. If all
    JNZYY                       ; switch are checked, the
    MOV AL,BL           ; number of on switches are
    ADD DX,04           ; in BL. Display it on port C
    OUT DX,AL           ; lower
    HLT
```

Interfacing to Alphanumeric Displays

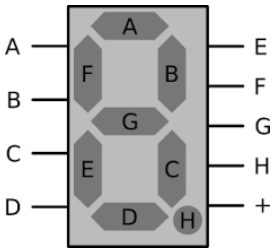
- Most of the microprocessor controlled instruments and machines need to display letters of the alphabet and numbers to give directions or data values to users.
- This can be displayed using CRT, LED or LCD displays.
- CRT displays are used when a large amount of data is to be displayed.
- In systems where only a small amount of data is to be displayed, simple LED and LCD displays are used.

Interfacing to Alphanumeric Displays Cont..

LED Displays

- LED Displays are available in two very common formats.

a. 7 segment displays
b. 5 by 7 dot-matrix displays



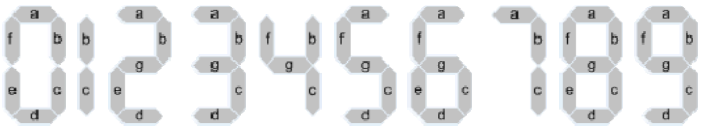


Interfacing to Alphanumeric Displays Cont..



Seven-Segment displays

- 7 segment displays are generally used as numerical indicators and consists of a number of LEDs arranged in seven segments.
- Any number between 0 to 9 can be indicated by lighting the appropriate segments.
- The seven segments are labeled a to g and dot is labeled as h.
- By forward biasing different LED segments, we can display the digits 0 through 9.



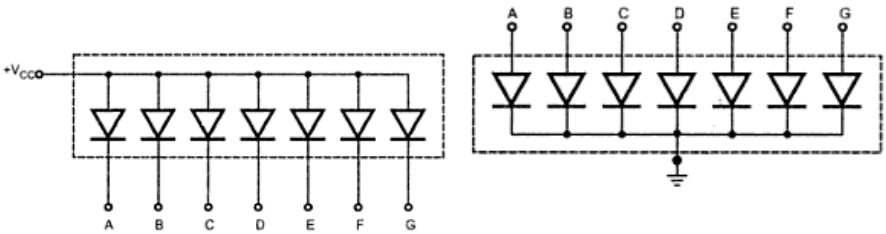
Interfacing to Alphanumeric Displays Cont..



Seven-Segment displays

- These 7 segment displays are of two types:

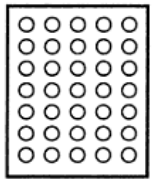
a. Common Anode Type
b. Common Cathode Type

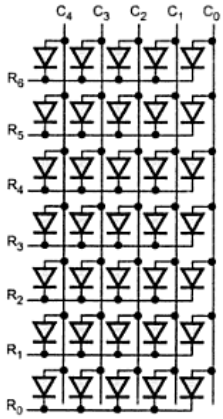


Interfacing to Alphanumeric Displays Cont..

5 by 7 DOT-matrix LED

- These display can be used to display number as well as alphabets.

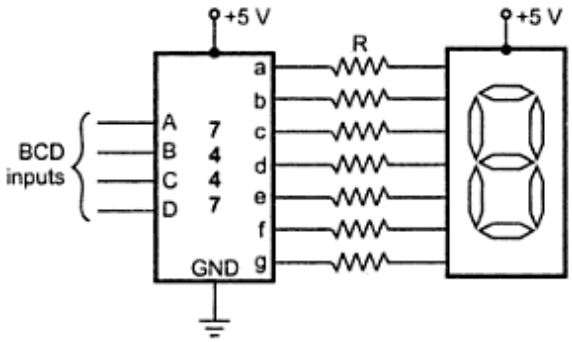




Interfacing LED Displays

Static Display

Fig shows a circuit to drive a single, seven segment, common anode LED display.



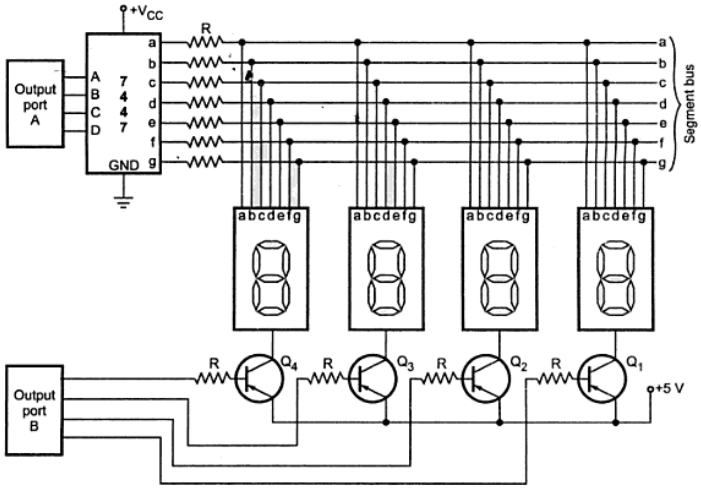
Interfacing LED Displays

Static Display

- For common anode, when anode is connected to positive supply, a low voltage is applied to a cathode to turn it on.
- Here BCD to seven segment decoder, IC7447 is used to apply low voltages at cathodes according to BCD input applied to 7447.
- To limit the current through LED segments resistors are connected in series with the segments.
- This circuit connection is referred to as a static display because current is being passed through the display at all times.

Interfacing LED Displays

Multiplexed Display



Interfacing LED Displays

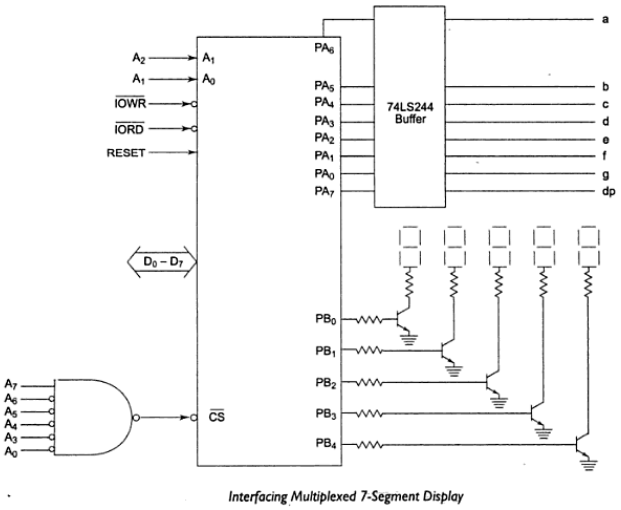
Multiplexed Display

- Here, common anode seven segment LEDs are used.
- Anodes are connected to +5V through transistors. Cathodes of all seven segments are connected in parallel and then to the output of 7447IC through resistors.
- In multiplexed display the segment information sent for all digits on the common lines, but only one display digit is turned on at a time.
- The PNP transistors connected in series with the common anode of each digit act as an ON and OFF switch for that digit.

Interfacing to Alphanumeric Displays

Problem: Interface an 8255 with 8086 at 80H as an I/O address of port A. interface five 7 segment displays with 8255. Write a sequence of instructions to display 1, 2, 3, 4 and 5 over five displays continuously as per their positions starting with 1 at the least significant position.

Interfacing to Alphanumeric Displays



Interfacing Multiplexed 7-Segment Display

69

Interfacing to Alphanumeric Displays

Solution: In this scheme. I/O port A is multiplexed to carry data to all the 7-segment displays. The port B selects one of the display at a time.

- The display used are common anode type.

Number to be displayed	PA_7 dp	PA_6 a	PA_5 b	PA_4 c	PA_3 d	PA_2 e	PA_1 f	PA_0 g	Code
1	1	1	0	0	1	1	1	1	CF
2	1	0	0	1	0	0	1	0	92
3	1	0	0	0	0	1	1	0	86
4	1	1	0	0	1	1	0	0	CC
5	1	0	1	0	0	1	0	0	A4

All these codes, decided above, are stored in a look up table starting at 2000:0001

Interfacing to Alphanumeric Displays

Program:

```
AGAIN: MOV CL,05H ;Count for displays
       MOV BX,2000H        ;Initialize data segment
       MOV DS,BX     ;for look up table
       MOV CH,01H ;1st number to be displayed
       MOV AL,80H    ;Load control word in the
       OUT 86H,AL    ;CWR
        MOV DL,01H ;Enable code for Least
            ;significant 7-seg display
```

Interfacing to Alphanumeric Displays

Program:

```
NXTDGT: MOV BX,0000H;Set pointer to look up table
        MOV AL,CH ;First no to display
                  ;Store number to be displayed in AL
        XLAT          ;Find code from look up table
        OUT 80H,AL ;Display the code
        MOV AL,DL             ;Enable the display
        OUT 82H,AL;
        ROL DL                ;Go for selecting next display
        INC CH                ;Next number to display
        DEC CL
       JNZ NXTDGT ;Go for next digit display
       JMP AGAIN
```

Interfacing to Keyboard

Problem: Interface a 4*4 keyboard with 8086 using 8255 and write an ALP for detecting a key closure and return the key code inAL. The debouncing period for a key is 10ms. Use software key debouncing technique. DEBOUNCE is an available 10ms delay routine.

Interfacing to Keyboard Cont..

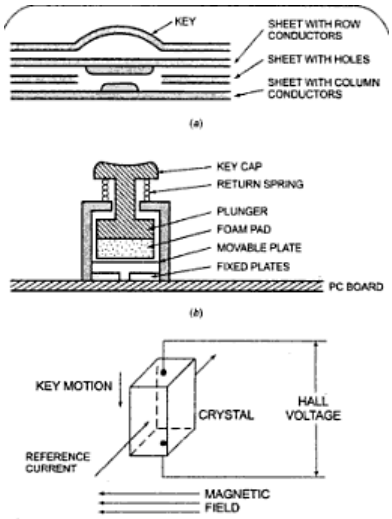
KeyboardTypes:

- When you press a key on your computer, you are activating a switch.There are many different ways of making these switches.

- Mechanical Key Switches
- Membrane Key Switches
- Capacitive Key Switches
- Hall Effect Key Switches

Interfacing to Keyboard Cont..

KeyboardTypes:



Interfacing to Keyboard Cont..

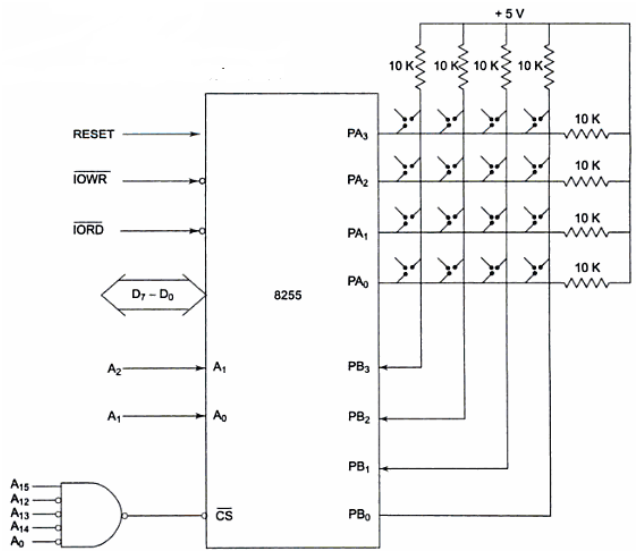
Keyboard circuit connections and interfacing

- In most keyboards, the key switches are connected in a matrix of rows and columns.
- We use simple mechanical switches but the principle is same for other types of switches.
- Getting a meaningful data from a keyboard requires three major tasks.

1. Detect a key press.
2. Debounce the key press
3. Encode the key press

- The three tasks can be done with hardware, software, or a combination of two depending on application.

Interfacing to Keyboard Cont..



77

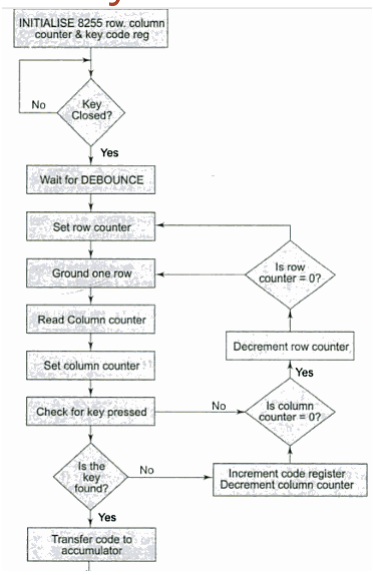
Interfacing to Keyboard Cont..

Solution:

- Port A is used as output port for selecting a rows of key while port B is used as an input port for sensing a closed key.
- Thus keyboard lines are selected one by one through port A and the port B lines are polled continuously till a key closure is sensed.
- Then routine DEBOUNCE is called for debouncing.
- The key code is decided depending upon the selected row and a low sensed column.
- The higher order lines of port A and port B are left unused.
- The address of port A and port B will be respectively 8000H and 8002H while the address of CWR will be 8006H.

Interfacing to Keyboard Cont..

Flow chart:



Interfacing to Keyboard Cont..

Check 1: whether any key is pressed or not ?

1. Make all row lines zero by sending low on all output lines.This activates all keys in the keyboard matrix.
2. Read the status of return lines. If status of all lines is logichigh, key is not pressed; otherwise pressed.

Check 2:

1. Activate keys from any one column by making any one column line zero.
2. Read the status of return lines.The zero on any return line indicates key is pressed from the corresponding row and selected column.
3. Activate the keys from next column and repeat 2 and 3 for all columns.

Interfacing to Keyboard Cont..

Program:

```
CODE --- SEGMENT
ASSUME   CS : CODE
START:   MOV AL, 82H        ; Load CWR with
         MOV DX, 8006H      ; control word
         OUT DX, AL         ; required
         MOV BL, 00H        ; Initialize BL for key code
         XOR AX, AX         ; Clear all flags
         MOV DX, 8000H      ; Port Address in AX.
         OUT DX, AL         ; Ground all rows.
         ADD DX,02          ; Port B address in DX.
WAIT :   IN AL, DX          ; Read all columns.
         AND AL, 0F H       ; Mask data lines D7–D4.
         CMP AL, 0F H       ; Any key closed?
         JZ WAIT            ; If not, wait till key
         CALL DEBOUNCE      ; closure else wait for 10 ms
         MOV AL, 7FH        ; Load data byte to ground
         MOV BH, 04H        ; a row and set row counter.
```

Interfacing to Keyboard Cont..

Program:

```
NXTROW : ROL AL, 01         ; Rotate AL to ground next row.
         MOV CH, AL         ; Save data byte to ground next row.
         SUB DX,02          ; Output port address is in DX.
         OUT DX, AL         ; Ground one of the rows.
         ADD DX,02          ; Input port address is in DX.
         IN AL, DX          ; Read input port for key closure.
         AND AL, 0FH        ; Mask D4–D7.
         MOV CL, 04H        ; Set column counter.
```

```
NXTCOL : ROR  AL, 01        ; Move D0 in CF.
         JNC CODEKY         ; Key closure is found, if CF=0.
         INC BL             ; Increment BL for next binary
                            ; key code.
         DEC CL             ; Decrement column counter,
                            ; if no key closure found.
         JNZ NXTCOL         ; Check for key closure in next column
         MOV AL,CH          ; Load data byte to ground next row.
         DEC BH             ; if no key closer found in column
                            ; get ready to ground next row.
         JNZ NXTROW         ; Go back to ground next row.
         JMP WAIT           ; Jump back to check for key.
                            ; closure again.
```

Interfacing to Keyboard Cont..

Program:

```
CODEKY : MOV AL, BL         ; Key code is transferred to AL.
         MOV AH, 4CH        ; Return to DOS prompt.
         INT 21 H

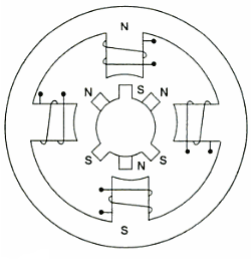
DEBOUNCE PROC NEAR
          MOV CL, OE2H
BACK:     NOP
          DEC CL
          JNZ BACK
          RET
DEBOUNCE  ENDP
CODE      ENDS
          END START
```

Interfacing to Stepper Motor

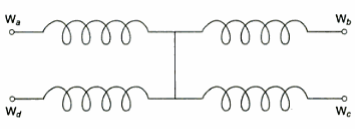
- A stepper motor is a device used to obtain an accurate position control of rotating shafts.
- It employs rotation of its shafts in terms of steps, rather than continuous rotation as in case of AC or DC motors.
- To rotate the shafts of the stepper motor, a sequence of pulses is needed to be applied to the windings of the stepper motor in a proper sequence.
- The number of pulses required for one complete rotation of the shaft of the stepper motor are equal to its number of internal teeth or its rotor.
- The stator teeth and the rotor teeth lock with each other to fix a position of the shaft.

Interfacing to Stepper Motor

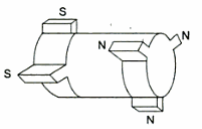
- With a pulse applied to the winding input the rotor rotates by one teeth position or an angle x.
- The angle $x = 360^0$ / no. of rotor teeth
- After the rotation of the shaft through angle x, the rotor locks itself with the next tooth in sequence on the internal surface of stator.



Internal Schematic of a Four Winding Stepper Motor



Winding Arrangement of a Stepper Motor



A Stepper Motor Rotor

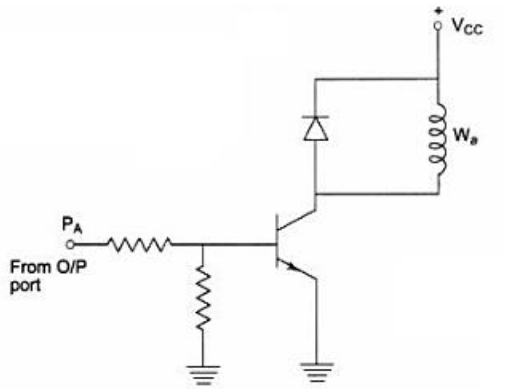
Interfacing to Stepper Motor

Interfacing to Stepper Motor

- A typical stepper motor may have parameters like torque 3Kg-cm, operating voltage 12V, current rating 0.2A and a step angle 1.8°, i.e. 200 steps/revolution (number of rotor teeth).
- The stepper motors are designed to work with digital circuits.
- Binary level pulses of 0-5V are required at its winding inputs to obtain the rotation of shafts.
- The sequence of pulses can be decided upon the required motion of the shaft.

Interfacing to Stepper Motor

- The circuit for interfacing a winding W_n with an I/O port is



- Each of the windings of a stepper motor need this circuit for its interfacing with output port

Interfacing to Stepper Motor

- A simple scheme for rotating the shaft of stepper motor is called a wave scheme.
- In this scheme, the windings W_a, W_b, W_c and Wd are applied with required voltage pulses, in a cyclic fashion.

Excitation Sequences of a Stepper Motor Using Wave Switching Scheme

Motion	*Step*	*A*	*B*	*C*	*D*
Clockwise	1	1	0	0	0
	2	0	1	0	0
	3	0	0	1	0
	4	0	0	0	1
	5	1	0	0	0
Anticlockwise	1	1	0	0	0
	2	0	0	0	1
	3	0	0	1	0
	4	0	1	0	0
	5	1	0	0	0

Interfacing to Stepper Motor

- Design a stepper motor controller and write an ALP to rotate shaft of 4-phase stepper motor:

a. In clock wise 5 rotations

b. In anticlockwise 5 rotations

The 8255 port A address is 0740H. The stepper motor has 200 rotor teeth. The port A bit PA_0 drives winding Wa, PA_1 drives Wb and so on. The stepper motor has an inertial delay of 10ms. Assume that the routine for this delay is already available.

Interfacing to Stepper Motor

- The stepper motor connections for all the four windings are

Interfacing to Stepper Motor

```
ASSUME CS:CODE
CODE SEGMENT
START: MOV AL,80H
       OUT CWR,AL
       MOV AL,88H ;Bit pattern 10001000 to start
       MOV CX,1000        ;the sequence of excitation
AGAIN1:OUT PORTA,AL       ;from Wa. Excite Wa,Wb,
       CALL DELAY ;Wc,Wd in sequence with
       ROL AL,01          ; delay. For clockwise
       DEC CX             ; rotations the count is
       JNZ AGAIN1         ; 200*5=1000. Excite till
                          ;count=0
```

Interfacing to Stepper Motor

```
        MOV AL,88H ;Bit pattern to excite Wa
        MOVCX,1000 ;count for 5 rotations
AGAIN2:OUT PORTA,AL      ;excite Wa,Wb,Wc,Wd
        CALL DELAY       ;Wait
        ROR AL,01        ;anticlockwise
        DEC CX
        JNZ AGAIN2
        MOV AH,4CH
        INT 21H
CODE ENDS
END          START
```

Interfacing to Stepper Motor

- The count for rotating the shafts of stepper motor through a specified angle may be calculated from number of rotor teeth.
- The number of rotor teeth is equal to the count of one rotation i.e. 360^0.
- Hence for any specified angle θ^0 the count is calculated as

$$C = \frac{Number\ of\ rotor\ teeth}{360^0} \times 0^0$$

Interfacing to A/D Converter

- The analog to digital converter is treated as an input device by the microprocessor , that sends an initializing signal to ADC to start the analog to digital data conversion process.
- The start of conversion signal is a pulse of a specific duration.
- The process of analog to digital conversion is a slow process, and the microprocessor has to wait for the digital data till the conversion is over.
- After conversion is over, theADC sends end of conversion (EOC) signal to inform the microprocessor about it and the result is ready at the output buffer of theADC.

Interfacing to A/D Converter Cont..

- These tasks of issuing an SOC pulse to ADC, reading EOC signal from the ADC and reading the digital output of the ADC are carried out by the CPU using 8255 I/O ports.
- The time taken by the ADC from the active edge of SOC pulse till the active edge of EOC signal is called as the conversion delay of theADC.
- The available ADCs in the market use different conversion techniques for conversion of analog signal to digital signals.
- Successive approximation and dual slope integration techniques are most popular techniques used in integrated ADC chips.

Interfacing to A/D Converter Cont..

A general algorithm for ADC interfacing contains:

1. Ensure the stability of analog input applied to the ADC
2. Issue start of conversion (SOC) pulse to ADC
3. Read end of conversion (EOC) signal to mark the end of conversion process
4. Read digital data output of the ADC as equivalent digital output

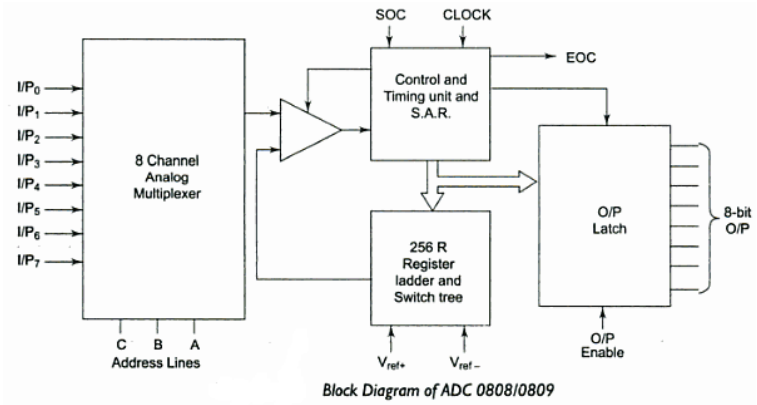
- It may be noted that the analog input voltage must be a constant at the input of the ADC right from the beginning to end of the conversion to get correct results.

Interfacing to A/D Converter Cont..

- This may be ensured by a sample and hold circuit which samples the analog signal and holds it constant for a specified time duration.
- The microprocessor may issue a hold signal to the sample and hold circuit.
- If the applied input changes before the complete conversion process is over, the digital equivalent of the analog input calculated by the ADC may not be correct.

ADC 0808/0809

The analog to digital converter chips 0808 and 0809 are 8-bit CMOS, successive approximation converters.



Block Diagram of ADC 0808/0809

ADC 0808/0809 Cont..

- These converters internally have 3:8 analog multiplexer so that at a time eight different analog inputs can be connected to chip.
- Out of these eight inputs only one can be selected for conversion using address lines ADD A, ADD B and ADD C

Analog I/P selected	Address lines		
	C	B	A
I/P 0	0	0	0
I/P 1	0	0	1
I/P 2	0	1	0
I/P 3	0	1	1
I/P 4	1	0	0
I/P 5	1	0	1
I/P 6	1	1	0
I/P 7	1	1	1

ADC 0808/0809 Cont..

- These are unipolar analog to digital converters, i.e. they are able to convert only positive analog input voltages to their digital equivalents.

ADC 0808/0809 Cont..

- Problem: Interface ADC 0808 with 8086 using 8255 ports. Use port A of 8255 for transferring digital data output of ADC to the CPU and port C for control signals. Assume that an analog input is present at I/P_2 of the ADC and a clock input of suitable frequency is available for ADC. Draw the schematic and write required ALP.

ADC 0808/0809 Cont..

ADC 0808/0809

- The analog input I/P_2 is used and therefore address pinsA, B, C should be 010 respectively to select I/P_2.
- The OE andALE pins are already kept at +5V to select theADC and enable the outputs.
- Port C upper acts as the input port to receive the EOC signal while port C lower acts as output port to send SOC to theADC.
- Port A acts as a 8-bit input data port to receive the digital data output from theADC.

ADC 0808/0809

- The required ALP

```
        MOV AL,98 H        ; Initialise 8255 as
        OUT CWR,AL         ; discussed above
        MOV AL,02H         ; Select I/P2 as analog
        OUT Port B,AL      ; input
        MOV AL,00H         ; Give start of conversion
        OUT Port C,AL      ; pulse to the ADC.
        MOV AL,01 H        ;
        OUT Port C,AL      ;
        MOV AL,00H         ;
        OUT Port C,AL      ;
WAIT  : IN AL,PortC        ; Check for EOC by
        RCL                ; reading port C upper and
        JNC WAIT           ; rotating through carry.
        IN AL,PortA        ; If EOC, read digital equivalent in AL
        HLT                ; Stop
```

Interfacing to D/A Converters

- The digital to analog converters converts binary numbers into their corresponding analog equivalent voltages.
- The DAC find applications in areas like digitally controlled gains, motor speed controls, programmable gain amplifiers etc.

DAC 0800 Cont..

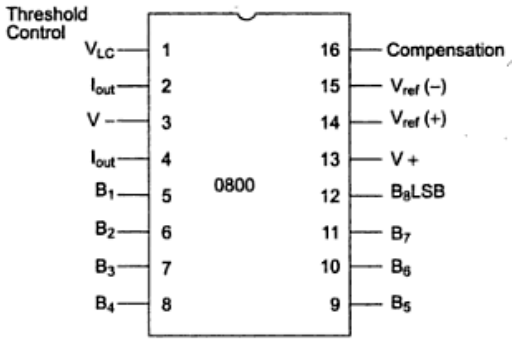
- The V_{ref+} should be tied to +5V to generate a wave of +5V amplitude.
- The required frequency of the output is 500 Hz i.e. the period is 2ms.
- Assuming the wave to be generated is symmetric, the waveform will rise for 1 ms and fall for 1ms. This will be repeated continuously.

DAC 0800 Cont..

```
ASSUME     CS : CODE
CODE       SEGMENT
START :    MOV AL,80 H        ; Initialise 8255 ports
           OUT CWR,AL         ; suitably.
           MOV AL,00H         ; Start rising ramp from
BACK :     OUT Port A,AL      ; 0V by sending 00H to DAC.
           INC AL             ; Increment ramp till 5V
           CMP AL,FFH         ; i.e. FFH.
           JB BACK            ; If it is FFH then,
BACK1 :    OUT Port A,AL      ; Output it and start the falling
           DEC AL             ; ramp by decrementing the
           CMP AL,00          ; counter till it reaches
           JA BACK1           ; zero. Then start again
           JMP BACK           ; for the next cycle.
CODE       ENDS
           END START
```

DAC 0800

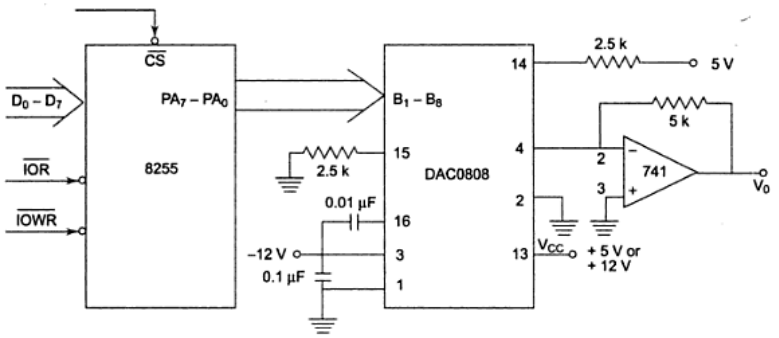
- The DAC 0800 is a monolithic 8-bit DAC manufactured by National Semiconductor.



Pin Diagram of DAC 0800

DAC 0800 Cont..

Problem: Write an ALP to generate a triangular wave of frequency 500 Hz using the DAC 0800 interfacing. The 8086 system operates at 8 MHz the amplitude of the triangular wave should be +5V.



Interfacing DAC0800 with 8086

Microprocessor System

Arithmetic instructions:

Arithmetic instructions:

1) **ADD instruction:**
2) **ADC instruction:**
3) **SUB instruction:**
4) **SBB instruction:**
5) **INC instruction:**
6) **DEC instruction:**
7) **CMP instruction:**
8) **MUL instruction:**
9) **DIV instruction:**

➢After each arithmetic operation we must check flag register.

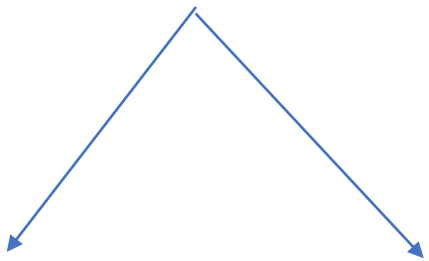
Number

1)sign number →second complement

2)unsigned number

Mov AL,-12

F4H



sign -12

unsigned 244

MOV AL,-12H → second complement = EEH

ADD instruction:

- Perform addition on binary numbers.
- The general format for ADD is:

ADD destination, source

- **Destination=destination + source**

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

- Registers and memory can be 8-bit or 16-bit length.

ADD AX,BX

AX = AX + BX

INC instruction:

- Increment by 1.
- The same as ADD dest,1.
- The general format for INC is:

INC destination

Destination=destination + 1

Destination: represents register or memory.

➢does not effect on the carry flags

INC AX

AX=AX+1

ADC instruction:

- Add with carry.
- The general format for ADC is:

ADC dest, src

Destination=destination + source + CF

Where:

Destination: represents register or memory.

Source : represents register, memory, or immediate.

- Used specially with 32-bit numbers. (can be used with others length)

ADC AX,BX

AX = AX + BX+CF

Ex:

MOV AL,-3 → FDH second complement

ADD AL,5 → AL=2

1111 1101 +

0000 0101

---

1 0000 0010 CF=1 AF=1 PF=0 SF=0 ZF=0 OF=0

FF00

FFFF

---

carry =1 FEFF

EX:

```
SUM DW 0FF00H
MOV AX,0FFFFH
MOV BL,7
ADD AX,SUM      → AX=FEFFH  CF=1
ADC BL,5        → BL=7+5+1=0DH  CF=0
INC AL          → AL=00  CF unchanged
ADC BL,3        → BL=0DH+3+0=10H  CF=0
```

AL=FF

INC AL 1111 1111

CF: unchanged 0000 0001

CARRY =1 0000 0000

Ex:

MOV AX,0FFFFH

INC AL → AL=00 CF unchanged

XCHG AH,AL → AX=00FFH

INC AX → AX=0100

AH	AL
01	00

1111 1111
0000 0001

Carry =1 0000 0000

EX: Write a program to add three numbers

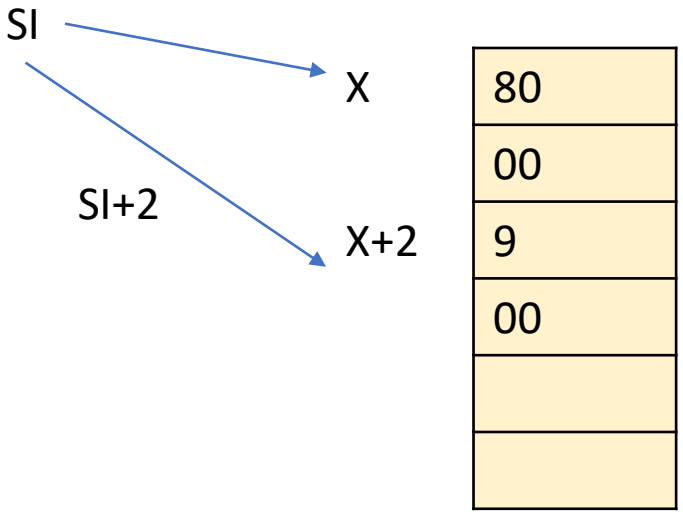
```
X DB 20H,16,9
SUM DB  ?
MOV AH,X
ADD AH,X+1      →   AH= 20H+10H= 30H   CF=0
ADC AH,X+2      →   AH=AH+[X+2]+CF=39H
MOV SUM,AH      →   sum=39H
```

EX: Write a program to add two numbers

```
X DW  80h,9
SUM DW  ?
MOV AX,X
ADD AX, X+2
MOV SUM,AX          sum= 89H
```

➢ Another method

```
MOV SI,OFFSET X
MOV  AX,[SI]
ADD SI,2
ADD AX,[SI]
MOV SUM,AX          sum=89H
```



SUB instruction:

- Subtraction in 8086 microprocessor done using Two's Complement.
- Subtract binary values.
- The general format for SUB is:

SUB destination, source

- **Destination=destination - source**

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

SUB AX,BX

AX = AX - BX

- After subtraction the flags value will be:
- If dest > src SF=0, ZF=0, CF=0
- If dest < src SF=1, ZF=0, CF=1

SBB instruction:

- Subtract with borrow.
- The general format for SBB is:

SBB destination, source

Destination=destination - source - CF

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

- Performs the same operation as ADC but with subtraction.

SBB AX,BX

AX = AX – BX-CF

DEC instruction:

- Decrement by 1.
- The same as SUB dest,1.
- The general format for DEC is:

DEC destination

Destination=destination - 1

Destination: represents register or memory.

➢does not effect on the carry flags

DEC AX

AX=AX-1

EX:

MOV AL,3

SUB AL,6 → AL=FDH CF=1 AF=1 SF=1 ZF=0 PF=0 OF=0

0000 0011

0000 0110

1111 1101 → second complement -3

- STC → Set Carry Flag →CF=1
- CLC →Clear Carry Flag →CF=0

STC → CF=1

MOV AL,7

SBB AL,3 → AL=7-3-1

MOV AL,3

SUB AL,6 → AL=-3= FDH CF=1

MOV AL,7

SBB AL,6 → AL=7-6-1=0 CF=0

DEC AL → AL=FFH CF unchanged

if use SUB AL,1 → CF changed → AL=FFH CF=1

MOV AL,-128 → 128=-128=80H

SUB AL,2 → AL=7EH CF=0 SF=0 OF=1 PF =1 AF=1

```
   0000 0000
 - 0000 0001
 ---------------
carry =1  1111 1111
```

```
   1000 0000
 -  0000 0010
 ---------------
carry =0  0111 1110
```

MUL instruction:

- Unsigned multiplying.
- The general format for MUL is:

MUL source

Where:

Source: represents register or memory.

- The instruction multiply the source with AL register content and store the result of multiplication in AX.

MUL BL (AX ← BL*AL)

MUL BX (DX. AX ← AX*BX)

➢Note: IMUL the same as MUL but used for signed numbers.

- If the result size greater than the operand size OF=CF=1 otherwise OF=CF =0 , other flags unchanged

EX: MOV AL,200

MOV BL,4

MUL BL AX=800=320H CF=OF=1

EX: MOV AL,-2

MOV BL,-4

IMUL BL AX= AL*BL= -2*-4=8 CF=OF=0

MOV AL,-2

MUL BL AX= AL*BL=FEH*FCH=254*252= FA08H=64008 CF=OF=1

DIV instruction:

- Unsigned divide.
- The general format for DIV is:

DIV source

Where:

Source: represents register or memory.

- The instruction divide AX register content by the source and store the result of division in AX.

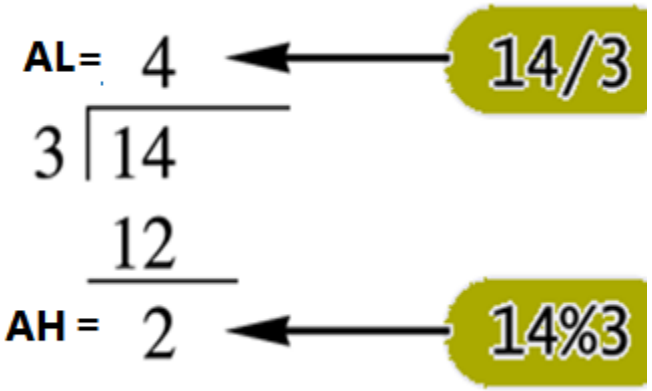
DIV BL (AX ← AX/BL), where AH contain the reminder, AL contain result

DIV BX (DX,AX ← DX,AX/BX), where DX contain the reminder, AX contain result

Note: IDIV the same as DIV but used for signed numbers.

- If use IDIV instruction :

+/+	result +	reminder +
+/-	result -	reminder +
-/+	result-	reminder -
-/-	result +	reminder -



EX: MOV AX,203

MOV BL,4

DIV BL →AX/BL → AH: reminder AL: result

AL=203/4 = 50=32H AH=AX%BL=203%4=3 AX=0332H

EX:MOV AH,0

MOV AL,-8 →AL=F8H=248

MOV BL,2

DIV BL → AL=248/2=124=7CH AH=248%2=0

EX: MOV AH,0

MOV AL,-8

MOV BL,3

IDIV BL → AL=-8/3=-2=FEH AH=-8%3=-2=FEH AX=FEFEH

EX: MOV AL,8

MOV BL,5

MUL BL → AX=8*5=40=0028H

SUB AX,2 →AX=0026H=38

MOV BL,6

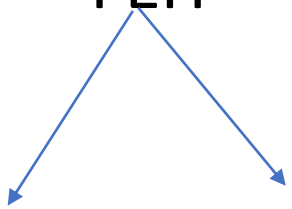
DIV BL → AL=38/6=6 AH=38%6=1

- MOV AL,FEH → MOV AL,-2

- MOV AX,-2 → MOV AX,FFFEH
- MOV AX,FEH → MOV AX,00FEH → MOV AX,254

- if register:8 bit

FEH



-2 254

if register:16bit

FEH → 00FEH

254

x dd 22334455h

mov ax,x → ax=4455h →special case

x dd 22334455h

mov AL,x → error

x dw 4455h

mov AL,x → error

x db 4455h

mov AX,x → error

CMP instruction:

- Compare between source and destination.
- The general format for CMP is:

CMP destination, source

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

- Perform the operation (dest – src) but without saving the value of subtraction. The main aim of this operation is to effects on flag register (CF, SF, ZF, OF).
- CMP instruction used in make decision.

CMP X,5

Jb L

Equivalent to:

If X<5 goto L

Microprocessor System

Bit Manipulation instructions

- **SHR instruction:**

- Perform shifting to right.
- Each bit shifted off enters the carry flag.

The general format for SHR is:

SHR destination, shifting value

Where:

Destination: represents register or memory.

Shifting value (immediate or CL): immediate must equal 1, if the shifting value grater than one we must use CL register.

Example:

MOV AL, 01100001b

SHR AL, 01 ; AL = 00110000b

MOV CL,03

SHR AL,CL ; AL = 00000110b



- **SAR instruction:**



- Perform Arithmetic shifting to right.
- Each bit shifted off enters the carry flag.
- SAR differs from SHR in one important way: SAR uses the sign flag bit to fill leftmost vacated bits.

The general format for SAR is:

SAR destination, shifting value

Where:

Destination :represents register or memory.

Shifting value (immediate or CL): immediate must equal 1, if the shifting value grater than one we must use CL register.

Example:

MOV BL, 10110111b = -73

SAR BL, 01 ; BL = 11011011b = -37

MOV CL,02

SAR BL,CL ; BL = 11110110b = -10

- **SHL instruction:**

- Perform shifting to left.
- Each bit shifted off enters the carry flag



The general format for SHL is:

SHL destination, shifting value

Where:

Destination: represents register or memory.

Shifting value (immediate or CL): immediate must equal 1, if the shifting value grater than one we must use CL register.

Example:

```
MOV DL, 01100001b
SHL DL, 01 ; DL = 11000010b
MOV CL,03
SHL DL,CL ; DL = 00010000b
```

- **SAL instruction:**

- Perform arithmetic shifting to left.

- Each bit shifted off enters the carry flag.

- Left shift always fill 0-bits to the right. As a result, SHL and SAL are identical.



- **ROL instruction:**

- Perform rotating to left.
- ROL rotate the leftmost bit into the rightmost vacated position and into the carry flag.
- Each bit rotated enters the carry flag.

The general format for ROL is:

ROL destination, rotating value

Where:

Destination: represents register or memory.

Rotating value (immediate or CL): immediate must equal 1, if the rotating value grater than one we must use CL register.

Example:

```
MOV AL, 11100001b
ROL AL, 01 ; AL = 11000011b
MOV CL,03
ROL AL,CL ; AL = 00011110b
```



- **RCL instruction:**

- Perform rotating to left including the carry flag bit.
- RCL differs from ROL in this way: each bit rotated off on the left moves into the CF and the CF bit moves into the vacated bit position on the right.

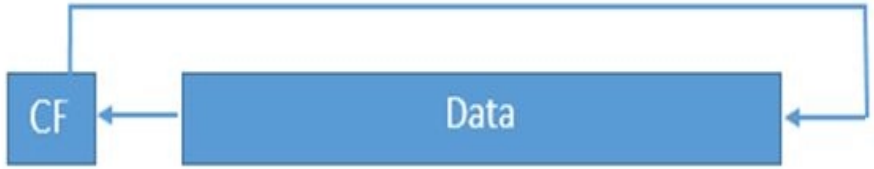
The general format for RCL is:

RCL destination, rotating value

Where:

Destination: represents register or memory.

Rotating value (immediate or CL): immediate must equal 1, if the rotating value grater than one we must use CL register.



Example: *assume initial CF=1*

```
MOV AL, 11100001b
RCL AL, 01 ; AL = 11000011b
MOV CL,03
RCL AL,CL ; AL = 00011111b
```

- **ROR instruction:**

- Perform rotating to right.

- ROR rotate the rightmost bit into the leftmost vacated position and into the carry flag.

- Each bit rotated enters the CF



The general format for ROR is:

ROR destination, rotating value

Where:

Destination: represents register or memory.

Rotating value (immediate or CL): immediate must equal 1, if the rotating value grater than one we must use CL register.

- Registers and memory can be 8-bit or 16-bit length.

Example:

```
MOV AL, 11100001b
ROR AL, 01 ; AL = 11110000b
MOV CL,02
ROR AL,CL ; AL = 00111100b
```

- **RCR instruction:**

- Perform rotating to right including the carry flag bit.

- RCR differs from ROR in this way: each bit rotated off on the right moves into the CF and the CF bit moves into the vacated bit position on the left

The general format for RCR is:

RCR destination, rotating value

Where:

Destination: represents register or memory.

Rotating value (immediate or CL): immediate must equal 1, if the rotating value grater than one we must use CL register.



Example: *assume initial CF=1*

```
MOV AL, 11100001b
RCR AL, 01 ; AL = 11110000b
MOV CL,03
RCR AL,CL ; AL = 00111110b
```

```
MOV AX,0
MOV BL,35H
INC AH
MOV CL,2
L1: ROR BL,CL
JC L2
TEST  BL,0CAH
JZ END
JMP L1
L2: INC AH
INC AL
JMP L1
END:
```

AX	BL	CL	CF	ZF
0100h	35H	2	0	0
0100h	4DH	2	0	0
0100h	53H	2	0	0
0201h	D4H	2	1	0
0201h	35H	2	0	1

- SHL and SHR are used to perform multiplication and division operations respectively
- EXAMPLE: MULTIPLICATION

MOV AL,4 → AL=0000 0100

SHL AL,1 → AL= 0000 1000 → 4*2=8

- EXAMPLE: DIVISION

MOV AL,4 → AL=0000 0100

SHR AL,1 → AL= 0000 0010 → 4/2=2

- EXAMPLE:

MOV CL,2

MOV AL,4 → AL=0000 0100

SHL AL,CL → AL= 0001 0000 → 4*4=16

- EXAMPLE:

Find the result of DX without using mul and div instructions

DX= 3*AX+7*BX

DX= 2+AX+AX + 8*BX-BX

- SOLUTION:

MOV SI,AX

SHL SI,1 → SI*2

ADD SI,AX → SI*2+SI → 3*AX

MOV CL,3

MOV DX,BX

SHL DX,CL → DX*8

SUB DX,BX → DX*8-DX → 7*DX

ADD DX,SI

Microprocessor System

Control transfer instructions:

Control transfer instructions:

There are two types of control transfer instruction:

1- Unconditional transfer instruction (e.g. JMP, CALL).

2- Conditional transfer instruction (e.g. JZ, JA).

CONDITIONAL_TRANSFER_INSTRUCTION LABEL

Unconditional Transfer Instruction

- In 8086 assembly language, the JMP and CALL instructions performs unconditional jump.

- The JMP (jump) instruction unconditionally transfers control to another point in the program. The syntax is:

JMP target

EX:

```
MOV AL, 1
ADD AL, 2
JMP L1                  ; jump to instruction labeled by L1
ADD AL, 3               ; skip this instruction
L1: ADD AL, 1           ; the final value of AL is 4
```

Conditional Transfer Instruction

- The conditional transfer instructions are the basic tool for creating loops and other conditionally executable statements like the if..then statement.

- The conditional transfer instructions are divided according to the condition type to:
  - Flags Condition Instructions
  - Relational Condition Instructions
    - **Signed Relational**
    - **Unsigned Relational**

Flags Condition Instruction

- The flags conditional jumps test one flag in the flag register to see if they match some particular value. If the value matches, control transfers to the target location. If the match fails, the CPU ignores the conditional jump and execution continues with the next instruction.
- The flags could be sets by the following instructions: CMP, ADD, SUB, TEST, AND, OR, XOR ….. etc.

- The syntax is:

$J_{condition}$ Target ; if the condition is true, then jump to the target

The following table list the possible flag conditions

Instruction	Description	Condition
JC	Jump if carry	CF = 1
JNC	Jump if not carry	CF = 0
JZ	Jump if zero	ZF =1
JNZ	Jump if not zero	ZF = 0
JS	Jump if sign (negative)	SF = 1
JNS	Jump if not sign (positive)	SF = 0
JO	Jump if overflow	OF = 1
JNO	Jump if not overflow	OF = 0
JP	Jump if parity (even)	PF = 1
JNP	Jump if not parity (odd)	PF = 0

- Example: Write an assembly code to find the summation of the three array elements (X DB 1, 2, 3)

```
X DB 1, 2, 3
MOV CL, 3
MOV AL, 0
MOV SI, 0
L1: ADD AL, X[SI]
INC SI
DEC CL
JNZ L1
```

CL	AL	SI	ZF
3	0	0	0
2	1	1	0
1	3	2	0
0	6	3	1

Note that the JNZ instruction depend on the last instruction affected the flags, which is DEC CL. While the CL value not equal zero, the condition in jump instruction is true (ZF = 0).

- Example: Convert the following C++ codes to equivalent assembly code.

1. C++ Code:

```
int x=10, y=0;
if( x!=10 ){
        x++;
        y=x;
}
y++;
```

Assembly Code

```
ORG 100H
MOV AL, X
MOV BL, Y
CMP AL, 10
JZ L1
INC AL
MOV BL, AL
L1: INC BL
MOV X, AL
MOV Y, BL
HLT
X DB 10
Y DB 0
```

2. C++ Code:

```
int x=5, y=4;
if (x == y)
        y = x+y;
else
        y = x-y;
```

- Assembly Code:

```
ORG 100H
MOV AL, X
MOV BL, Y
CMP AL, BL
JZ L
SUB AL, BL
MOV BL, AL
JMP EXIT
L: ADD BL, AL
EXIT: MOV Y, BL
HLT
X DB 5
Y DB 4
```

```
MOV AL,0FEH
MOV BL,3
 L1: ADD AL,1
JNC L1
L2:DEC BL
JNS L2
L3: XCHG AL,BL
```

AL	BL	CF	SF	ZF
FE	3	0	0	0
FF	3	0	1	0
00	2	1	0	1
00	1	1	0	0
00	0	1	0	1
00	FF	1	1	0
FF	00	1	1	0

Relational Condition Instruction

- Usually, the programmer use relational operator (>, >=, <, <=, ==, !=) to compare the values.

- signed and unsigned numbers are used in programming, and the order of these numbers is different, there are two sets of relational conditional jump instructions for magnitude comparisons.

- compare the number FFFEH with the number 0001H
- unsigned numbers

FFFE > 0001H → 65534>1

- signed numbers

FFFEH < 0001 H → -2<1

- FFFEH is considered as negative number =-2

0001 considered as positive number=1

- When signed numbers are compared, the terms greater than and less than refer to signed numbers. And when unsigned numbers are compared, the terms above and below refer to unsigned numbers.

Signed Relational Instructions

Instruction	Description	Flag(s)
JG	Jump if greater than (if opr1 > opr2)	SF = OF and ZF = 0
JGE	Jump if greater than or equal (if opr1 >= opr2)	SF = OF
JNL	Jump if not less than (if opr1 not < opr2)	SF = OF
JL	Jump if less than (if opr1 < opr2)	SF <> OF
JLE	Jump if less than or equal (if opr1 <= opr2)	ZF = 1 or SF <> OF
JE	Jump if equal (if opr1 = opr2)	ZF = 1
JNG	Jump if not greater than (if opr1 not > opr2)	ZF = 1 or SF <> OF
JNE	Jump if not equal (if opr1 <> opr2)	ZF = 0

Unsigned Relational Instructions

Instruction	Description	Flag(s)
JA	Jump if above (if opr1 > opr2)	CF = 0 and ZF = 0
JNBE	Jump if not below or equal (if opr1 not <= opr2)	CF = 0 and ZF = 0
JAE	Jump if above or equal (if opr1 >= opr2)	CF = 0
JNB	Jump if not below (if opr1 not < opr2)	CF = 0
JB	Jump if below (if opr1 < opr2)	CF = 1
JNAE	Jump if not above or equal (if opr1 not >= opr2)	CF = 1
JBE	Jump if below or equal (if opr1 <= opr2)	ZF = 1 or CF = 1
JNA	Jump if not above (if opr1 not > opr2)	ZF = 1 or CF = 1

The equality operators (== and !=) are the same for signed and unsigned numbers, which are JE (jump if equal) and JNE (jump if not equal). Notice that JE has an alternative opcode JZ and JNE has an alternative opcode JNZ.

Example: what is the differences between the following two codes?

1.

```
MOV CL, 1
MOV AL, 7FH
CMP AL, 80H
JA L1
INC CL
L1: INC CL
```

2.

```
MOV CL, 1
MOV AL, 7FH
CMP AL, 80H
JG L1
INC CL
L1: INC CL
```

- MOV SI,OFFSET X → LEA SI,X
- MOV AL,X → MOV AL,[SI]
- MOV AL,X+1 → MOV AL,[SI+1] →

```
INC SI
MOV AL,[SI]
```

X

1
2
3
6
7
8

*same size is not important

MOV AX,[SI] IS TRUE AX=0201

MOV SI,0

MOV AL,X[SI]

INC SI

MOV AL,X[SI]

*same size is important

MOV AX,[SI] ERROR

- **Example:** Convert the C++ code (compound expression) to equivalent assembly code

C++ Code:

```
if (AL > BL && BL > CL)
x=1;
```

Assembly Code:

```
CMP AL, BL
JLE NEXT      ; exit if false
CMP BL, CL
JLE NEXT      ; exit if false
MOV X, 1
NEXT:
```

- **Example:** Convert the C++ code (compound expression) to equivalent assembly code

```
for (i=1;i<=4;i++)
{s=s+i
y=s+2}
```

ANSWER: ASSUME AL=S AND BL=Y

```
S DB 0
Y DB 0
MOV AL,S
MOV BL,Y
MOV CX,1
L:
ADD AL,CL
MOV BL,AL
ADD BL,2
INC CX
CMP CX,4
JBE L
MOV S,AL
MOV Y,BL
```

CX	AL	BL	
1	0	0	
2	1	1+2	
3	3	3+2	
4	6	6+2	
5	0A	0A+2	

- **Example:** Convert the C++ code (compound expression) to equivalent assembly code

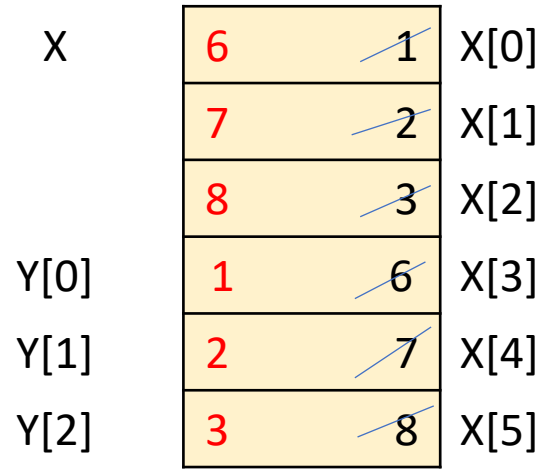
```
while (x<y)
{ x=x+1
y=y-1
}
s=x+y
```

```
X DB 0
Y DB 4
S DB 0
MOV AL,X
MOV BL,Y
MOV CL,S
L:
CMP AL,BL
JB L1
ADD CL,AL
ADD CL,BL
JMP END
L1:
INC AL
DEC BL
JMP L
END:
```

CL	AL	BL	
0	0	4	
0	1	3	
0	2	2	
2+2			

- What is the function of the following codes:

```
X DB 1,2,3
Y DB 6,7,8
MOV SI,0
L1: MOV AL,X[SI]
XCHG AL,Y[SI]
MOV X[SI],AL
INC SI
CMP SI,3
JB L1
```



JCXZ Instruction

- The JCXZ (jump if CX is zero) instruction jumps to the target address if CX contains zero. The syntax is:

JCXZ Target

- Many instructions use CX as a counter register, and JCXZ allows you to test and jump to see if CX has become 0. If CX is any other value than 0, execution falls through to the next instruction.

EX:

```
MOV CX,3
MOV AL,1
L1: ADD AL,3
DEC CX
JCXZ END
JMP L1
END:
```

CX	AL	
3	1	
2	4	
1	7	
0	0A	

Microprocessor system

Data Definition &

Addressing mode

Classification of programming languages:

1- High level languages. (programming using language similar to human language) (C++)

2- Mid level languages. (e.g. Basic)

3- Low level languages. (e.g. assembly)

An assembly program has two types of sentence:

- **Instructions**: such as MOV and ADD, which are operations that the computer can execute.

- **Directives**: such as END or SEGMENT, which used to provide information to the assembler.

Instruction format:

Label : op. code operands ; comments.

- **Label** : refers to the address of an instruction, procedure, or segment.
- **Op. code**: the required operation (e.g. MOV).
- **Operand**: used in the expression to perform operation using it (e.g. constants, register name, …). Operands can be registers or memory locations.
- **Comment**: begins with a semicolon (;) and used to improve clarity.

- Labels and comments are selective and can be ignored, whereas operand range from 0 to 2.
- Names in assembly language must begin with letter and its maximum length is 31.

Example of instruction:

Add AX, BX ; addition operation

Where:

AX: destination.

BX: source

Directive format:

Name directive operands ; comments

- **Name**: directive name and it selective can be ignored.
- **Directive**: divided into four groups:

1- Data definition.

2- Segment declaration. (start and end)

3- Procedure. (start and end)

4- Linkage operations: link an assembly program with higher level language.
(extern, public)

Data definition directives:

The directives used to define data, along with the names introduced are:

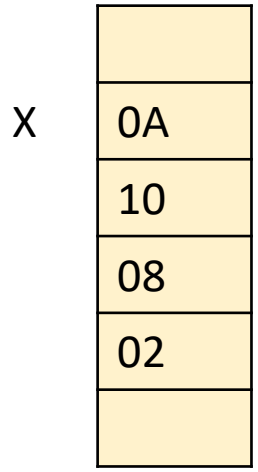
- DB: define byte (8 bit).
- DW define word (16 bit).
- DD: define double word (32 bit).
- DQ: define 4 words (64 bit).
- DT: define ten bytes (80 bit).

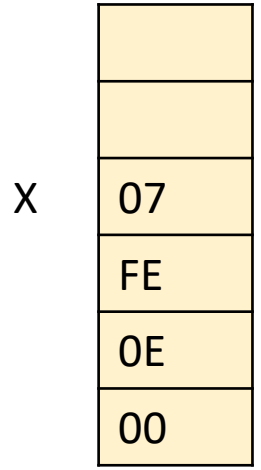
- Data definition performed in data segment.

Data definition directives:

```
X  DB  10
   DB 10H
   DB 10O
   DB 10B
```

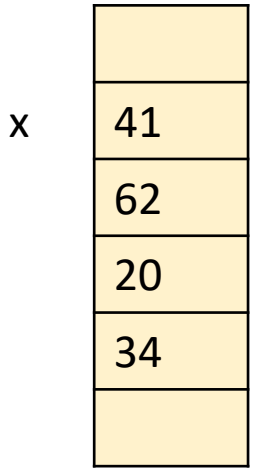
```
X DB  7,-2,14,?
```

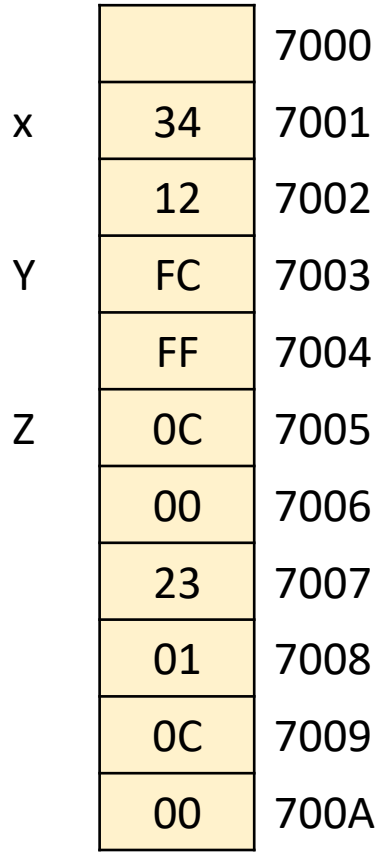




- ASCII code A:41H, a=61H, ‘0’=30h ‘1’=31h

- X DB “Ab 4”
- X DW “Ab 4”





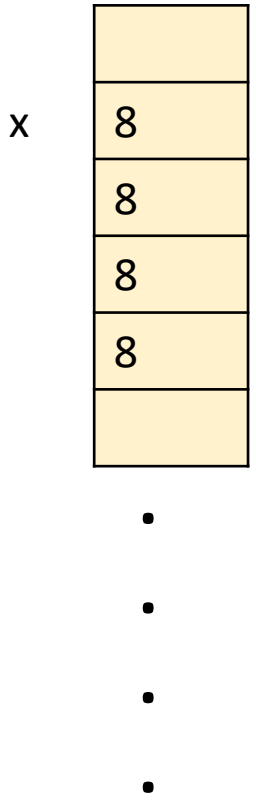
X DW 1234H

Y DW -4

Z DW 12,123H,0CH

- X DB 100 DUP (8)





- X EQU 12H
- Used to define constants that can not be changed.
- This variable does not store in the memory

```
X1  DB ?
X3 DB 32
X4 DB 20H
X5 DB 01011001B
X6  DB 01,’a’
X7 DB ‘32’
X8 DB 2 DUP(0)
YI DW FFF0H
Y2 DW 01011001B
Y4  DW 17
```

X1	00	7000
X3	20	7001
X4	20	7002
X5	59	7003
X6	01	7004
	61	7005
X7	33	7006
	32	7007
X8	00	7008
	00	7009
Y1	F0	700a
	FF	700b
y2	59	700c
	00	700d
y4	11	700e
	00	700f

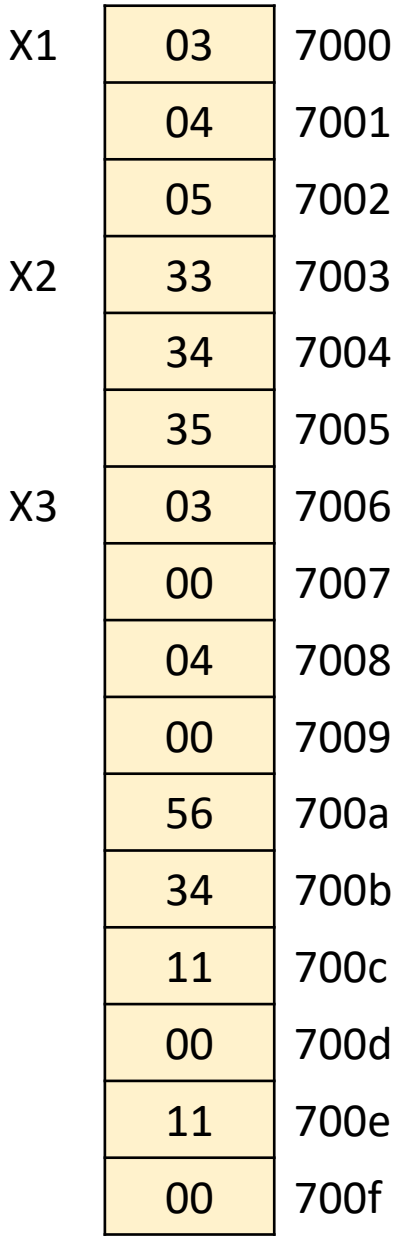
X1 DB 3,4,5

X2 DB ‘345’

X3 DW 3,4

X4 DW 3456H

X5 DW 2 DUP(11h)



Mov instruction

MOV destination, source

Destination=source

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

- Registers and memory can be 8-bit or 16-bit length.

MOV R,R	MOV AL,BL
MOV R,M	MOV AL,X
MOV M,R	MOV X,AL
MOV M,DATA	MOV X,5
MOV R,DATA	MOV AL,6

Wrong operations:

MOV M,M (can not move memory location to other memory location directly).
MOV DS,ES (can not move segment register content to other segment register directly).

X DB 22H
MOV BX,AL (can not move registers or memory of different sizes).
MOV AX,X

Ex: MOV X,Y solution: → MOV AL,Y
MOV X,AL

Addressing modes

1-Immediate addressing mode

Mov AX, 40H → ah=00 al=40h

2- Register addressing mode:

Mov AX, BX

3- memory addressing mode

A- Direct address mode

Use → **1)** variable name → example X

2) [offset]→ (displacment) → example [100h]

A-Direct address mode:

by default the reference of offset is data segment DS → DS : offset

X DB 22H,44H

Y DW 6611H

MOV AL,X → AL=22H

MOV AH,X+1 → AH=44H

MOV BX,Y →

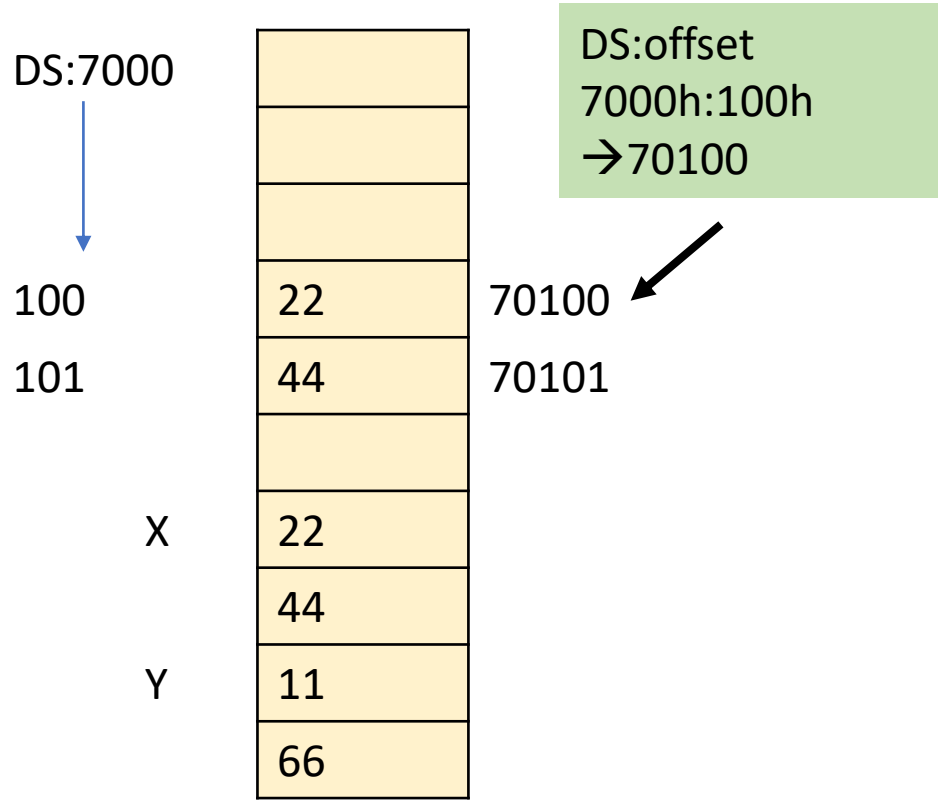
BH	BL
66	11

MOV [100H],AX →

MOV DH,[100H] → DH=22H

MOV CX,[100H] →

CH	CL
44	22



A-Direct address mode:

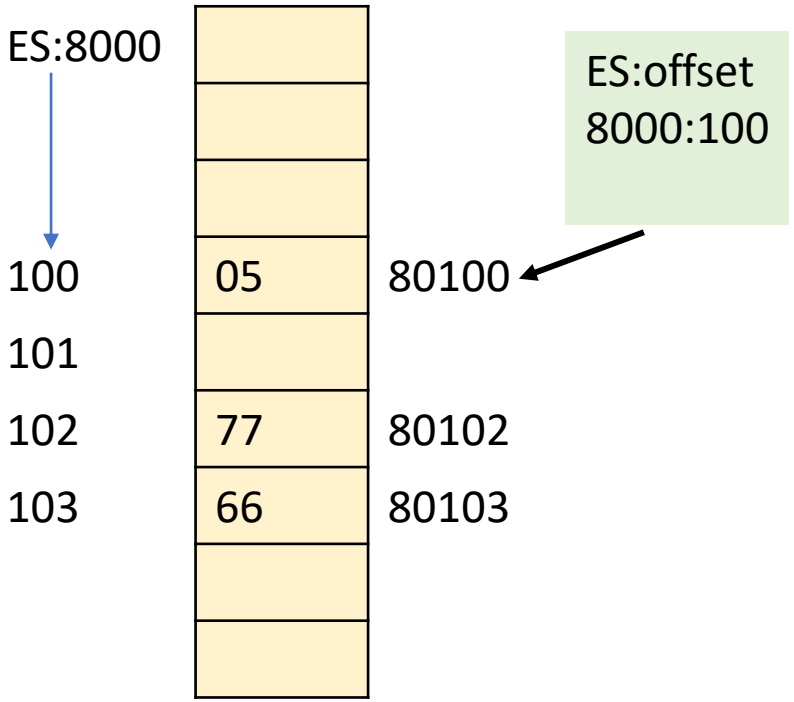
MOV AL,5

MOV BX,6677H

MOV ES:[100H],AL

MOV ES:[102H],BX

MOV CL,ES:[103H] →CL=66



B- register indirect addressing mode

B- register indirect addressing mode

- Give the address in indirect way using one of the addressing registers (BX, SI, DI, BP).

by default the reference of offset is data segment DS expect BP the reference is stack segment SS

1) Indirect addressing mode
2) Indexed addressing mode
3) Base addressing mode
4) Base _ indexed addressing mode

MOV DL,[AX] → ERROR → AX is not addressing register:(SI,DI,BX,BP)

MOV [SI],[BX] → ERROR → (can not move memory location to other memory location directly).

MOV [BL],AL → ERROR →can not use half register in addressing s

MOV [BX],AL → CORRECT

- MOV AX,BX ≠ MOV AX,[BX]

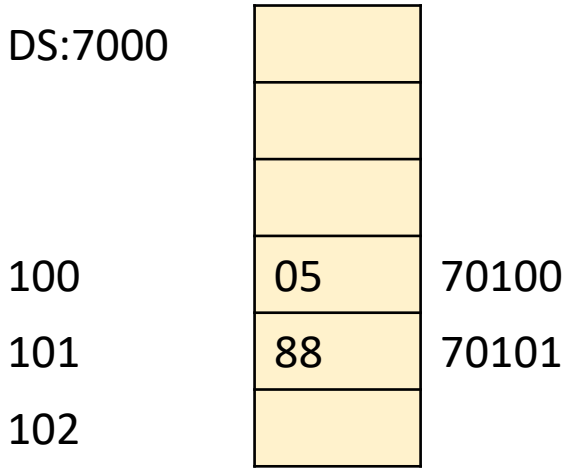
MOV BX,100H

MOV AX,BX → AX=0100H

MOV AX,[BX] → AX= 8805H

MOV AL,[BX] → AL=05

MOV AL,BX → ERROR



Indirect addressing mode

MOV BX,100H

MOV AX,[BX] →

AH	AL
88	05

MOV DH,[BX] → DH=05

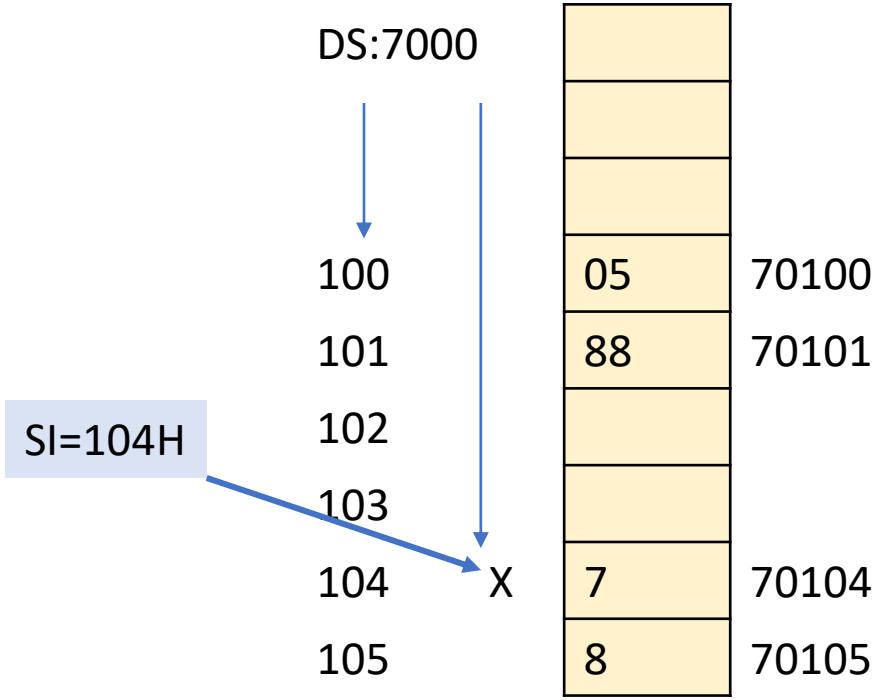
X DB 7,8

MOV SI, OFFSET X

MOV CX,[SI] →

CH	CL
08	07

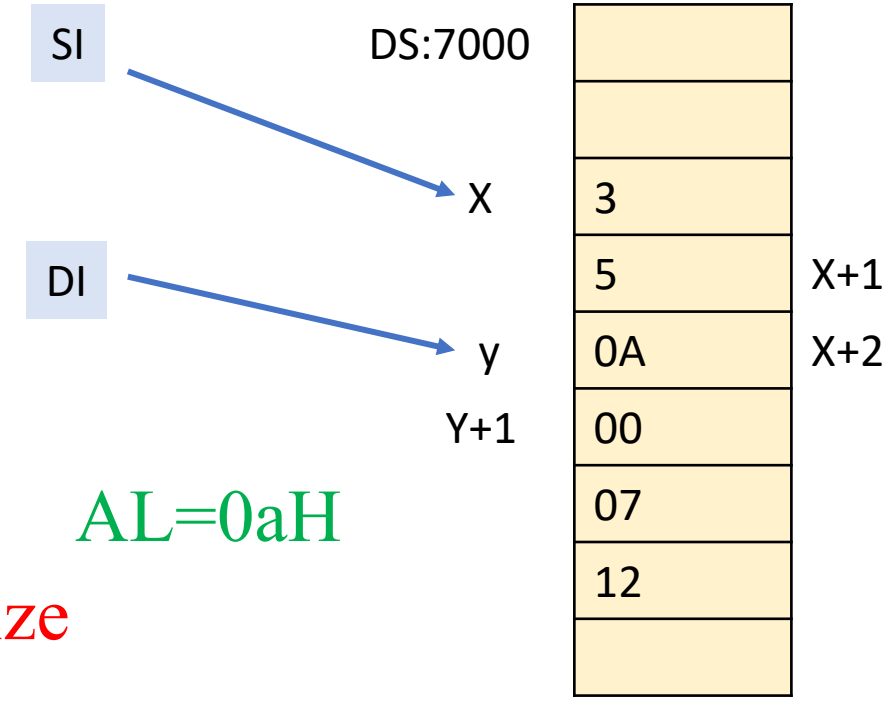
MOV DL,[SI] → DL=07



Indexed addressing mode

- Use index registers (SI, DI) and value (displacment)

X DB 3,5
Y DW 10, 1207H



MOV DI, OFFSET Y
MOV SI, 2
MOV AL, X[SI] → DS:X+SI → AL=0aH
MOV AX,X[SI] → error differnt size

CH	CL
07	00

MOV CX,[DI+1]
MOV DL,[DI+1] →DL=00

Base addressing mode

- Use index registers (BX, BP) and value(displacment)

X DB 3,5

Y DW 10, 1207H

MOV BX,100H

MOV AL,[BX+1] → AL=0B

MOV CX,2[BX] → DS:2+BX

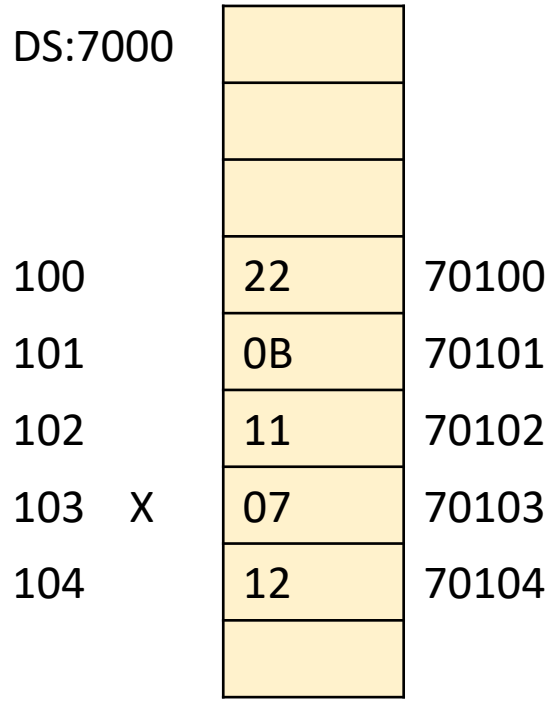
CH	CL
07	11

MOV BX,1

MOV DL,X[BX] → DS:X+BX → DL=12H

MOV BP,3

MOV DL,4[BP] → SS:4+BP



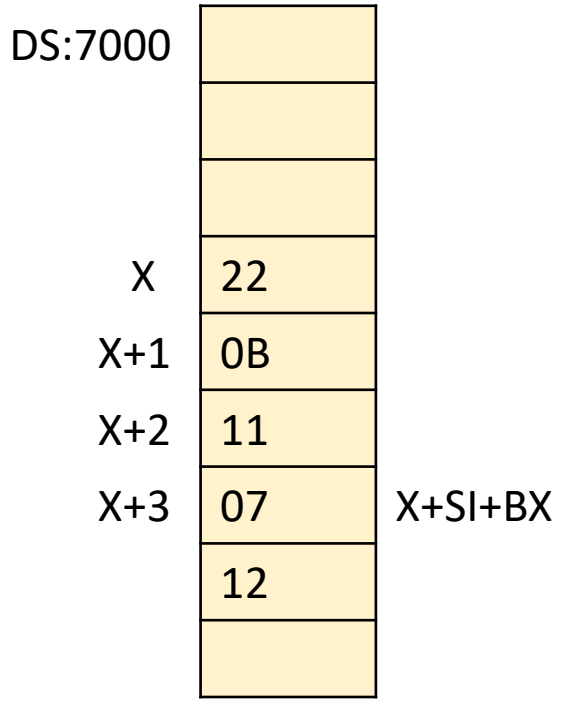
base indexed addressing mode

X DB 3,5
Y DW 10, 1207H

Mov BX,2
MOV SI,1
MOV AL,X[SI][BX] → DS:X+SI+BX → AL=07
MOV AX,X[SI][BX] → ERROR → different size
MOV AL,X[SI][DI] ERROR → can not use the same type of regis addressing in one instruction

---

MOV AX,[SI][BX] → DS:SI+BX
MOV AX,[SI+3]4[BX] → DS: SI+BX+3+4
MOV AX,5[BX][SI] → DS:SI+BX+5



addressing mode summary

	Index	Base	Offset
CS			
DS	SI	BX	VALUE
ES	DI	BP	VARIABLE
SS			

Indirect

Direct

- BP the reference is stack segment

Microprocessor System

Data transfer instructions

8086 Instructions:

Instructions are operations that the microprocessor can execute. 8086 instructions can be divided into seven categories.

1- Data transfer instructions.
2- Arithmetic and logic instructions.
3- Bit manipulation instructions.
4- Control transfer instructions. (program sequences and decisions (if then))
5- String instructions.
6- Interrupt instructions.
7- Microprocessor control instructions.

Data transfer instructions:

a- Data moving instructions:

MOV

XCHG

b- Stack instructions:

PUSH POP

c- Input/Output instructions:

IN OUT

d- Address moving instructions:

LEA

e- Flag moving instructions: (use AH register)

LAHF SAHF

➢Do not effect on the flag register

MOV instruction:

MOV destination, source

destination=source

Where:

Destination: represents register or memory.

Source: represents register, memory, or immediate.

➢ Registers and memory can be 8-bit or 16-bit length.

MOV R,R

MOV R,M , MOV M,R

MOV M,Data , MOV R,Data

Wrong operations:

MOV M,M (can not move memory location to other memory location directly).

MOV DS,ES (can not move segment register content to other segment register directly).

MOV BX,AL (can not move registers or memory of different sizes).

MOV instruction: error cases

1) mov segment register, immediate

MOV DS,7000H error

Solution:

MOV AX,7000H

MOV DS,AX

2) mov segment register, segment register

MOV DS,ES error

Solution:

MOV AX,ES

MOV DS,AX

3) MOV 16bit,8bit

MOV AL,BX error

4) Mov memory, memory

MOV [SI],[DI] error

MOV X,Y error

Solution:

```
X DB  5
Y DB  ?
MOV  AL,X
MOV  Y,AL
```

5) error

MOV CS, -------

MOV IP,--------

6) MOV CX,[AL] error can not use AL register for addressing

MOV CX,[SP] error can not use sp register for addressing

Ex:

MOV AX,0B800H

MOV DS,AX

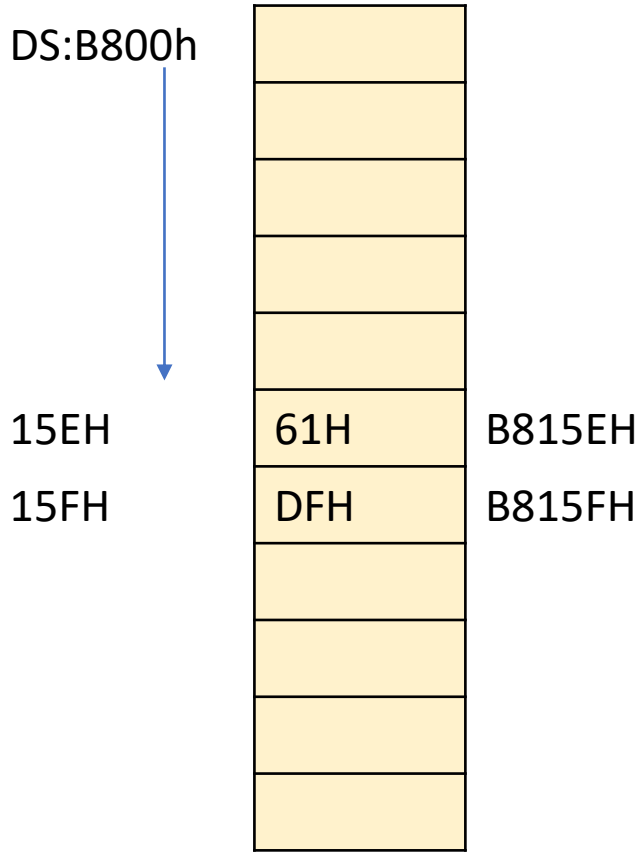
MOV CL, ’a’

MOV DX,’AB’ → DX=4142H

MOV CH,11011111B

MOV BX,15EH

MOV [BX],CX → ds:[15Eh]=cl ds:[15Fh]=ch



output:

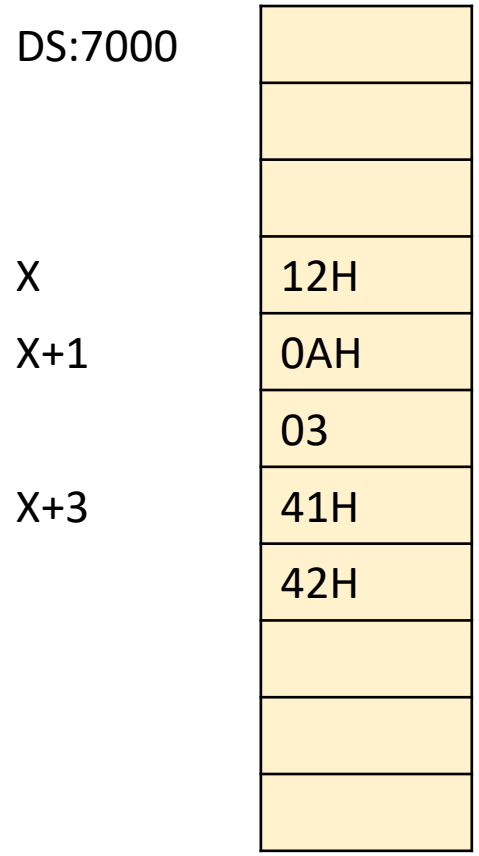
AX=B800 CX=DF61H BX=15EH

EX:

```
X DB 12H,10,3
     DB  'AB'
MOV AL,X
MOV CL,X+1
MOV DL,X[3]
MOV AX,X                    error
```

OUTPUT :

AL=12H CL=0AH DL=41H



LEA instruction:

- Load effective address. Useful for initializing a register with an offset address.
- The general format for LEA is:

LEA Register, variable name

Is equivalent to **MOV register, OFFSET variable name**

Where:

- Registers must 16-bit length. (can use all registers except segment registers)

LEA BX,X

Equivalent to:

MOV BX, offset X

MOV AX, word ptr x = MOV AX,W.X

MOV AL, byte ptr x = MOV AL,B.X

EX:

X DW 1AB2H,34H,4EFH

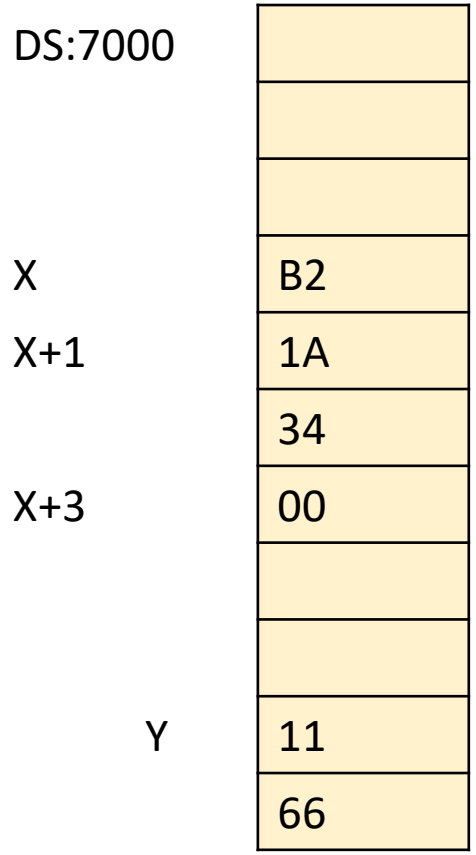
MOV BX,X

MOV DX,X+1

MOV AL,X+3 error → solution MOV AL,B.X+3

output:

BX= 1AB2H DX=341AH AL=00



XCHG instruction:

- Swap the data items.

XCHG destination, source

destination=source and source =destination

Where:

Destination: represents register or memory.

Source: represents register or memory.

Wrong operations:

XCHG AL,3

XCHG AL,BX

XCHG DS,ES

EX:

```
MOV AX,1234H
MOV DX,3FEAH
XCHG AL,DH
```

OUPPUT

AX= 123FH DX=34EAH

EX: write the code to swab two memory locations x,y

```
MOV AL,X
XCHG AL,Y
MOV X,AL
```

Stack instructions

PUSH instruction

Store data in the stack.

- The general format for PUSH is:

PUSH source

Where:

Source: represents register or memory and immediate .

- Registers and memory must be 16-bit length. (length of stack locations are 16-bit) instruction

- Top of the stack defined by the microprocessor at the start to be FFFFH using SP

- Push write to stack
- **PUSH instruction performed at to steps**:

1- SP=SP-2. (in memory each location is 8-bit length so we need to move two times to get the 16 bit for the stack operations)
2- Store into memory location SS[SP].

PUSH instruction

-stack mechanism: Last In First Out (LIFO)

SP=FFF9H

MOV AX,3322H

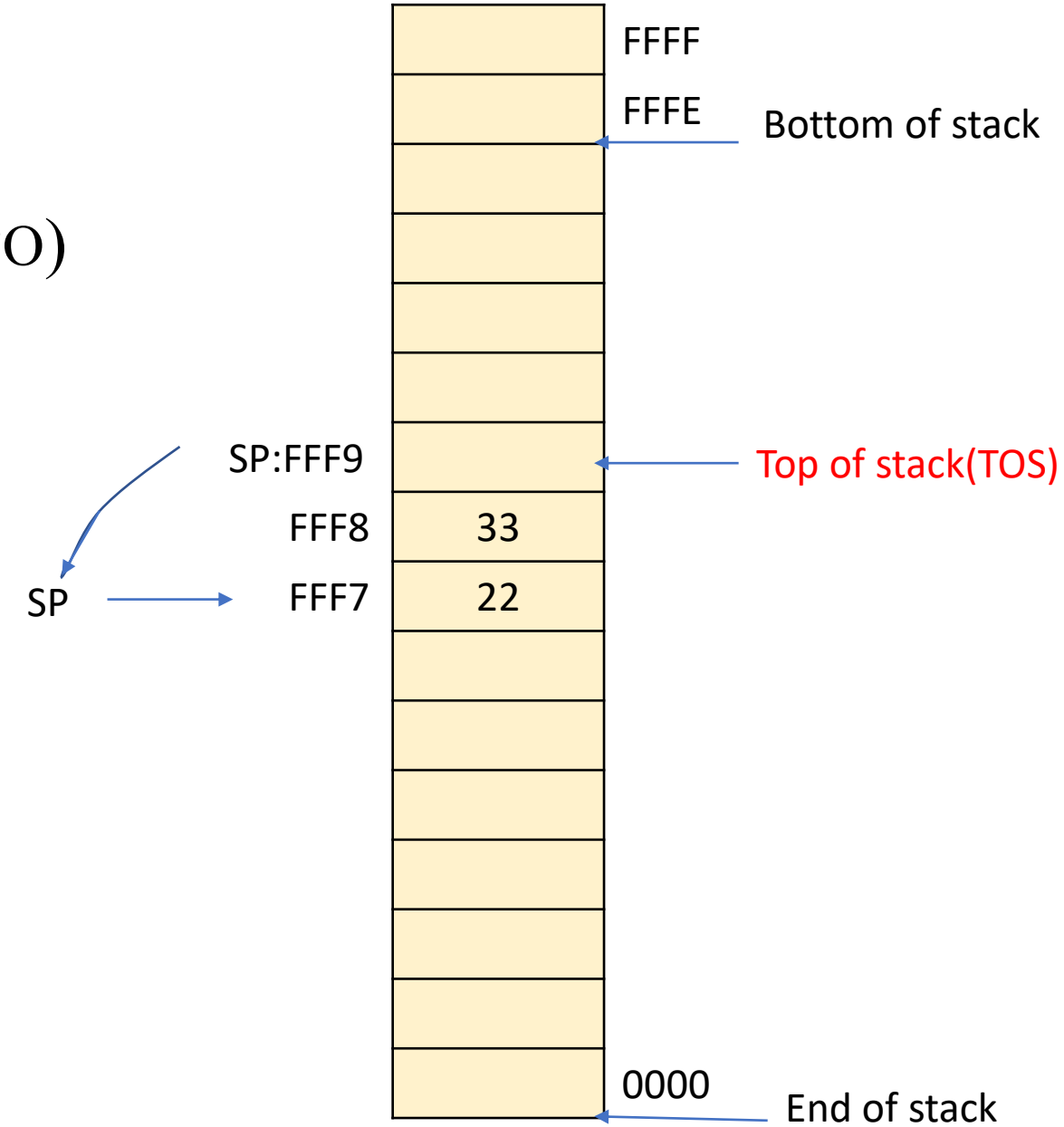
Push AX

SP=SP-2

SP=FFF7H

AL → FFF7H

AH →FFF8H



Stack instructions

POP instruction:

- Retrieve data in the stack.
- The general format for POP is:

POP destination

Where:

Destination: represents register or memory.

- Registers and memory must be16-bit length. (length of stack locations are 16-bit)

- Pop read from stack

- **POP instruction performed at to steps:**

1- Retrieve data from memory location SS[SP].
2- SP=SP+2. (in memory each location is 8-bit length so we need to move two times to get the 16 bit for the stack operations)

POP instruction

SP:FFF7

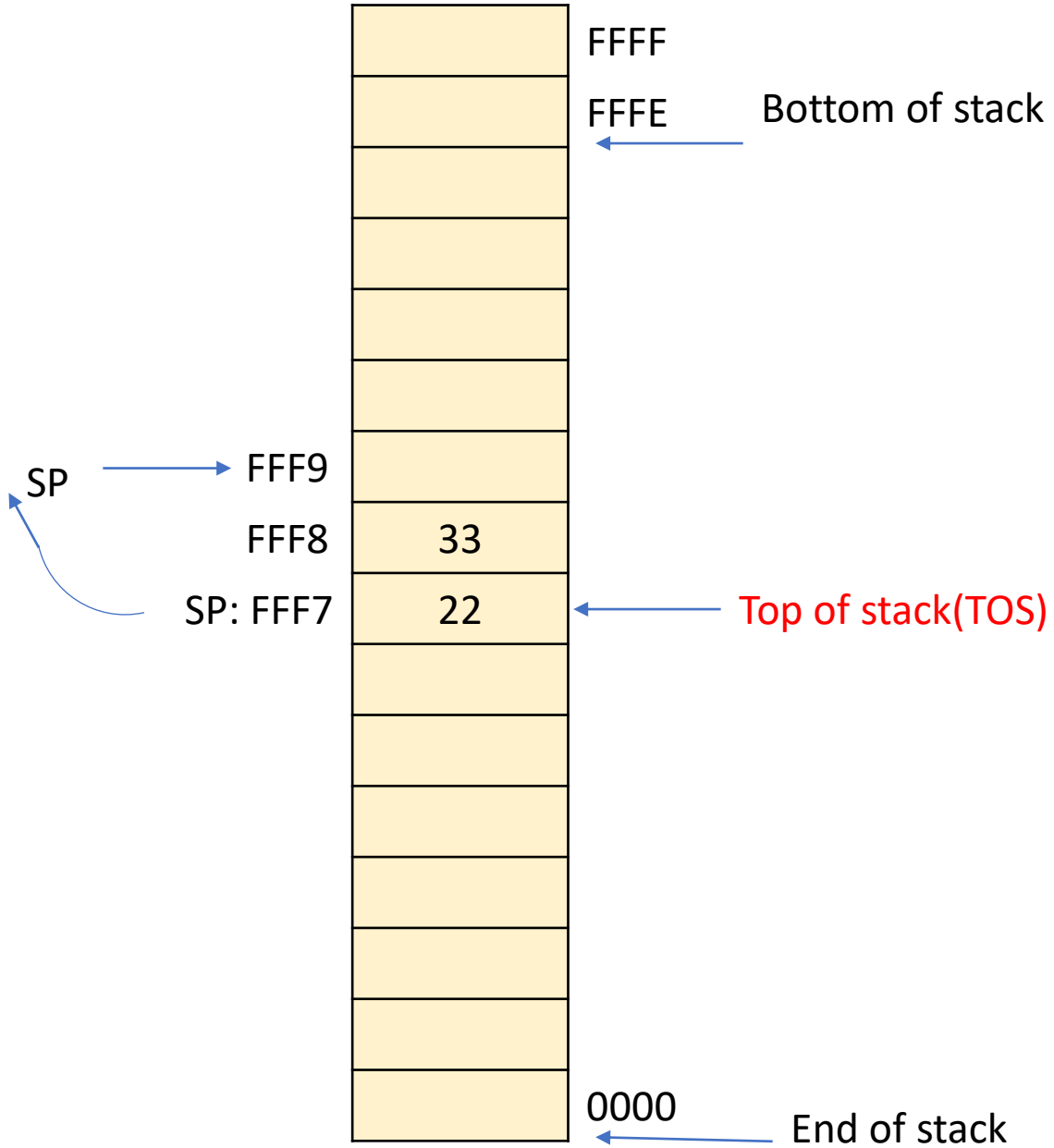
POP AX

[FFF7H] → AL = 22

[FFF8H] → AH = 33

SP=SP+2

SP=FFF9H



Wrong operation :

PUSH AL should be 16 bit

MOV AX,[SP] can not use SP in address only SI,DI,BX and BP

Solution:

MOV BP,SP

MOV AX,[BP] BP reference SS

➢If use SI,DI, BX to reference to SS :

MOV SI,SP

MOV AX,SS:[SI]

EX: SS=0140

1. find the value of SP (Physical address of Top=0145ah)

Physical address of Top=SS+SP

0145ah=01400h+sp

SP=005ah

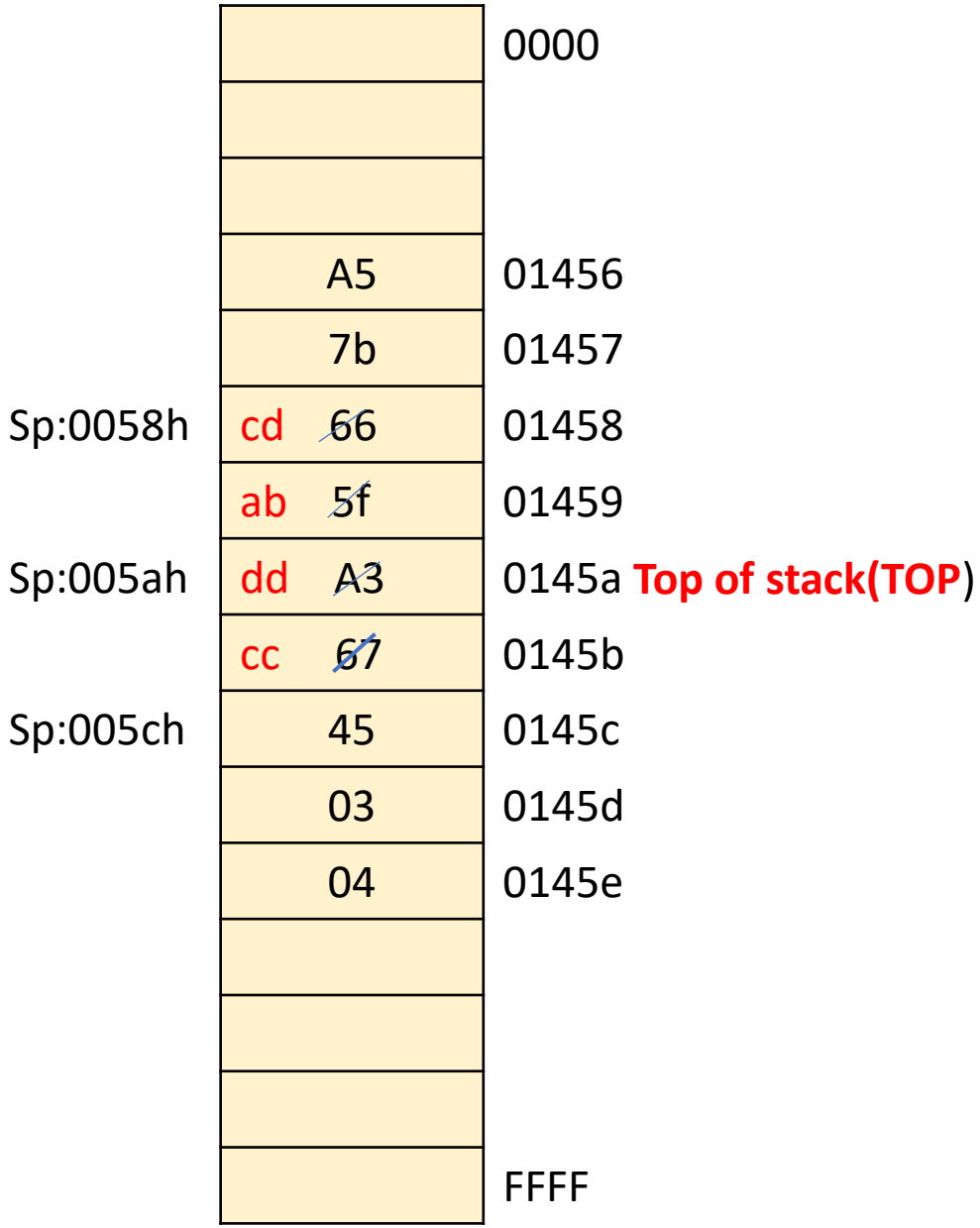
2. POP AX FIND AX, SP

AX= 67A3H → SP=SP+2=005C

3. MOV BX,CCDDH

PUSH BX → SP=SP-2=005AH

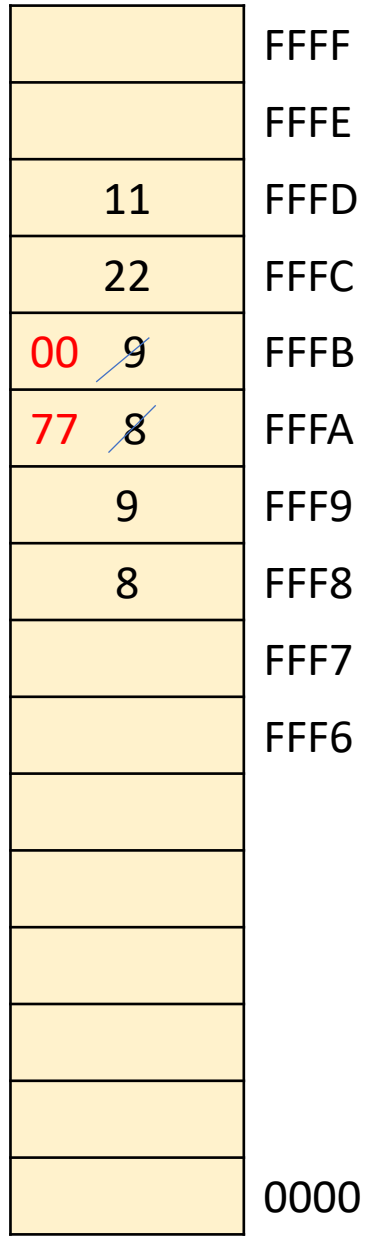
PUSH ABCDH → SP=SP-2=0058H



```
X DB 8,9
MOV SI, OFFSET X

PUSH 1122H
PUSH [SI]
POP AX          AX=0908
PUSH 77H
PUSH AX
POP BX          BX=0908
POP [SI]        X  DB  77,00
```

Top of stack(TOP)



EX: Use stack to swap between ax and bx

Mov ax,1234h

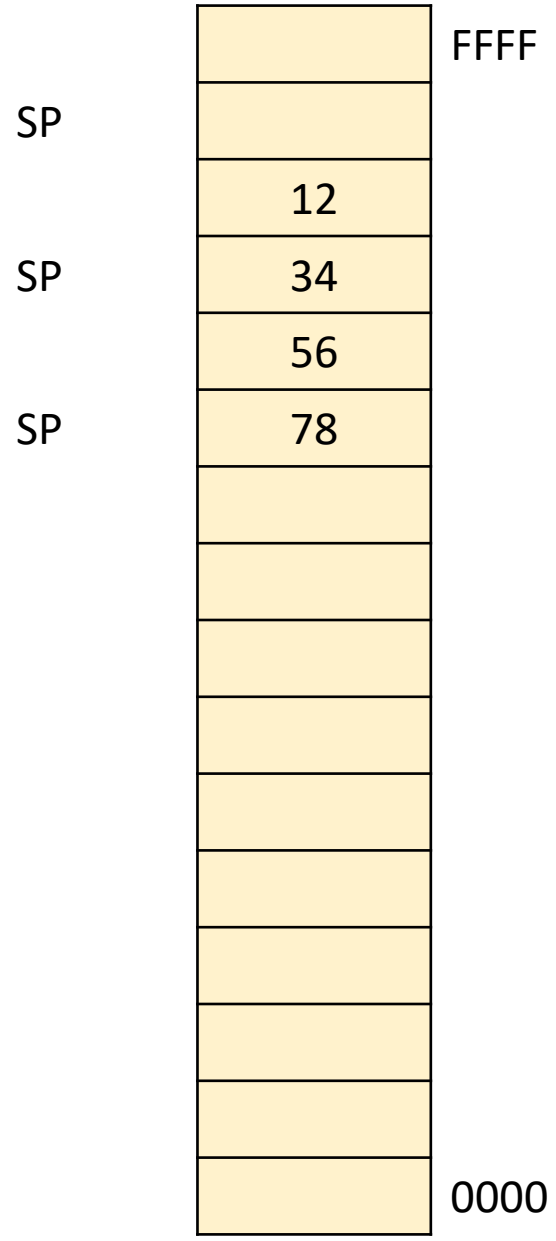
Mov bx,5678h

Push ax

Push bx

Pop ax

Pop bx



Interrupts

Interrupt Instructions:

- Interrupts can be seen as a number of functions.
- These functions make the programming much easier, instead of writing a code to print a character you can simply call the interrupt and it will do everything for you.
- There are also interrupt functions that work with disk drive and other hardware. We call such functions **software interrupts**.
- Interrupts are also triggered by different hardware, these are called **hardware interrupts**.
- *Currently we are interested in software interrupts only.*

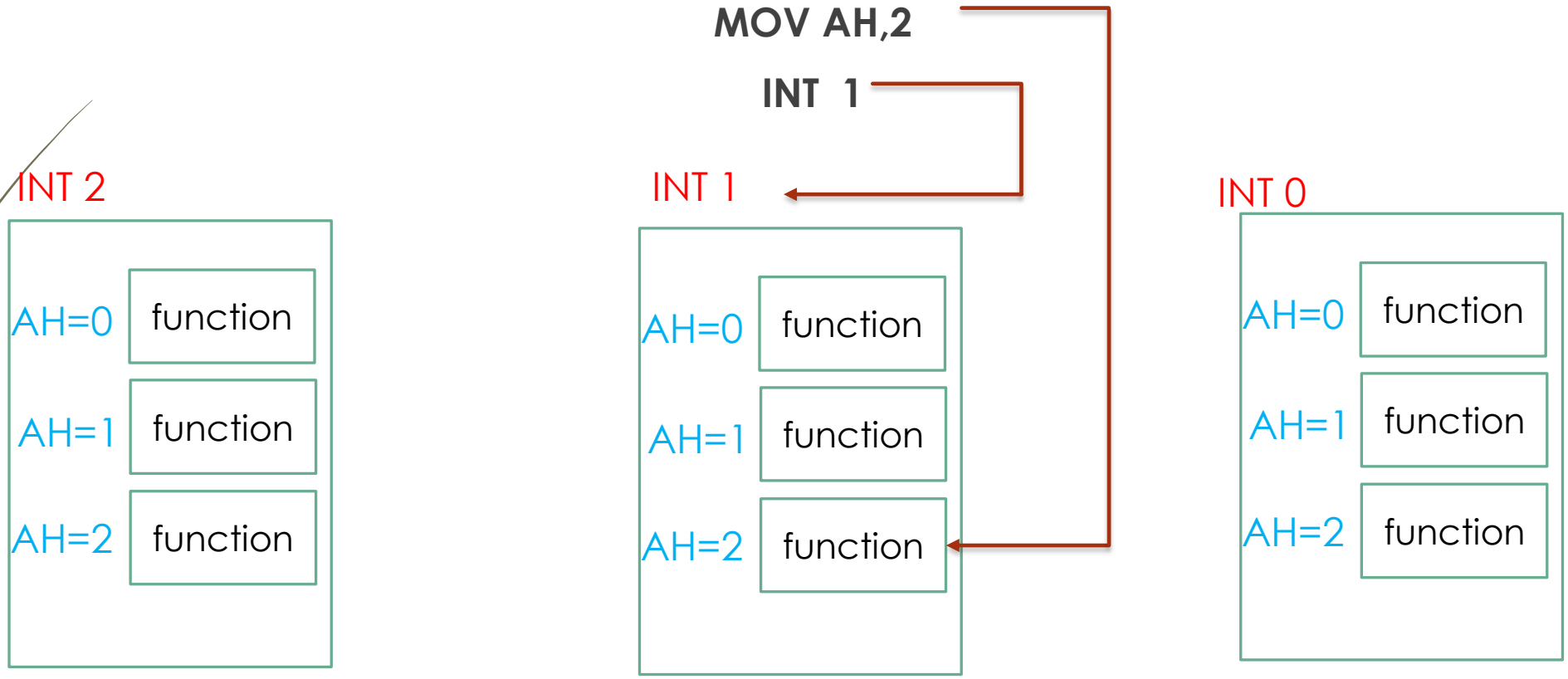
- To make software interrupt there is an **INT** instruction.

- The general format for INT is:

INT **value**

- Where **value** can be a number between 0 to 255 (or 0 to 0FFh).

- AH,VALUE → (0-255)
- INT VALUE → (0-255) → every interrupt can have 256 subfunction
- Function→ **Interrupt service routine**



- interrupt may have **sub-functions**.
- To specify a sub-function **AH register** should be set before calling interrupt.
- Each interrupt may have up to 256 sub-functions (so we get 256 * 256 = 65536 functions).
- Other registers are used to pass parameters and data to sub-function.
- Each interrupt function address is stored in an **interrupt vector.** An interrupt vector occupies 4-bytes of memory location.

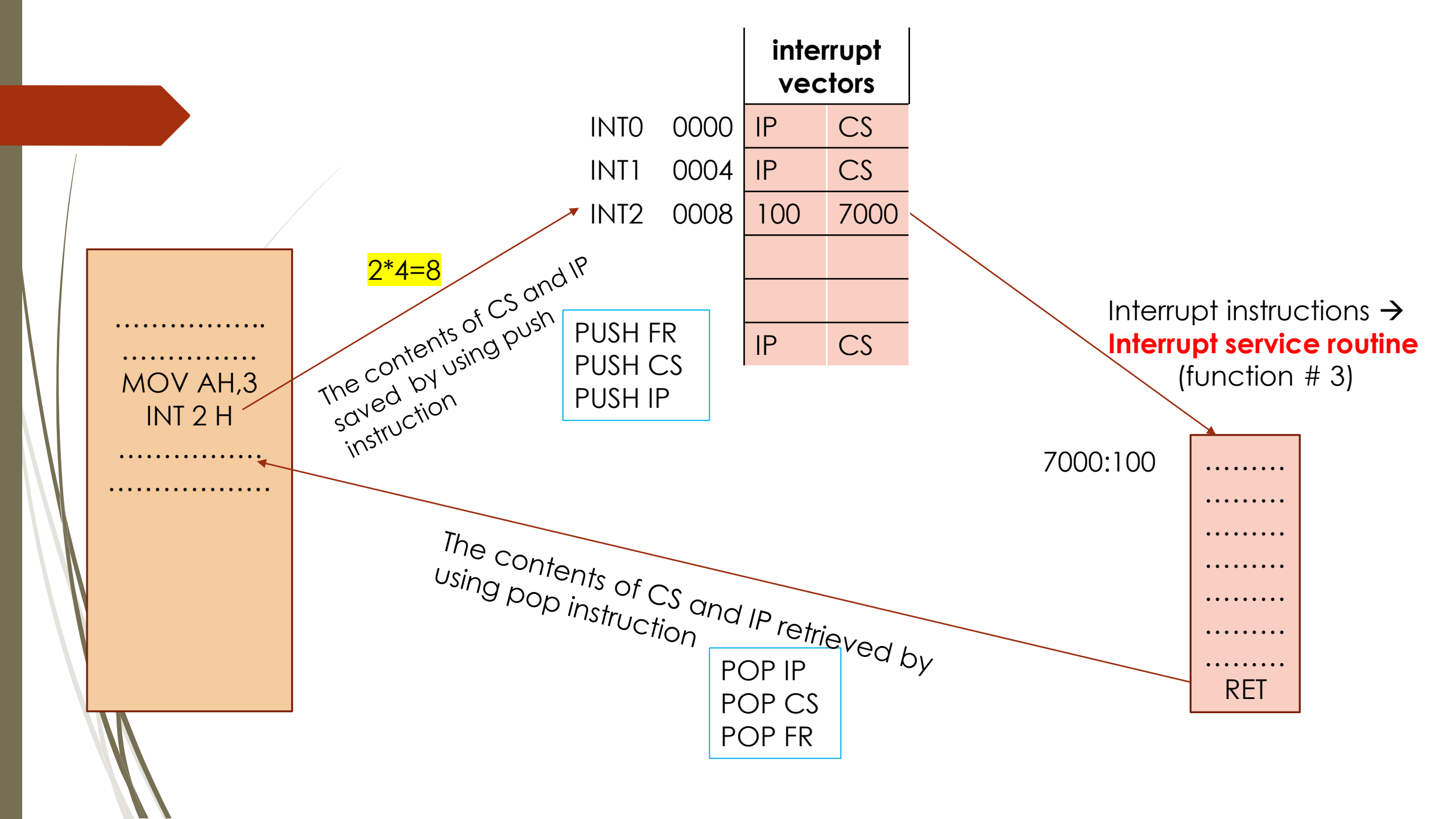
The first word of each interrupt vector contains the ***IP*** for the interrupt.

The second word of each interrupt vector contains the ***CS*** address for that interrupt.

- *The first 1Kbyte of memory contain the interrupt vectors.*

interrupt vector

IP	CS


interrupt vectors
INT0 0000 IP CS
INT1 0004 IP CS
INT2 0008 100 7000
IP CS
2*4=8
The contents of CS and IP saved by using push instruction
PUSH FR
PUSH CS
PUSH IP
MOV AH,3
INT 2 H
Interrupt instructions →
Interrupt service routine
(function # 3)
7000:100
RET
The contents of CS and IP retrieved by using pop instruction
POP IP
POP CS
POP FR

- To obtain an interrupt address the processor multiply the interrupt value by 4. (e.g. INT 10H interrupt vector address = 4*10 = 40H)

- - When an interrupt occurs, regardless of source, the 8086 does the
- following:
- 1- Push flag register into stack.
- 2- Clear IF and TF flags.
- 3- Push CS into stack.
- 4- Calculate the address of interrupt vector (interrupt type * 4).
- 5- Load the second word of the interrupt vector into CS register.
- 6- Push IP into stack.
- 7- Load the first word of the interrupt vector into IP.

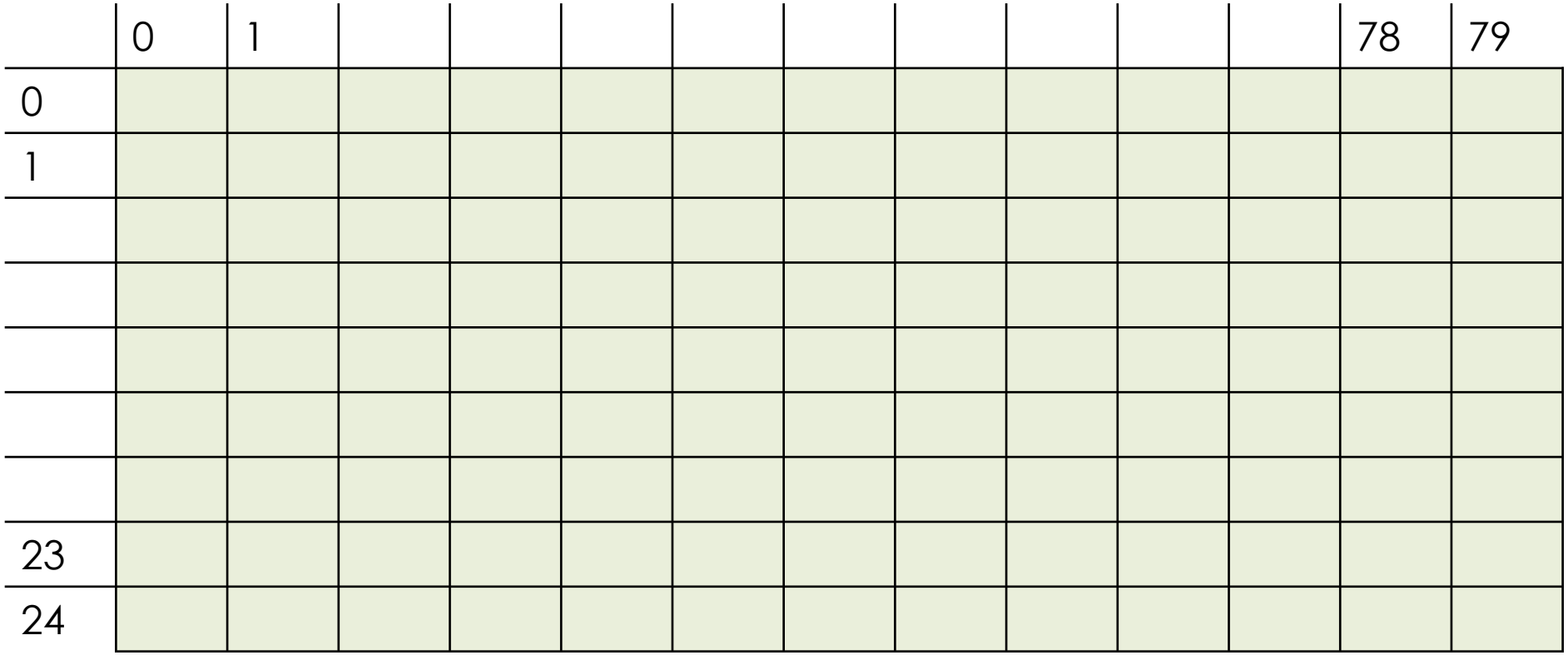
- The most common interrupts are INT 10H and its functions for screen handling,
- and INT 21H and its functions for display screen output and accepting keyboard input.
- - Low-level **BIOS** operations such as INT 10H transfer control directly to BIOS.
- However, to facilitate some of the more complex operations, INT 21H provides an interrupt service that first transfer control to **DOS**.

The screen:

- - Before tacking about display operation we must know how 8086 handles the screen.
- - A typical video monitor has 25 rows (numbered 0 to 24) and 80 columns (numbered 0 to 79). The rows and column provide a grid of addressable locations at any one of which the cursor can be set. Here are some examples of cursor
- locations:

Screen location	Decimal format		Hex format	
	Row	Column	Row	Column
Upper left corner	00	00	00H	00H
Upper right corner	00	79	00H	4FH
Center of screen	12	39/40	0CH	27H/28H
Lower left corner	24	00	18H	00H
Lower right corner	24	79	18H	4FH

Screen



Setting the cursor:

- Setting the cursor is a common requirement for text mode, because its position determines where the next character is to display. (Graphics mode does not support the cursor).

INT 10H function 02H for setting the cursor:

- INT 10H is the BIOS operation for screen handling, and function 02H in the 10H tells the operation to set the cursor.
- - Before calling the interrupt services you must do the following:

1- Load the required page (or screen) number, normally 0, in the BH register.

2- Load the row in the DH, and the column in the DL.

set cursor position

NT 10h / AH = 2 *input:*

DH = row.

DL = column.

BH = page number (0..7).

example:

```
mov dh, 10
mov dl, 20
mov bh, 0
mov ah, 2
int 10h
```

- The following example sets the cursor to row 08, column 15:

```
MOV AH,02H
MOV BH,00
MOV DH,08
MOV DL,15
INT 10H
```

- To set the row and column in the DX, you could also use MOV instruction with an immediate hex value:

```
MOV DX,080FH
```

Clearing the screen:

INT 10H function 06H/07H handles screen clearing or scrolling.

- You can clear all or part of a display beginning at any screen location and ending at any higher number location.
- Before calling the interrupt services you must load the following registers:

· AH = function 06H /07H

· AL = number of lines to scroll, or 00H for the full screen.

· BH = attribute value (color, reverse video, blinking).

· CX = starting row:column.

· DX = ending row:column.

- The CX and DX together define the screen area (or window) to be scrolled, and
- the AL provides the number of lines to be scrolled up.

INT 10h / AH = 06h - scroll up window.
INT 10h / AH = 07h - scroll down window.

input:

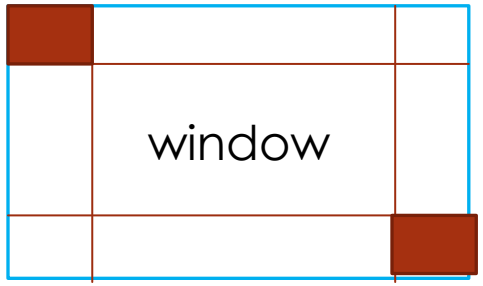
AL = number of lines by which to scroll (00h = clear entire window).
BH = attribute used to write blank lines at bottom of window.
CH, CL = row, column of window's upper left corner.
DH, DL = row, column of window's lower right corner.

BH =attribute ➔

BH

Background	foreground
4 BIT	4 BIT

CX: row, column



DX: row, column

- To clear the entire screen, specify the starting row:column in CX as 00:00H and the ending row:column in DX as 18:4FH.
- - Attribute **71**H loaded to the BH, sets the entire screen to light gray background (attribute 7) with blue foreground (attribute 1).

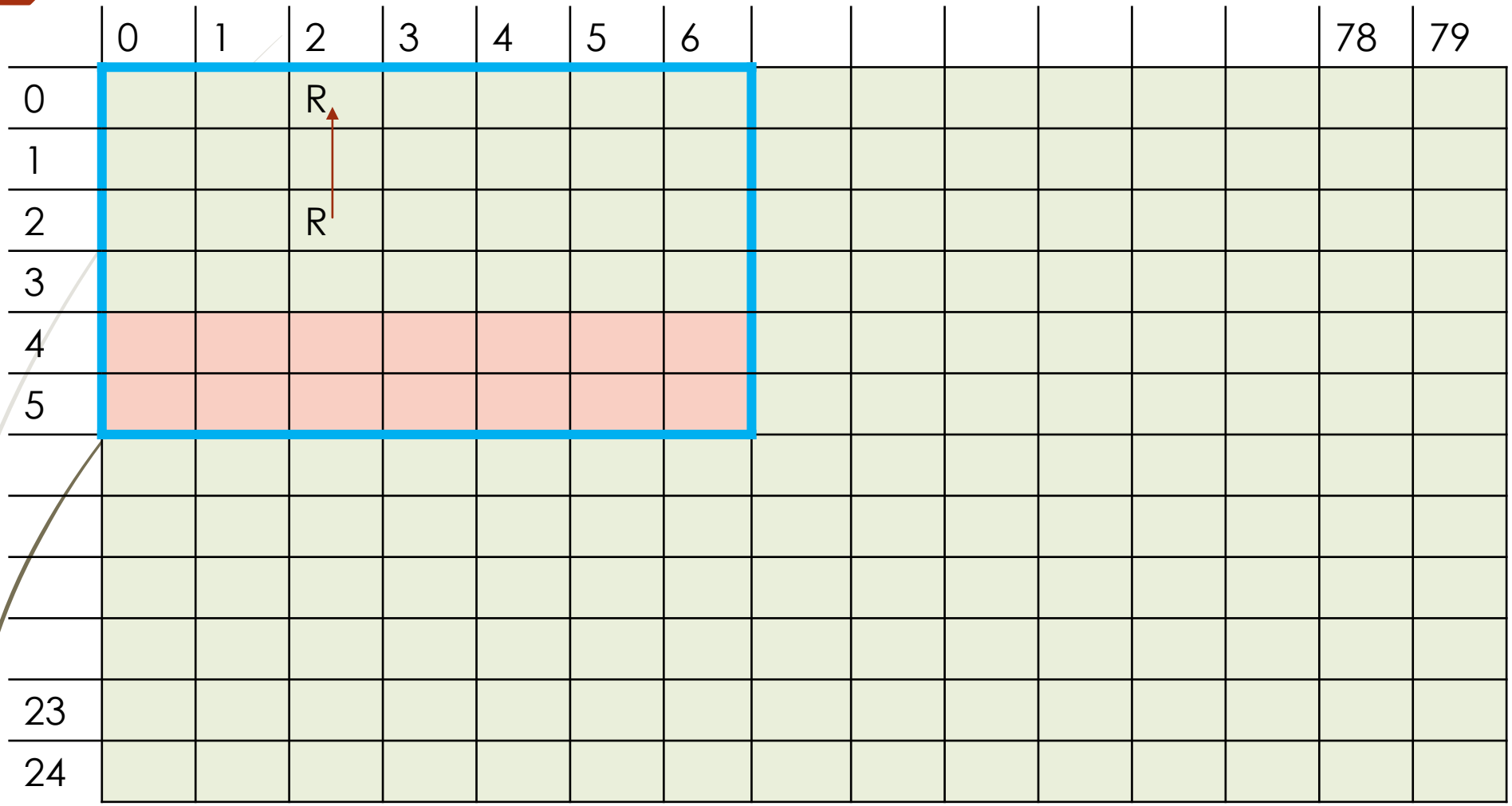
- EXAMPLE:

```
MOV AX,0600H
MOV BH,71H
MOV CX,0000H
MOV DX,184FH
INT 10H
```

HEX	BIN	COLOR
0	0000	black
1	0001	blue
2	0010	green
3	0011	cyan
4	0100	red
5	0101	magenta
6	0110	brown
7	0111	light gray
8	1000	dark gray
9	1001	light blue
A	1010	light green
B	1011	light cyan
C	1100	light red
D	1101	light magenta
E	1110	yellow
F	1111	white

```
mov ah,6      ;6 up
mov AL,2
mov BH,42h
mov Ch,0
mov cl,0      ;= row, column of window's upper left corner.
mov Dh,5
mov dl,6     ;= row, column of window's lower right corner.
int 10h
```

Screen



```
MOV AH,7
MOV CX,0004
MOV DX,0208H
MOV BH,41H
MOV AL,1
INT 10H
```

ABCDEFGHIJKLM NOPQRSTVWXYZ

ABCD EFGHI JKLM NOPQRSTVWXYZ



INT 21H function 02H for screen display

- **Using control characters in a screen display:**

- One way to make more effective use of displays is to use the Carriage return, Line feed, and Tab control characters. You can code them as Decimal or Hex values like this:

Control character	Decimal	Hex	Effect on cursor
Carriage return	13	0DH	Reset to left position of screen
Line feed	10	0AH	Advance to next line
Tab	09	09H	Advance to next tab stop

- You can use these control characters for handling cursor whenever you display output or accept input.

- **EXAMPLE**

```
MOV AH,02H ;Request display character
MOV DL,CHAR ;Character to display
INT 21H ;Call interrupt service
```

```
MOV CX,0
MOV AX, 0600H
MOV BH, 71H
MOV DX, 184FH
INT 10H
MOV AH,2
MOV DX,0D1BH
MOV BH,0
INT 10H
MOV AH,2
MOV DL,'A'
MOV CX,8
L:INT 21H
INC DL
LOOP L
```



- For displaying on an empty screen do the following:

1- Clear the screen.

2- Set the cursor.

3- Display the string.

INT 21H function 01H input single character

Input a Single Character from Keyboard

- The operating system subroutine uses the AL register to store the character it reads from the keyboard.

```
MOV AH, 1H      ; store keyboard function number in AH
INT 21h  ; call interrupt
; character is stored in AL
```

- The following example combines the two previous ones, by reading a character from the keyboard and displaying it.

- Write an assembly code to read a character from the keyboard and display it on the screen.

```
; Read/Input a character from keyboard
MOV AH, 1H
INT 21H
 ; Display a character on the screen
MOV DL, AL
MOV AH, 2
INT 21H
```

INT 21H function 09H for screen display:

- The simplicity of the original INT 21H function 09H for displaying still keeps it in common use. It requires definition of a display string in the data area, immediately followed by a dollar sign **($ or 24H)** delimiter, which the operation uses to end the display. The following example illustrates:

Data DB 'how are you?','$'

- You can code the dollar sign immediately following the display string as

'how are you?$', or

on the next line as DB '$'.

Data DB 'how are you'
DB '$'.

- The disadvantage, however, is that you **can not use this function to display a $** character on the screen.

- **To perform displaying operation do the following**:

1. ·Move 09H to the AH.
2. ·Use LEA (or offset) to load the address of the display string in the DX.
3. ·Issue an INT 21H instruction.

- The INT operation displays the character from left to right and recognize the end of data on encountering the dollar sign ($) delimiter.
- A displayed string that exceeds the rightmost screen column automatically continues on the next row and scrolls the screen as necessary.
- If you omit the dollar sign at the end of the string, the operation continues displaying characters from consecutive memory locations until it encounter one (if there).

- For displaying on an empty screen do the following:

1- Clear the screen.

2- Set the cursor.

3- Display the string.

```
mov dh, 5
mov dl, 5
mov bh, 0
mov ah, 2
int 10h

mov dx, offset msg
mov ah, 9
int 21h

 hlt
 msg db “abcdefgh“
      y  db    “****”
           db  ‘$’
```



```
STR  DB  'ABC',20H,41H,0AH,'EF',24H,'ab'
MOV AH,09
MOV DX,OFFSET STR
INT 21H
```



```
X DB ‘AB’,0AH,0AH,’CD’,0DH
DB ‘E’,’$’
MOV AH,9
MOV DX,OFFSET X
INT 21H
```



```
MOV AX,2000H
MOV DS,AX
MOV DX,1
MOV AH,9
INT 21H
```



DS:2000H

61H
62H
63H
64H
0AH
0AH
65H
09
66H
24H
67H

INT 21H function 0AH for keyboard input:

- - INT 21H function 0AH for accepting data from the keyboard is particularly powerful.
- - The input area for keyed-in characters requires a **parameter list** containing specified fields that the INT operation is to process.

The parameter list consists of these elements:

- **The first byte** of the parameter list contains your limit for the maximum number of input characters. The minimum is 0 and because this is a 1-byte field, the maximum is FFH, or 255. You decide on the maximum, based on the kind of data you expect users to enter.
- **The second byte** is for the operation to store the actual number of characters typed as a binary value.
- **The third byte** begins a field that is to contain the typed characters, from left to right.
- The following example defines a parameter list for an input area:

PARALST DB 20,?,20 DUP(‘ ‘)

- ✓ **20 example on : Maximum number of input characters**
- ✓ **? Is the actual number of input characters**
- ✓ **20 DUP(' ') locations to save characters entered from keyboard**

- **To request input**:

-**set function 0AH in the AH**

- load the address of the parameter list (PARALST in the example into the DX)

- issue INT 21H

- Example

```
MOV AH,0AH
LEA DX, PARALST
INT 21H
```

- The INT operation waits for a user to type characters and check that they **do not exceed the maximum** of 20.
- The operation echoes each typed character onto the screen and advances the cursor.
- -The user **presses (Enter) to signal the end of an entry**. The **operation also transfers the Enter character (0DH) to the reversed locations**.

buffer db 10,?, 10 dup(' ')

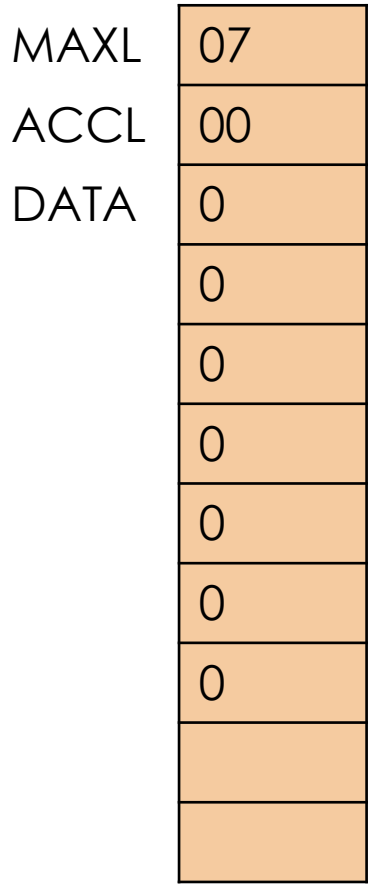
```
mov dx, offset buffer
mov ah, 0ah
int 21h
```

123 ab

enter by keyboard ==0DH

buffer	
	0A
	~~00~~ 6
	31H
	32H
	33H
	20H
	61H
	62H
	0D

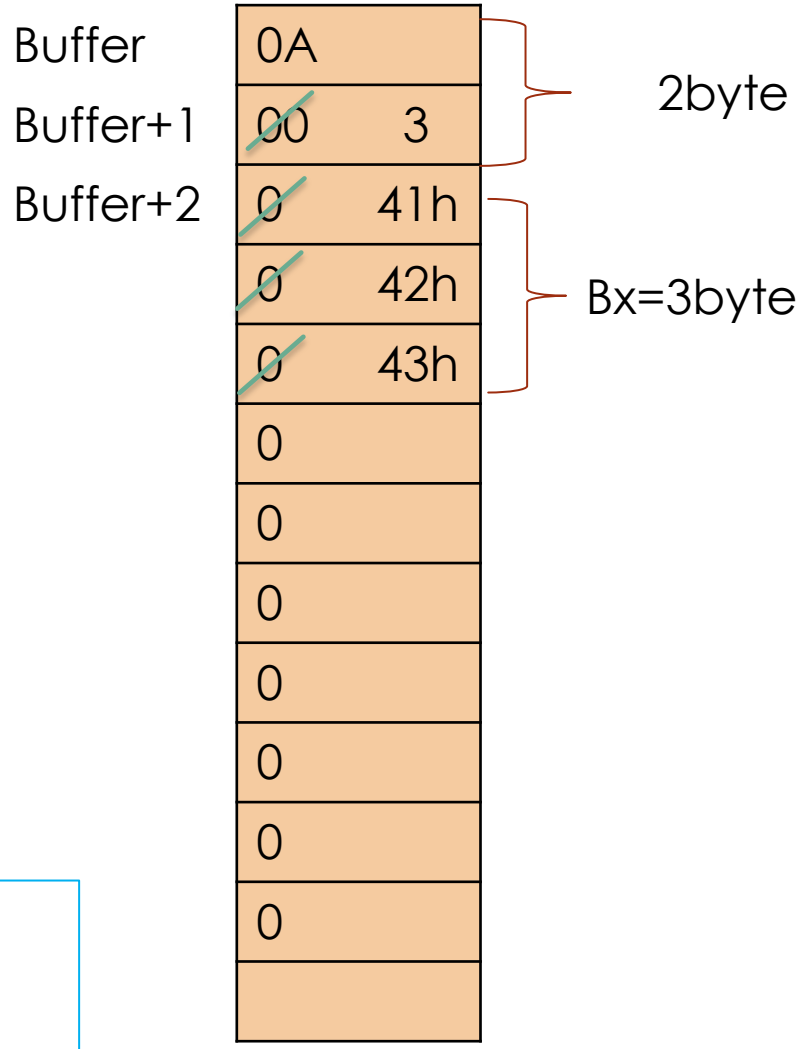
```
MAXL  DB 7
ACCL  DB ?
DATA  DB  7 DUP(?)
MOV AH,0AH
MOV DX,OFFXSET MAXL
INT 21H
```



```
org 100h
        mov dx, offset buffer
        mov ah, 0ah
        int 21h
        jmp print
        buffer db 10,?, 10 dup(' ')
        print:
        xor bx, bx
        mov bl, buffer[1]
        mov buffer[bx+2], '$'
        mov dx, offset buffer + 2
        mov ah, 9
        int 21h
        ret
```

Enter by the keyboard

ABC



Microprocessor System

Logical instructions:

AND instruction:

- Perform Logical AND between all bits of two operands. Result is stored in first

operand.

- The general format for AND is:

AND dest, src

Destination =destination AND source

Where:

Dest: represents register or memory.

Src: represents register, memory, or immediate.

- Registers and memory can be 8-bit or 16-bit length (memory can be more than 16-bits).

- Example:

```
MOV AL, 01100001b
AND AL, 11011111b                ; AL = 01000001b
```

Application:

use to do mask

```
MOV AL,75H
AND AL,0F      →  AL=05H
```

OR instruction:

- Perform Logical OR between all bits of two operands. Result is stored in first operand.
- The general format for OR is:

OR dest, src

Destination =destination OR source

Where:

Dest: represents register or memory.

Src: represents register, memory, or immediate.

- Registers and memory can be 8-bit or 16-bit length.

- Example:

```
MOV AL, 01000001b
OR AL, 00100000b                ; AL = 01100001b
```

- *Application*:

Conversion from capital latter to small latter and vice versa.

A - Z = 41H - 5AH

a - z = 61H - 7AH

NOT instruction:

- Invert each bit of the operand. →first complement

- The general format for NOT is:

NOT dest

Destination = NOT(destination)

Where:

Dest: represents register or memory.

- Example:

MOV AL, 00011011b

NOT AL ; AL = 11100100b

NEG instruction:

- Negate. Makes operand negative (two's complement).
- It computes the following: **dest := 0 - dest**
- This effectively reverses the sign of the destination operand.
- The general format for NEG is:

NEG dest

Destination = 0- destination

Where:

Dest represents register or memory.

- Example:

MOV AL, 5 ; AL = 05h

NEG AL ; AL = 0FBh (-5) →

NEG AL ; AL = 05h (5

```
0000 0000
0000 0101   -
-------------
1111 1011
```

CF= 1 SF=1 PF= 0 ZF=0 OF=0

- Equivalent instructions for NEG:

MOV AL,5

NOT AL

ADD AL,1

XOR instruction:

- Perform Logical XOR (Exclusive OR) between all bits of two operands. Result is stored in first operand.

- The general format for XOR is:

XOR dest, src

Destination =destination XOR source

Where:

Dest: represents register or memory.

Src: represents register, memory, or immediate.

- Example:

```
MOV AL, 00000111b
XOR AL, 00000010b                ; AL = 00000101b
```

- XOR can be used to test equality state (e.g. XOR 7,7 = 0. because 7=7).

1 XOR 1 = 0

1 XOR 0 = 1

0 XOR 1 = 1

0 XOR 0 = 0

TEST instruction:

- Perform Logical AND between all bits of two operands for flags checking only (without saving the result of AND). *The main aim of this operation is to effects on* the following flag register (SF, ZF, PF).

The general format for TEST is:

TEST dest, src

destination AND source

Where:

Dest represents register or memory.

Src: represents register, memory, or immediate.

- Example:

MOV AL, 00000101b

TEST AL, 1 ; ZF = 0.

TEST AL, 10b ; ZF = 1.

EXAMPLE #1: Write an assembly code to convert capital letters in the following string to small letters.
STR DB ‘HELLO’

```
ORG 100H
MOV BX,OFFSET STR
MOV CX,5
L1: OR [BX],20H
INC BX
LOOP L1
HLT
STR DB ‘HELLO’
```

BX	[BX]	CX
STR	h	5
STR+1	e	4
STR+2	l	3
STR+3	l	2
STR+4	o	1
STR+5		0

EXAMPLE #2: Write an assembly code to test if X is odd or even without using DIV/IDIV instructions.
If it is even store value ‘E’ in BL register , else store ‘O’ in BL

```
ORG 100H
MOV AL,X
AND AL,01H
JNZ L1
MOV BL,’E’
JMP END
L1: MOV BL,’O’
END:
HLT
X DB 4
```

OR

```
ORG 100H
TEST X,01H
JNZ L1
MOV BL,’E’
JMP END
L1: MOV BL,’O’
END:
HLT
X DB 4
```

EXAMPLE #3: Write an assembly code to set bit#2 in each element in array Y and reset bit#5 in each element in array Z.

```
ORG 100H
 MOV DI,0
MOV BX,3
L1:
OR Y[DI],04H
AND Z[DI],0FFDFH
ADD DI,2
DEC BX
JNZ L1
HLT
Y DW 20H,43H,51H
Z DW 33H,62H,74H
```

DI	BX	Y[DI]	Z[DI]
0	3	0024H	0013H
2	2	0047H	0042H
4	1	0055H	0054H
6	0		

EXAMPLE: convert the negative number in the array x to positive number

```
X DB -2,3,-1,4
MOV SI, offset x
MOV CX,4
L2: TEST [SI],80H
JNS L1
NEG [SI]
L1:INC SI
LOOP L2
```

SI	[SI]	CX	SF
X	2	4	1
X+1	3	3	0
X+2	1	2	1
X+3	4	1	0
X+4		0	0

Homeworks

- Question1 : Write an assembly code to set bit#3 and reset bit#4 in each element in array Z.

  Z DB 66H,78H,33H

- Question 2: convert the capital letters to small letters and the small to capital in string str

  str db ‘ABcdEFgh’

Microprocessor System

Looping instructions:

LOOPing Instructions

The jump instructions can be used for implementing loops. For example, the following code can be used for executing the loop-body 5 times.

```
MOV CX, 5
L1:
DEC CX
JNZ L1
```

- The 8086 assembly, provides a group of loop instructions for implementing iteration. The loop instructions are:
- LOOP instruction
- LOOPE / LOOPZ instruction
- LOOPNE / LOOPNZ instruction

- The loop instruction is useful when the number of iterations can be determined before the execution of the loop begins.

LOOP Instruction

- The basic LOOP instruction has the following syntax:

LOOP Target

- The LOOP instruction decrements the CX register and then jumps to the target location if the CX register does not contain zero. Since this instruction decrements CX then checks for zero. The target label identifies the target instruction as in the jump instructions. The loop instruction does not affect any flags.

LOOP instruction

- The general format for LOOP is:

LOOP LABEL

Decrease CX and jump to label if CX not zero.

*loop instruction does not affect any flags.

- The LOOP instruction work as the following algorithm:

CX = CX – 1 does not affect any flags.

If (CX != 0)

    Jump to Target

Else

    No jump (continue)

```
MOV CX, 5
L1:
DEC CX
JNZ L1
```

- The above code snippet could be written as:

```
MOV CX, 5
L1:
; LOOP BODY
LOOP L1
```

Note

- If the CX register originally contained zero, any loop you create using the LOOP instruction will repeat **65,536** times. If you do not want to execute the loop when CX contains zero, use JCXZ to skip over the loop.

- Example: The following program prints the number 1 to 9 on the screen

```
ORG 100H
MOV CX, 9
MOV DL, ‘1’
MOV AH, 2
L1: INT 21H
INC DL
LOOP L1
HLT
```

- Example: The following program store the number 1 to 4 on the memory x

```
X DB 4 dup(0)
MOV SI, offset X
MOV CX, 4
MOV DL, 1
L1: MOV [SI],DL
INC DL
INC SI
LOOP L1
HLT
```

SI	CX	DL	
X	4	1	
X+1	3	2	
X+2	2	3	
X+3	1	4	
X+4	0	5	

X	1
X+1	2
X+2	3
X+3	4

LOOPE / LOOPZ Instructions

- LOOPE and LOOPNE are essentially LOOP instructions with one additional check. LOOPE / LOOPZ will jump to the target address if CX is not zero and the zero flag is set (ZF = 1).
- This instruction is quite useful after CMP instruction. This instruction work as the following algorithm:

LOOPE LABEL

LOOPZ LABEL

CX = CX – 1

If (CX != 0 and ZF = 1)

Jump to Target

Else

No jump (continue)

- The LOOPE/LOOPZ instruction falls through on one of two conditions. Either the zero flag is clear or the instruction decremented CX to zero.
- By testing the zero flag after the loop instruction (with a JE or JNE instruction, for example), you can determine the cause of termination.
- The LOOPE/LOOPZ instruction does not affect any flags.
- EXAMPLE:

```
MOV AX, 0
MOV CX, 5
L1:  ADD AX, 100
CMP AH, 0
LOOPE L1
```

This code loop until the result fits into AL alone, or 5 times. The result will be over 255 on third loop (100+100+100), so loop will exit. The following table illustrate the tracing of the code at each loop.

LOOP #	AX	AH	CX	ZF (Compare AH with 0)
0	0	0	5	0
1	100D = 0064H	0	4	1
2	200D = 00C8H	0	3	1
3	300D = 012CH	1	2	0 >> Exit Loop

After executing the code, AX = 012CH and CX = 0002

LOOPNE / LOOPNZ Instruction

- This instruction is just like the LOOPE/LOOPZ instruction except LOOPNE/LOOPNZ repeats while CX is not zero and the zero flag is clear (ZF = 0). The LOOPNE/LOOPNZ work as the following algorithm:

LOOPNE LABEL

LOOPNZ LABEL

- The LOOPNE/LOOPNZ instruction does not affect any flags

```
CX = CX – 1
If (CX != 0 and ZF = 0)
        Jump to Target
Else
        No jump (continue)
```

.

- Example : Consider the following code, which is loop until the value 7 is found or 5 times.

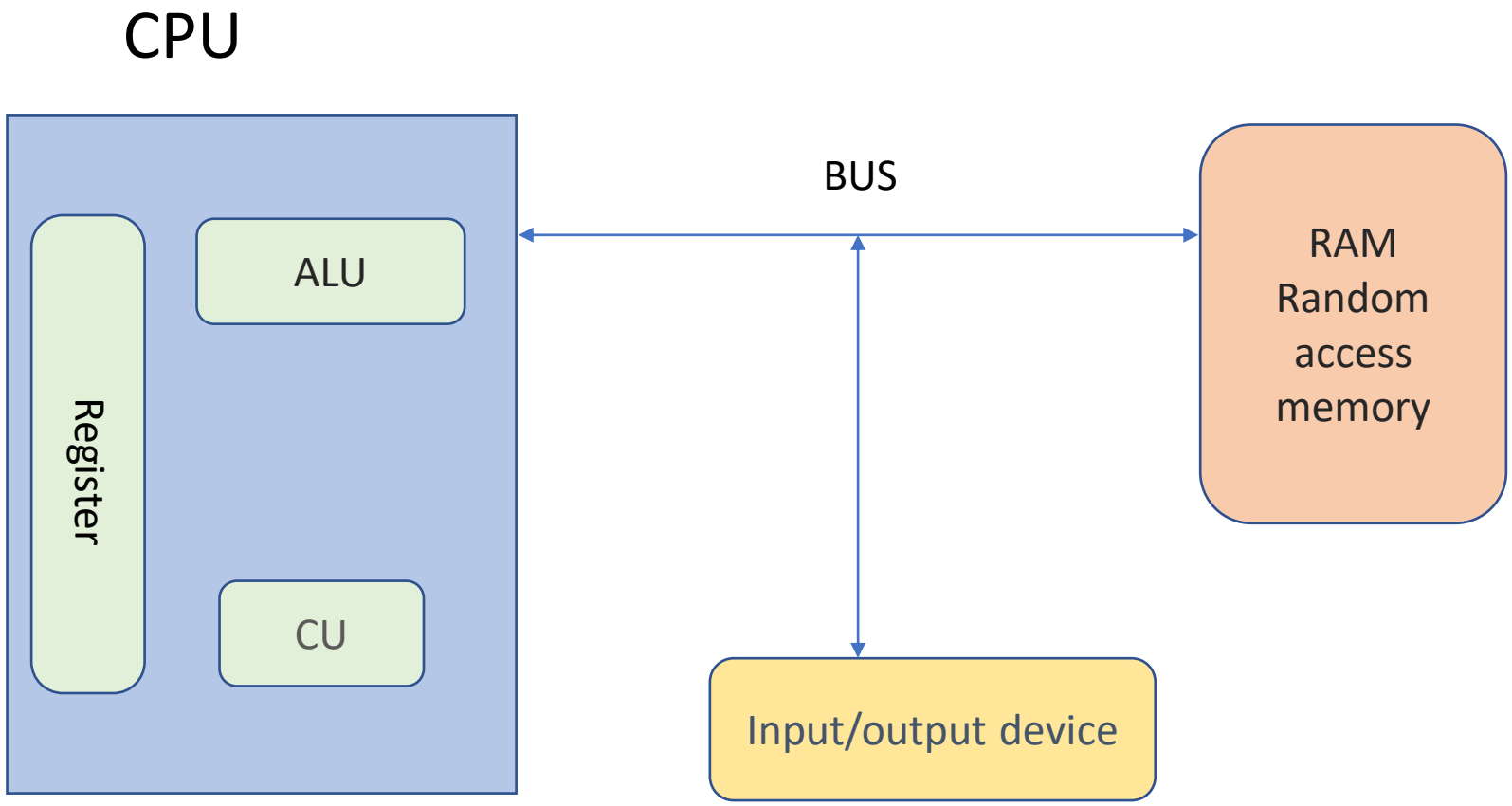
```
V DB 9, 8, 7, 6, 5
MOV SI, 0
MOV CX, 5
L1:
MOV AL, V[SI]
INC SI
CMP AL, 7
LOOPNE L1
```

CX	SI	AL	ZF
5	0	0	0
4	1	9	0
3	2	8	0
2	3	7	1

Microprocessor System

Microprocessor 8086 architecture

Simple computer model



Memory organization

- System bus:

1- data bus

2-address bus

3- control bus

- Maximum of memory of size= 2^n

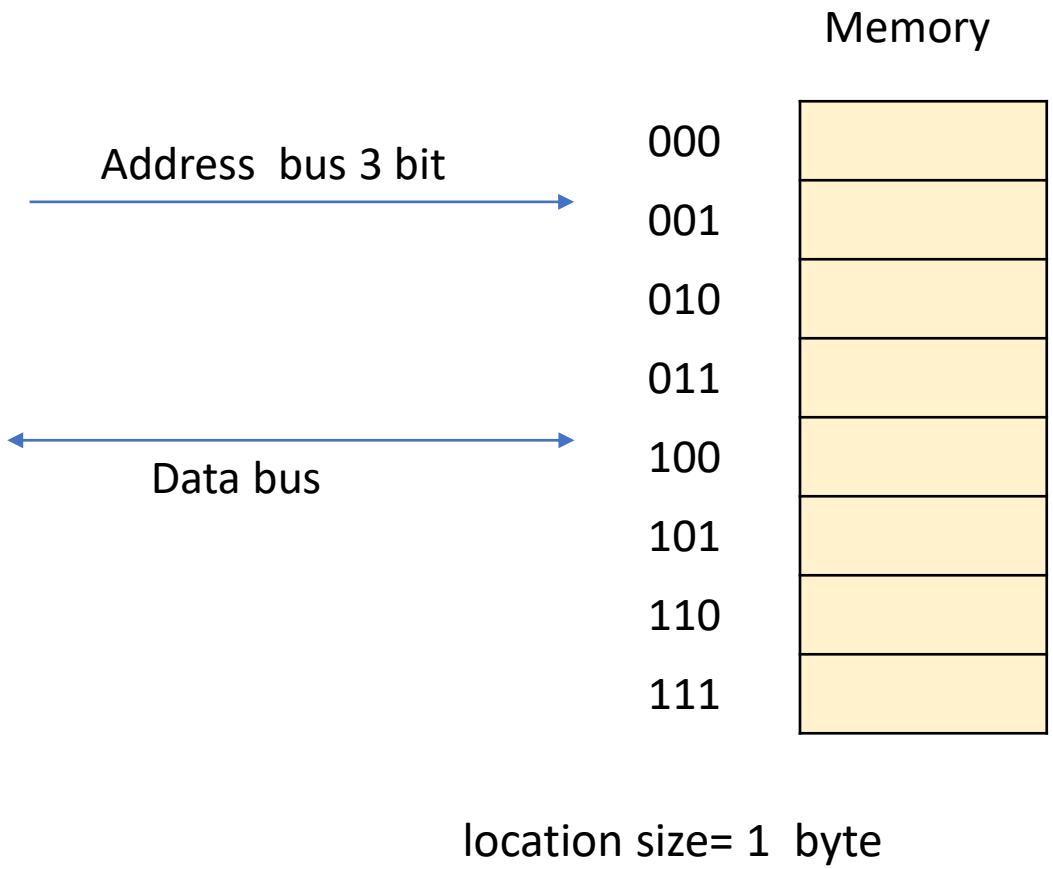
n: address bus size

EX: address bus size =3 bit

Memory size = 2^3 = 8 locations

If location size= 1 byte then memory size = 8 bytes

Range of addressing 000- 111



Intel's 8086 and 8088: The Birth of the PC:

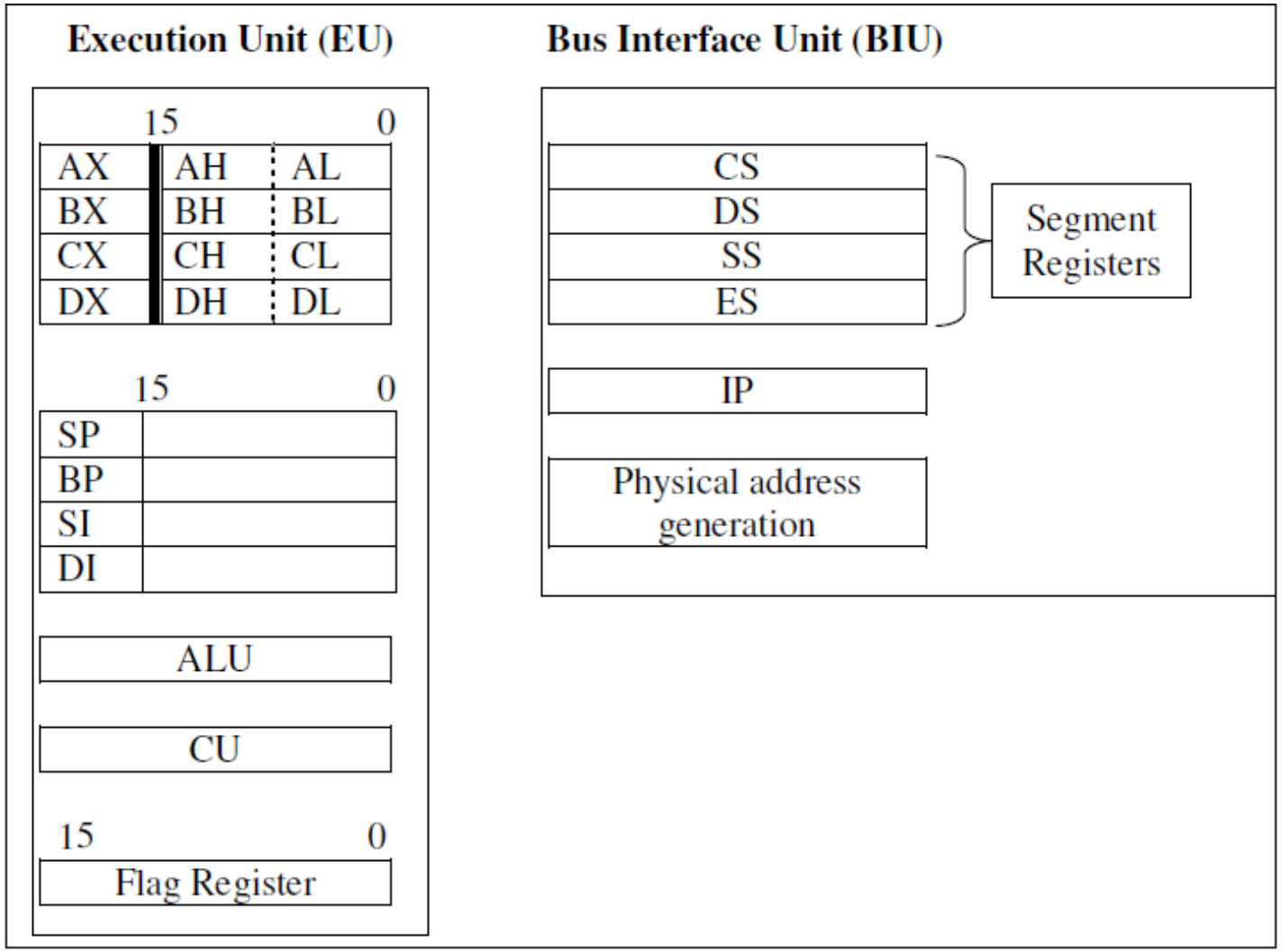
Chip	Number of Transistors	CPU Speed (MHz)	Register Width	External Data Bus	Address Bus	Internal Cache
Intel 8088	29,000	4.77-8	16-bit	8-bit	20-bit	None
Intel 8086	29,000	4.77-10	16-bit	16-bit	20-Bit	None

address bus size =20 bit

Memory size = 2^{20} = 1MByte locations

Range of addressing 00000h- FFFFFh

Microprocessor 8086 architecture:



Microprocessor 8086 architecture

- processor is partitioned into two logical unit

1- Execution unit (EU):

- Execute instructions.

- Contains an arithmetic and logic unit (ALU), a control unit (CU), and a number of registers.

2- **Bus interface unit (BIU):**

- Delivers instructions and data to the EU.

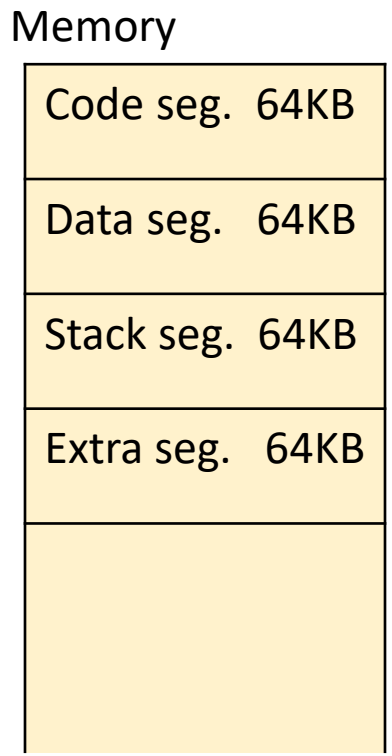
- Manage the bus control unit, segment registers, and instruction queue.

- The BIU controls the buses that transfer data to the EU, to memory, and to external input/output devices, whereas the segment registers control memory addressing.

- BIU provides access to instructions:

Segments and Addressing:

A segment is a special area defined in a program. In 8086 processor there are four types of segment, and each segment at most contains 64KByte of the memory:



1- **Code segment**: contains the machine instructions that are to execute. Typically the first executable instruction is at start of this segment, and the operating system links to that location to begin program execution.

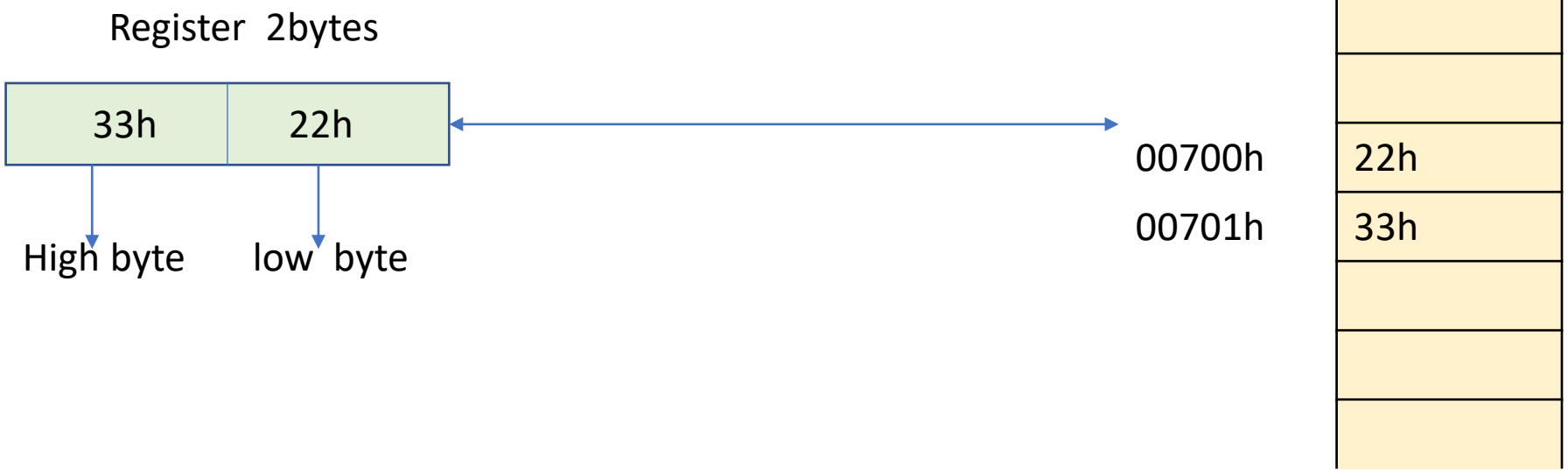
2- Data segment: contains a program's defined data, constants, and work areas.

3- Stack segment: contains any data and addresses that a programmer need to save temporarily or for use by a programmer defined subroutines.

4- Extra segment: used for string operations and defined just on need.

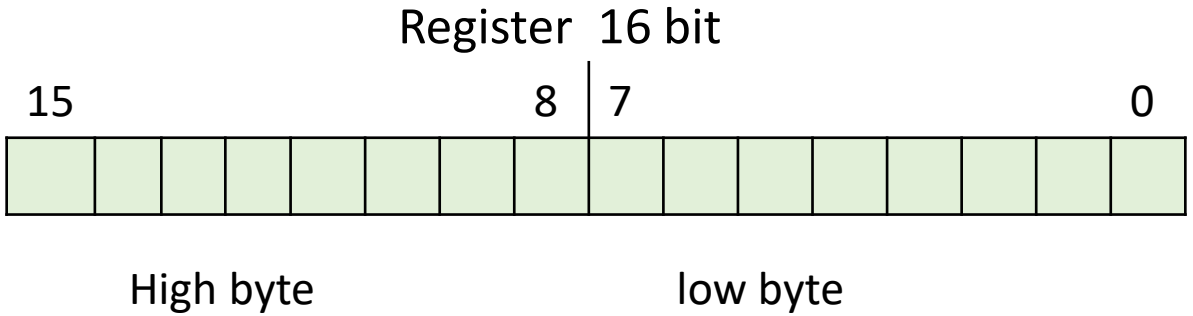
Addressing Data in Memory:

- Register width = 16bit = 2 bytes
- location size in the memory = 1 byte
- Data bus size =16 bit = 2 bytes
- Address bus size= 20 bit



Registers:

- The processor registers are used to control instructions being executed, to handle addressing of memory, and to provide arithmetic capability



1-**Segment registers: CS,DS,SS,ES**

2-Pointer registers: IP,SP, BP

3-Index Registers: SI,DI

4- General Purpose Registers: AX,BX,CX,DX

5-Flag Register

General Purpose Registers:

-AX register: is the primary accumulator, is used for operations involving input/output and most arithmetic.



- **BX register:** is known as the base register since it is the only general purpose register that can be used as an index to extend addressing. The BX may also be used for computations.



- **CX register:** is known as the count register, it may contain a value to control the number of times a loop is repeated or a value to shift bits left or right. The CX may also be used for computations.



- **DX register:** is known as the data register, some input/output operations require its use.



Segment registers:

- Is 16 bits long and provides for addressing an area of memory known as the current segment.

- CS register: contains the starting address of a program's code segment. This segment address plus an offset value in the instruction pointer (IP) register, indicates the address of an instruction to be **fetched** for execution. For programming purposes, you need not reference the CS register.

- **DS register**: contains the starting address of a program's data segment. Instructions use this address to locate data. This address plus an offset value in an instruction, causes a reference to a specific byte location in the data segment.

- **SS register:** a program uses a stack for temporary storage of address and data. The system stores the starting address of a program's stack segment in the SS register. This segment address, plus an offset value in the stack pointer (SP) register, indicates the current word in the stack being addressed. For programming purposes, you need not reference the SS register.

- **ES register**: used by some string (character data) operations to handle memory addressing. ES register is associated with the DI (index) register.

Pointer registers:

- The three pointer registers are the IP, SP, and BP.

- **Instruction Pointer (IP) register:** the 16 bit IP register contains the offset address of the next instruction that is to execute. You do not reference the IP register in a program

- **Stack Pointer (SP) register**: the 16 bit SP register provides an offset value, which, when associated with the SS register, refers to the current word being processed in the stack

- **Base Pointer (BP) register**: the 16 bit BP facilitate referencing parameters which are data and addresses that a program passes via the stack. The processor combines the address in the SS with the offset in the BP.

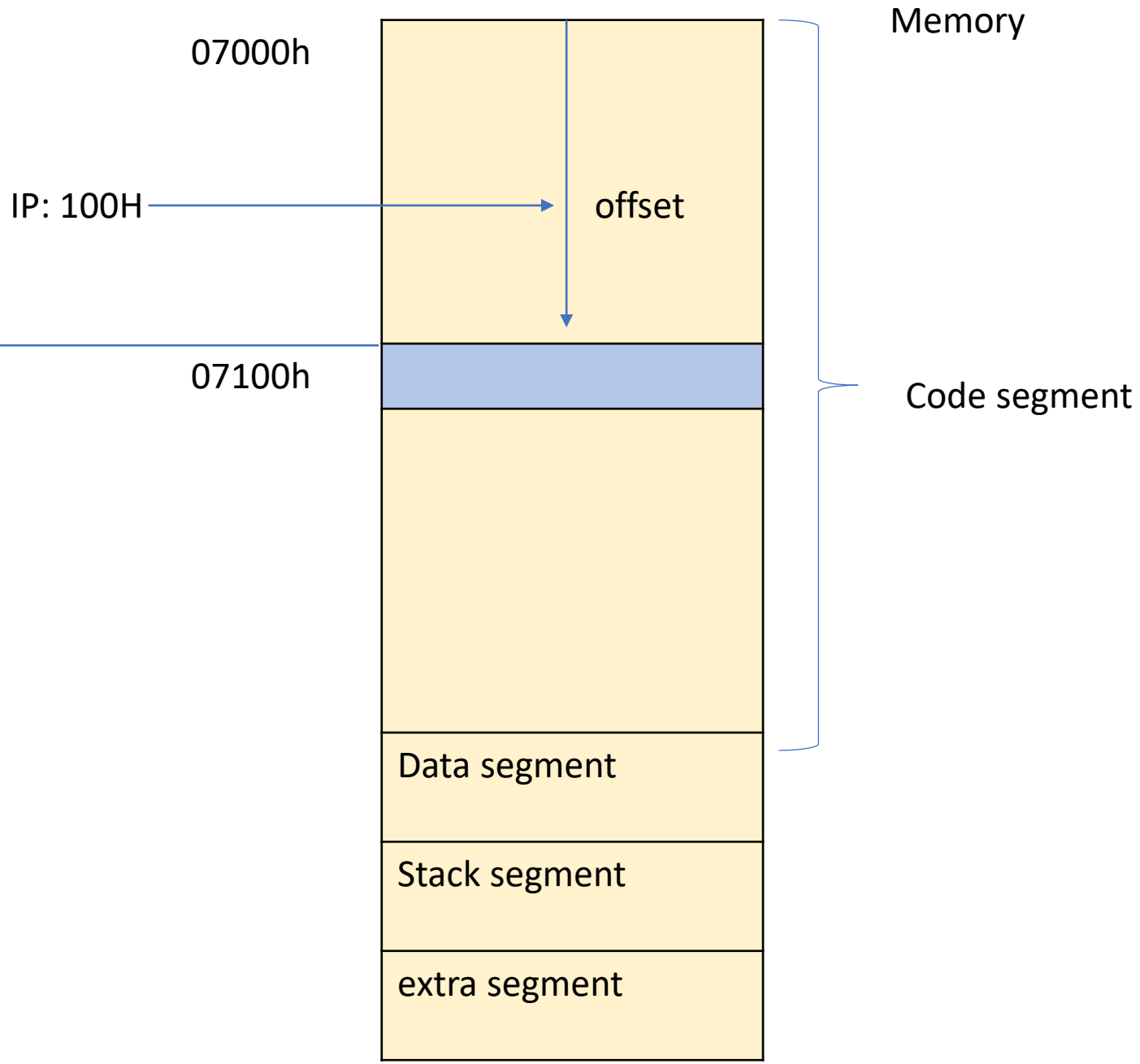
Index Registers:

- The SI and DI registers are available for indexed addressing and for use in addition and subtraction.

-SI register: the 16 bit source index register is required for some string operations. The SI is Associated with the DS register.

- **DI register**: the 16 bit destination index register is also required for some string operations. The DI is Associated with the ES register.

CS:0700h
IP:100h
Address of location ?



Address of location=
=07000 + 100
= 07100 h

Code segment store the codes of program(instructions)

IP indicate offset address of the next instruction that is to execute

Types of address in 8086

- Logical address : in program 16 bit

  segment register: offset

- Physical address: 20 bit
- How can convert logical address 16 bit into physical address

Segment	Segment reg.	offset
Code seg.	CS	IP
Data seg.	DS	SI,DI,BX
Stack seg.	SS	SP,BP
Extra seg.	ES	DI

Types of address in 8086

Physical address= segment register *10h+offset

=(CS,DS,ES,SS)*10H+offset

Ex: CS=0700H , SS=0800 , IP=234H ,SP=20H

segment register: offset

CS:IP	SS:SP
0700H:234H	0800H:20H

Physical address=CS*10H+IP=

= 07000H+234H=07234H

Physical address=SS*10H+SP=

= 08000H+20H=08020H

Types of address in 8086

CS: 16 bit, IP: 16 bit → address: 20 bit ?

Physical address =CS*10h+IP

EX: CS register contains 39B4H and the IP contains 514H. Find the next instruction to be executed?

CS*10h= 39B4H * 10h= 39B4[0]H

The processor combines the address in the CS with the offset in the IP to find the next instruction to be executed.

Segment address in CS 39B40H

Plus offset address in IP + 514H

---

address of the next instruction 3A054H

Types of address in 8086

- EX: physical address 00054h,segment register=0005 find offset

Offset= 4

Ex: : physical address 10009h, IP=9 find CS

CS=1000H

Flag Register:

Flag					OF	DF	IF	TF	SF	ZF		AF		PF		CF
Bit no	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

- OF (overflow): indicates overflow of high order (leftmost) bit following arithmetic.

- DF (direction): determines left or right direction for moving or comparing string data (0= left-to-right and 1= right-to-left).

- IF (interrupt): indicates that all external interrupts, such as keyboard entry are to be processed or ignored. IF=1: allowed ,IF: not allowed

- TF (trap): TF=1 run the program step by step, TF=0:run the program one step

Flags

SF (sign): contains the result of an arithmetic or comparison operation (0=positive and 1= negative).

- ZF (zero): indicates the result of an arithmetic or comparison operation (0=nonzero and 1= zero result).

- AF (auxiliary carry): contains a carry out of bit 3 on 8-bit data, for specialized arithmetic (BCD operation).

- PF (parity): indicates even or odd parity of low order (rightmost) 8-bit data operation (0= odd parity and 1= even parity result).

- CF (carry): contains carries from a high order (leftmost) bit following an arithmetic operation; also, contains the contents of the last bit of a shift or rotate operation.

Flags

0000 1101

0000 1101 -

0000 0000 ZF=1 PF= 1

0000 1100

0000 1101 +

0001 1001 AF=1 PF=0

Flags

1000 1001

1000 1101 +

0001 0110 CF=1 AF=1 OF=1

0100 1001

0100 1101 +

1001 0110 SF=1 OF=1 CF=0 PF=1 AF=1

1112

0 2

0000 1001

0000 1101 -

1111 1100 SF=1 PF=1 OF=0 CF=1 AF=1

---

11 1 112

1000 0000

0000 0010 -

0111 1110 OF=1 AF=1 CF=0

Microprocessor System

Parallel port

Parallel and serial port

- Port: set of signal lines to send and receive data with other components(devices)

- Type :

1- Parallel port : send 8-bit at the same time

2- serial port: sent bit,bit ….

- When a PC sends data to a printer or other device using a parallel port, it sends 8 bits of data (1byte) at a time.

- These 8 bits are transmitted **parallel** to each other, as opposed to the same eight bits being transmitted **serially** (all in a single row) through a serial port.



Programmable peripheral input and output port

- a parallel input/output port is Programmable peripheral input and output port
- If you want to control something electronic with your PC, well, the PC parallel port is the easiest way to go. You can use the parallel port to control motors, lights etc. via simple circuits.

- The BIOS defines the mode of the parallel port. The following modes are commonly found in Pentium-class motherboards: SPP, ECP, EPP and ECP+EPP. Auto and Bidirectional mode are sometimes included in newer boards.

- We'll be using SPP or Standard Parallel Port mode in our examples. It has 8 DATA lines, 5 STATUS lines and 4 CONTROL lines. A total of 12 output lines and 5 input lines.

- The original specification for parallel ports was unidirectional, meaning that data only traveled in one direction for each pin.
- With the introduction of the PS/2 in 1987, IBM offered a new **bidirectional** parallel port design. This mode is commonly known as **Standard Parallel Port** (SPP) and has completely replaced the original design. Bidirectional communication allows each device to receive data as well as transmit it.
- Many devices use the eight pins (2 through 9) originally designated for data. Using the same eight pins limits communication to half-duplex, meaning that information can only travel in one direction at a time.
- But pins 18 through 25, originally just used as grounds, can be used as data pins also. This allows for full-duplex (both directions at the same time) communication.

Registers in parallel port

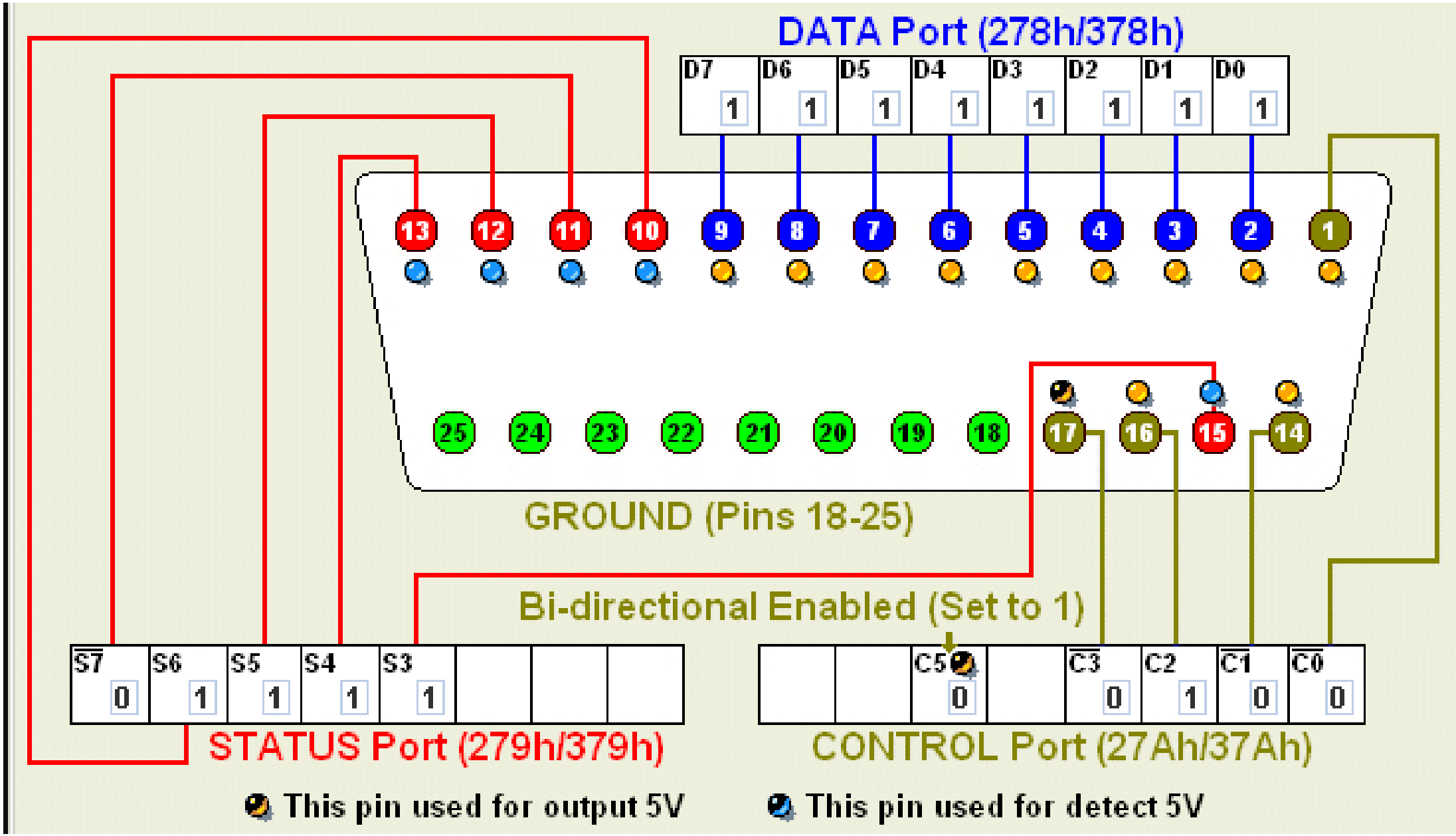
Port	Direction	address	# of used bits	Used Bits
Data	output	378h	8	D0-D7
Status	input	379h	5	S3-S7
control	output	37ah	4	C0-C3

All input and output pins are TTL levels. 5V comes out of each pin of the parallel port and can source a few milliAmperes.

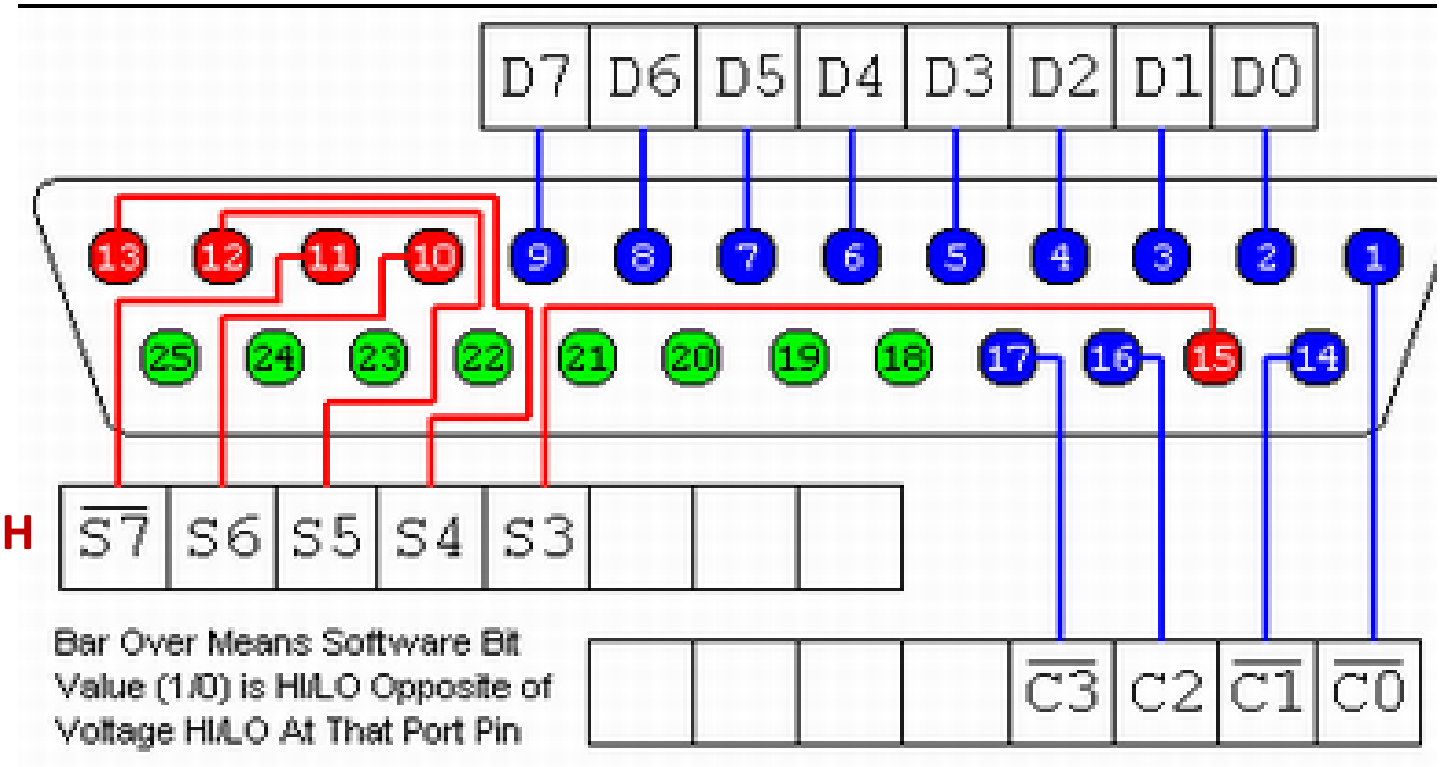
The printer port or parallel port has 25 pins and here is a summary of their pin number, name, brief description, state.

Pin No.	Name	Description
1	strobe	control
2	d0	data - active high
3	d1	data - active high
4	d2	data - active high
5	d3	data - active high
6	d4	data - active high
7	d5	data - active high
8	d6	data - active high
9	d7	data - active high
10	acknowledge	status
11	busy	status
12	paper out	status
13	select in	status
14	auto	control
15	error	status

16	initialize	control
17	select prt	control
18	Ground	
19	Ground	
20	Ground	
21	Ground	
22	Ground	
23	Ground	
24	Ground	
25	Ground	


DATA Port (278h/378h)
D7 D6 D5 D4 D3 D2 D1 D0
1 1 1 1 1 1 1 1
13 12 11 10 9 8 7 6 5 4 3 2 1
25 24 23 22 21 20 19 18 17 16 15 14
GROUND (Pins 18-25)
Bi-directional Enabled (Set to 1)
S7 S6 S5 S4 S3
0 1 1 1 1
C5 C3 C2 C1 C0
0 0 1 0 0
STATUS Port (279h/379h)
CONTROL Port (27Ah/37Ah)
This pin used for output 5V
This pin used for detect 5V

DATA REGISTER 378H



STATUSE REGISTER 379H

CONTROL REGISTER 37AH

- Active high means that the pin is normally low or does not have voltage. Active low is you guessed it, the pin is normally has voltage. Active low pins are sometimes represented with a bar line on top. To elaborate further, the pins can have two states, they can either be with voltage or without voltage.

- These pins are divided into three groups. They are **DATA**, **STATUS** and **CONTROL** groups or we'll simply call them **ports or registers**.

- Each port can be accessed using their default assigned addresses usually in hexadecimal notation. Each address can hold 8 bits of data.

Architecture of Ports

DATA port (address 378H)

As I've mentioned earlier, each port contain 8 bits or simply, 1 byte. For the DATA port, bit d7 is the most significant bit and bit d0 is the least significant. Most of experimentations and actual projects will use this port.

d7	d6	d5	d4	d3	d2	d1	d0

Here are the corresponding pin numbers on the DB25 connector.

9	8	7	6	5	4	3	2

Not all DATA ports are capable of input/output. They are commonly output-only ports but some motherboards offer bidirectional. Even if it is not bidirectional, you can also read the content of the port and you will get the last byte that was sent.

CONTROL port (address 37AH)

The control lines are input/output ports. The software register of the control lines is accessed at address 37AH or 890 in decimal.

Here are the pins and pin numbers.

c7	c6	c5	c4	c3	c2	c1	c0

Each bit stands for: c7-c5 (reserved); c4=interrupts(0:disabled); c3=always 1; c2=0 to initialize printer (1 normal); c1= LF and CR (0 no auto LF); c0= 0 normal but if 1 (set it to output data to printer)

				17	16	14	1

STATUS port (address 379H)

s7'	s6	s5	s4	s3	s2	s1	s0

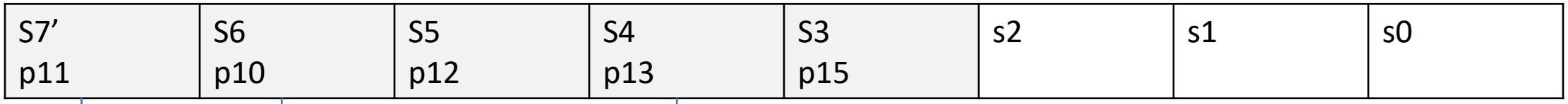
Each bit stands for: s7'= busy; s6=acknowledge; s5=paper out; s4=select in; s3=error; s2 to s0= reserved.

The pin numbers follow.

11'	10	12	13	15			

The STATUS lines are input only ports. If you try to use it as output, it will not accept the byte sent. Its address is located at 379H or 889 in decimal. Pins 10, 12, 13, 15 are normally hi and pin 11 is normally low. The default value of this port when there is no peripheral connected is 7F in hexadecimal. If a printer or scanner is connected, its value will be 87H. This is used for feedback from the external system.

The Status Register 379h



0 printer error
1 printer normal

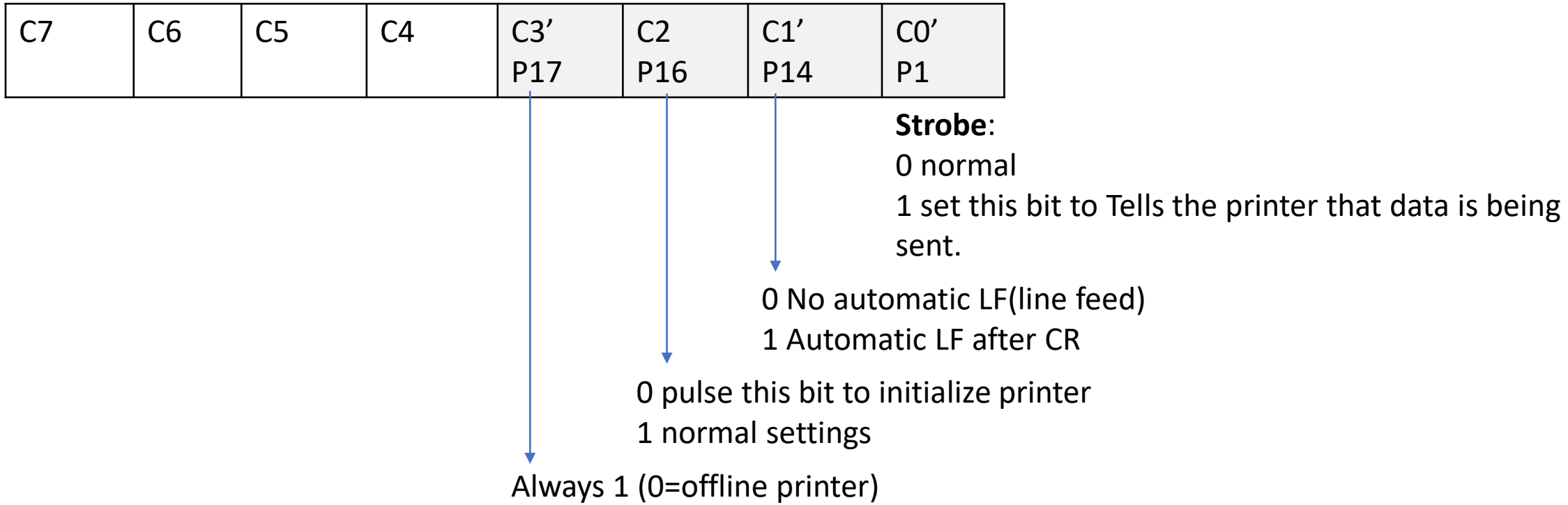
Select in
0 printer not online
1 printer online

0 the printer has printer
1 the printer has run out of paper

0 acknowledge pulse: to let the computer know that the data was received.
1 normal input

0 the printer is busy, so don't send data to the printer
1 the printer is not busy

The Control Register 37Ah



Using Assembly Language

This is the best way to manipulate ports and do low level i/o. You can do bit by bit manipulation. To write to a port use [out] and to read a port use [in].

Here's there syntax:

OUT port, register

IN register, port

Register must be AL and port must be a number stored in DX (can be decimal, hex, or binary). The port is the address e.g. 378H, 379H, 37AH.

Number of port : DX → if number of port > 255

Immediate value (1 byte) → if number of port < 255

IN instruction : Read from port

IN REGISTER,PORT

- **REGISTER :**Input from port into AL or AX.
- **PORT :**Second operand is a port number.

DX register should be used → If required to access port number over 255
Immediate value can be used → If required to access port number less 255

- Example:

IN AX, 4

- Example:

MOV DX,378H
IN AL, DX

OUT instruction : Write to port

OUT PORT,REGISTER

- **REGISTER :**output from AL or AX to port.
- **PORT :**First operand is a port number.

DX register should be used → If required to access port number over 255
Immediate value can be used → If required to access port number less 255

- Example:

MOV AX, 0FFFh
OUT 4, AX

- Example:

MOV DX,378H
MOV AL, 100b
OUT DX, AL

Example: use parallel port with printer

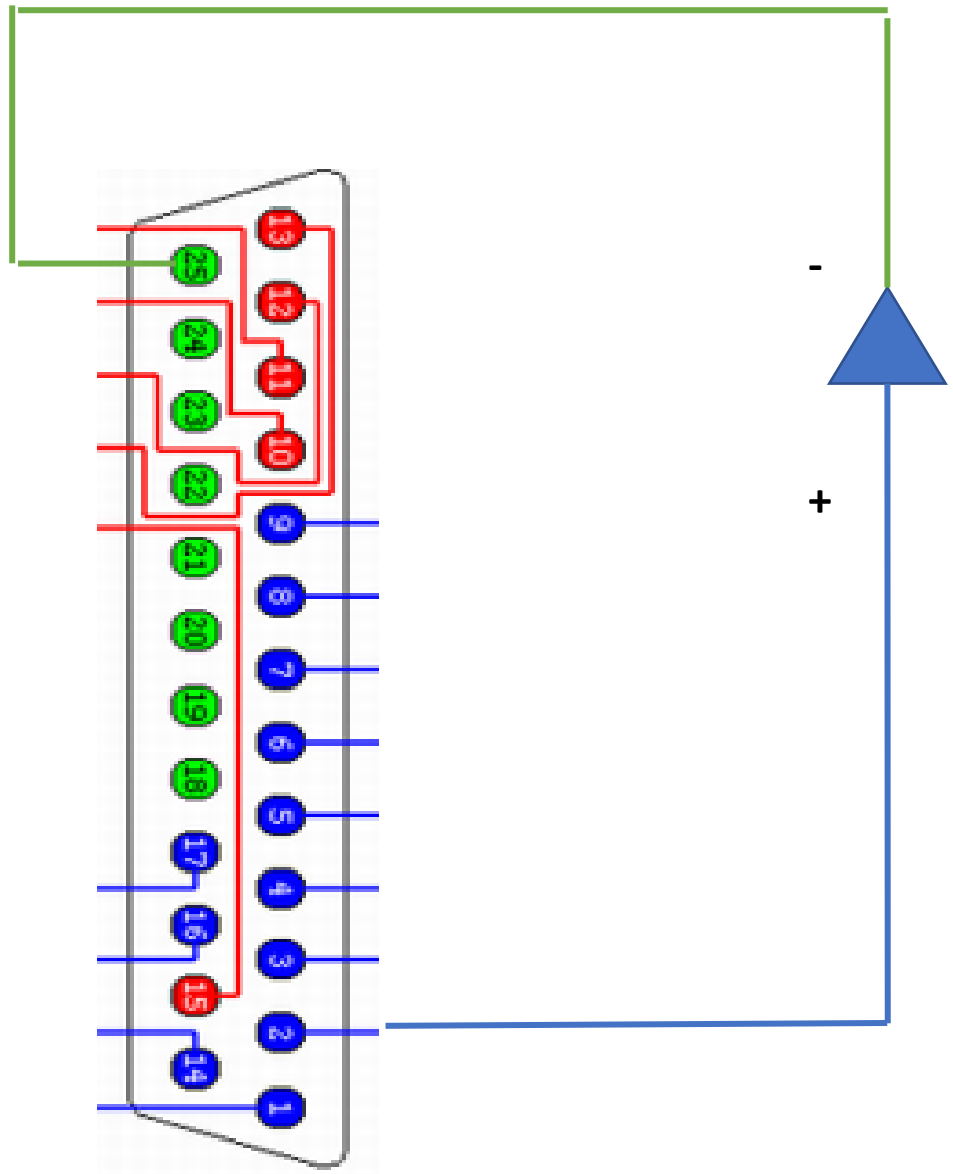
```
MOV DX,37AH          ;control reg
MOV AL,08H           ;  C3=1  &  C2=0:make the printer online and initialize
OUT DX,AL
MOV CX,50
L:NOP                ;wait
LOOP L1
```

```
MOV DX,379H          ;status reg
BUSY: IN AL,DX
TEST AL,80           ;S7=0 printer busy :test if the printer is busy
JZ BUSY
MOV DX,378H          ;data reg
MOV AL,41H           ; data  'A'
OUT DX,AL
MOV DX,37AH          ;control reg
MOV AL,0BH           → 1011 → C0=1: computer tells the printer that data is sent
OUT DX,AL
```

```
MOV DX,379H              ;status reg
ACK: IN AL,DX
TEST AL,40H      ; S6=0 data received: test if data receive from printer
JNZ ACK
MOV DX,37AH              ;control reg
MOV AL,0AH       →1010   → C1=1 automatic LF after CR
OUT DX,AL
```

Example:

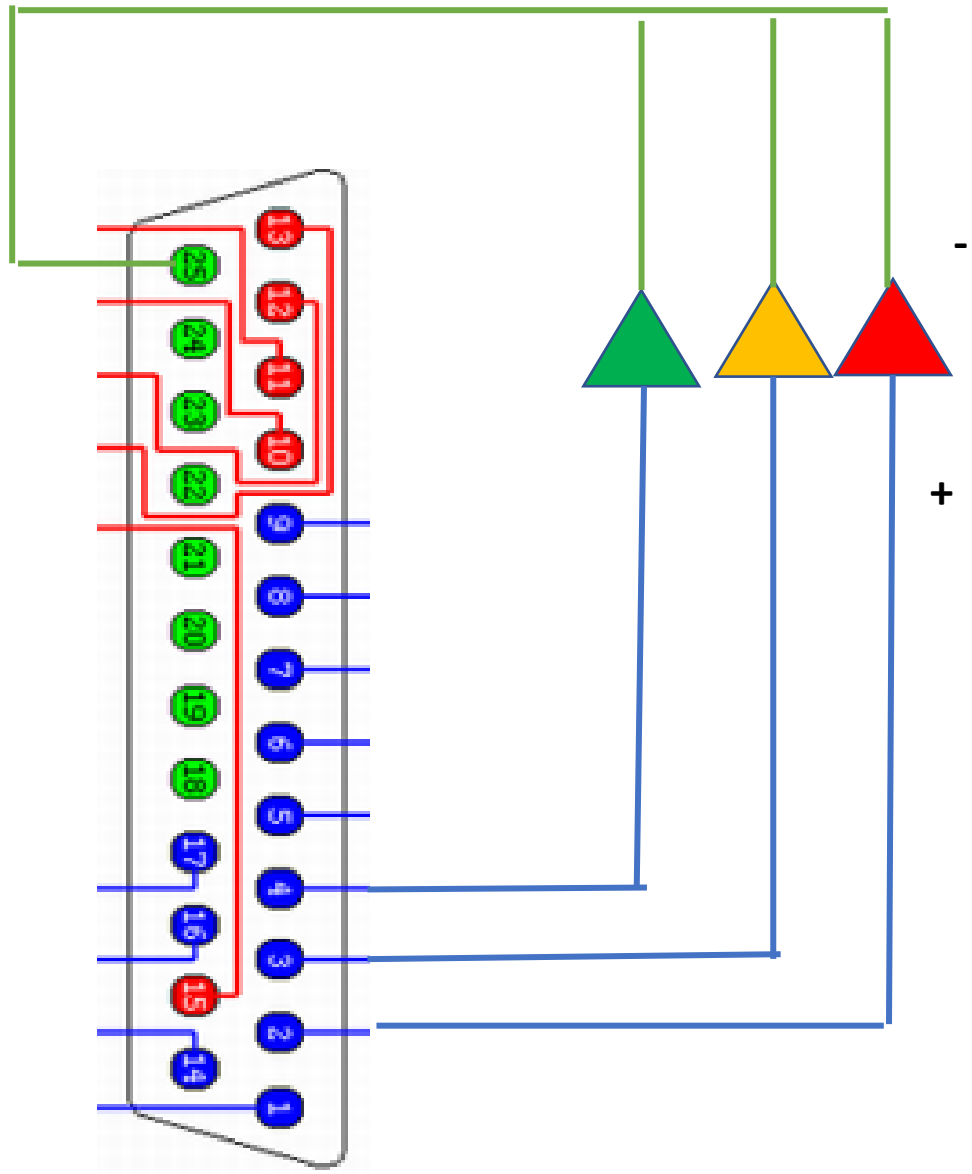
```
ORG 100H
MOV DX,378H
L:MOV AL,1
OUT DX,AL
 CALL DELEY
NOT AL
OUT DX,AL
CALL DELAY
JMP L
HLT
```



```
DELAY PROC
MOV BX,0FFFFH
L3: MOV CX,0FFFFH
L2:NOP
LOOP L2
DEC BX
JNZ L3
RET
DELAY ENDP
```

Example: traffic light

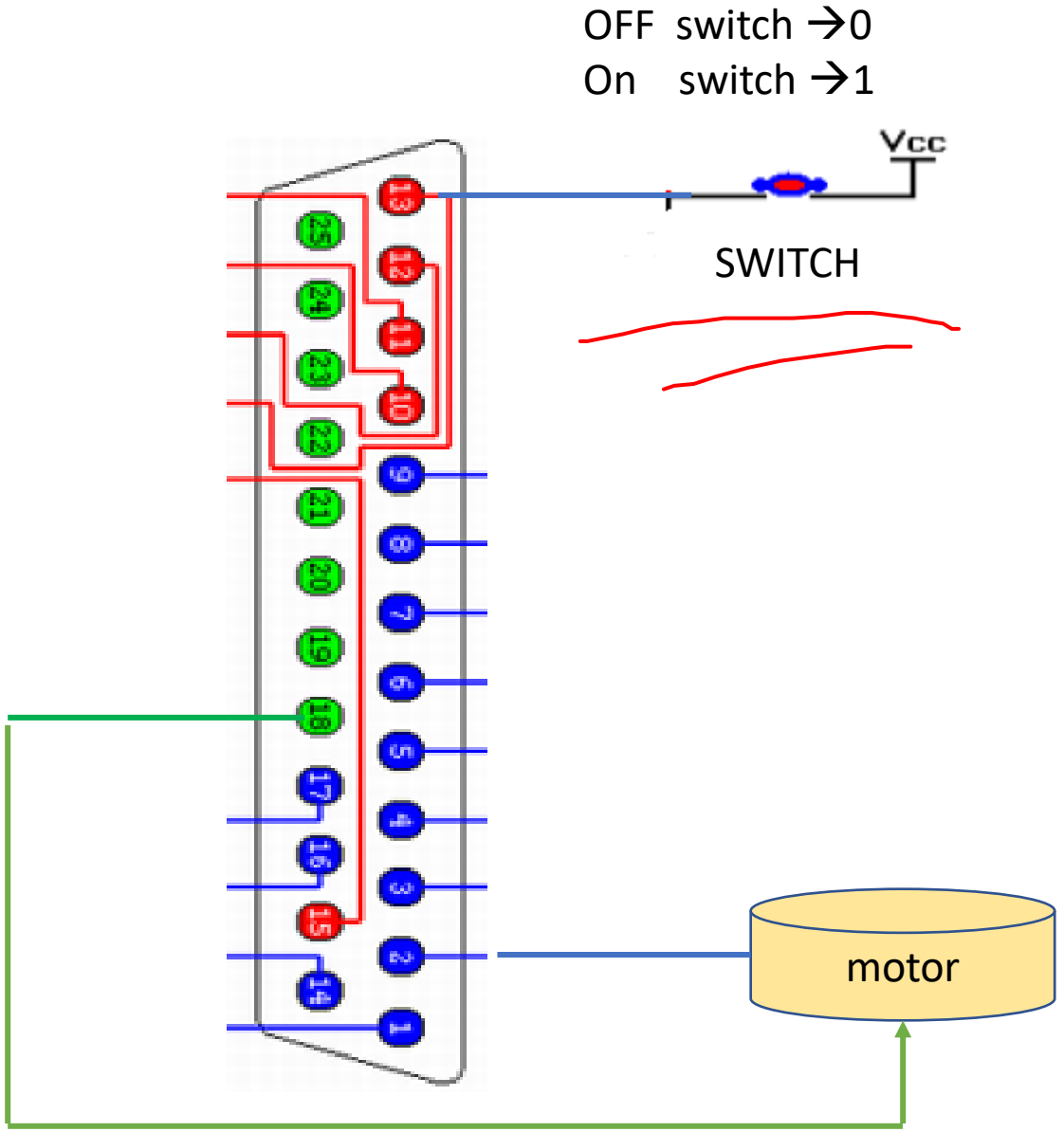
```
Org 100h
MOV DX,378H
ST: MOV SI,0
MOV CX,4
L: MOV AL,DATA [SI]
OUT DX,AL
CALL DELLAY
INC SI
 LOOP L
JMP ST
```



```
DATA DB 1,2,4,2
DELAY PROC
MOV BX,OFFFFH
L3:MOV CX,0FFFFH
L2: NOP
LOOP L2
DEC BX
JNZ L3
RET
DELAY ENDP
```

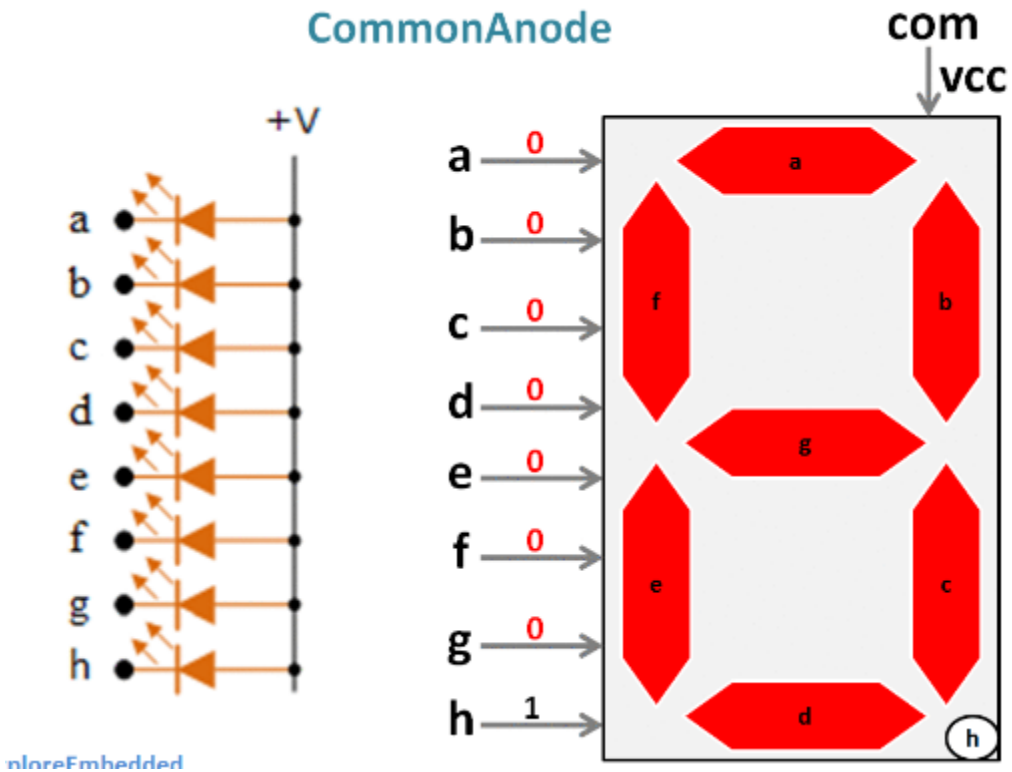
Example : step motor

```
START:
MOV DX,379H
L1:IN AL,DX
TEST AL,10      ;test switch s4 if  it is on
JZ L1
MOV DX,378H
MOV AL,1            ;on the motor
OUT DX,AL
MOV DX,379H
L2:IN AL,DX
TEST AL,10H     ;test switch s4 if  it is off-
```

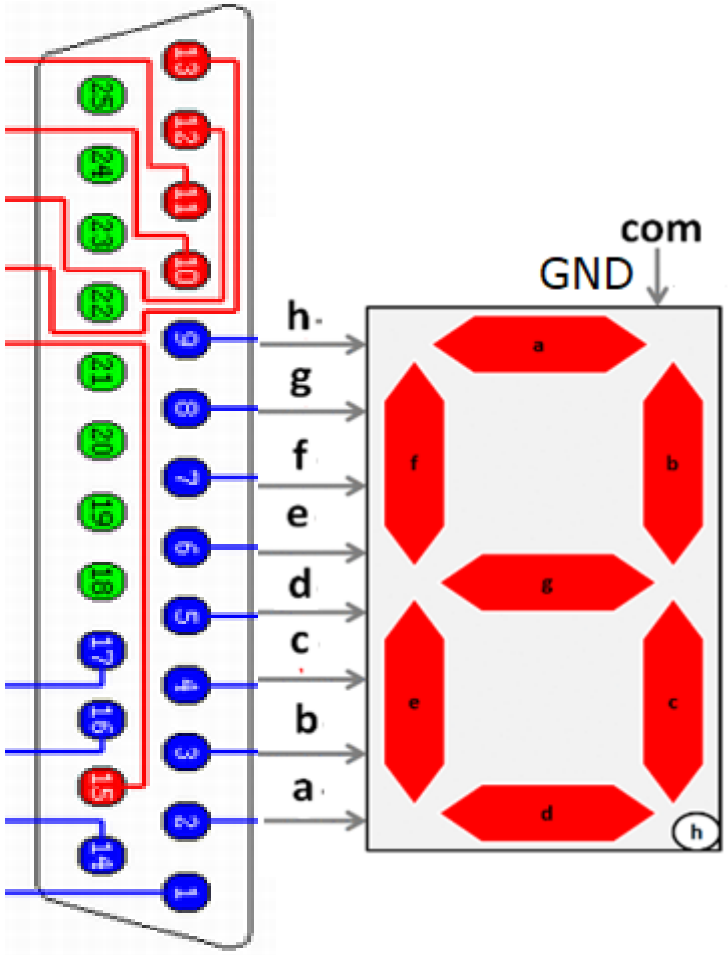


```
JNZ L2
MOV DX,378H
MOV AL,0        ;off  the motor
OUT DX,AL
JMP START
HLT
```

7-segment display (common anode)



Connect 7-segment display directly with parallel port



Cathode 7-seg table

		D7	D6	D5	D4	D3	D2	D1	D0
s	BCD	G	F	E	D	C	B	A	
0	0000	0	1	1	1	1	1	1	
1	0001	0	0	0	0	1	1	0	
2	0010	1	0	1	1	0	1	1	
3	0011	1	0	0	1	1	1	1	
4	0100	1	1	0	0	1	1	0	
5	0101	1	1	0	1	1	0	1	
6	0110	1	1	1	1	1	0	1	
7	0111	0	0	0	0	1	1	1	
8	1000	1	1	1	1	1	1	1	
9	1001	1	1	0	1	1	1	1	

DATA DB 3FH,06H,B7H,4FH,66H .....

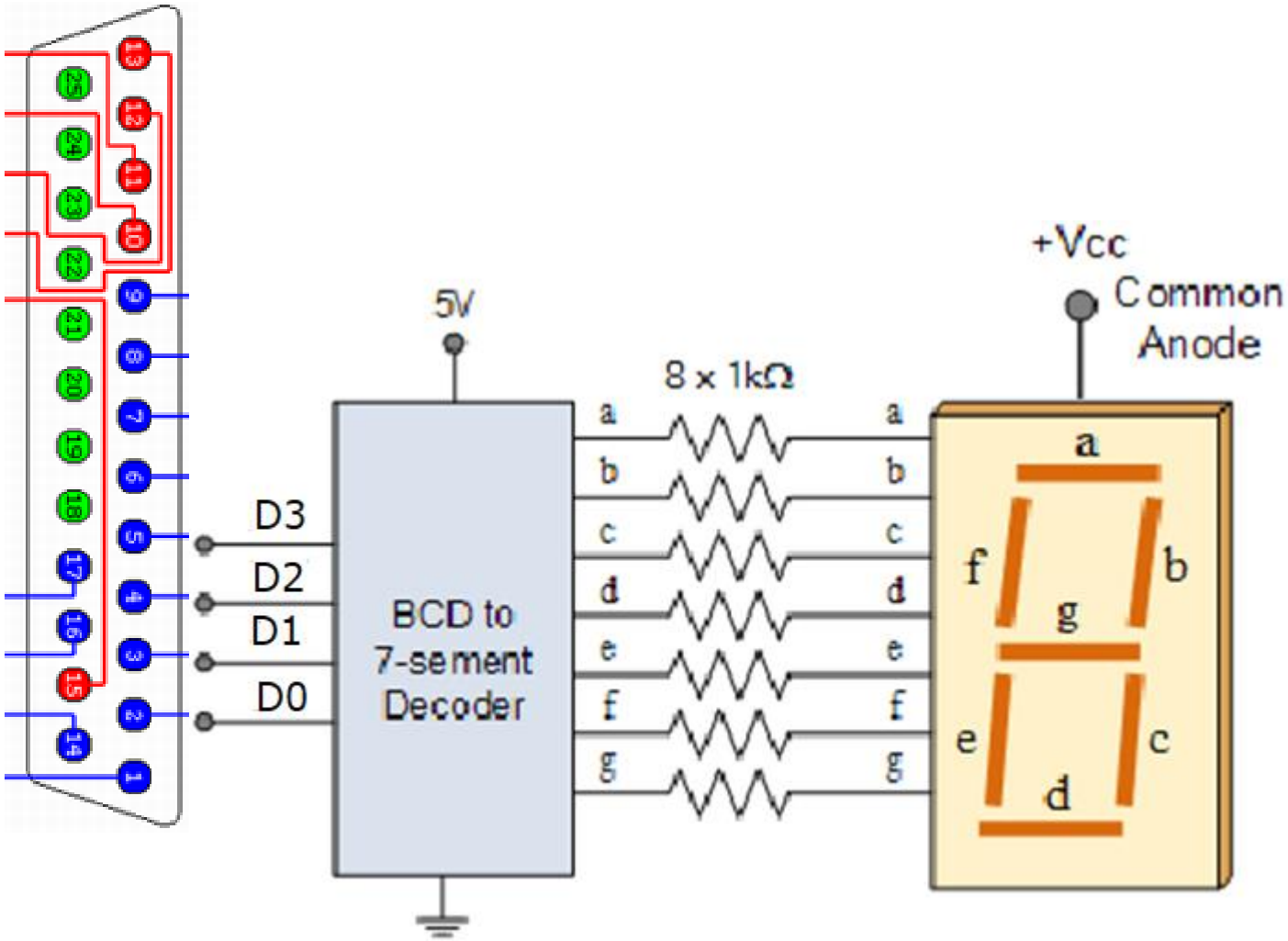
Example:

To display 0

X DB 3FH → cathode

X DB C0h → anode

Connect 7-segment display with parallel port through 7447 decoder



s	BCD	G	F	E	D	C	B	A
0	0000	0	1	1	1	1	1	1
1	0001	0	0	0	0	1	1	0
2	0010	1	0	1	1	0	1	1
3	0011	1	0	0	1	1	1	1
4	0100	1	1	0	0	1	1	0
5	0101	1	1	0	1	1	0	1
6	0110	1	1	1	1	1	0	1
7	0111	0	0	0	0	1	1	1
8	1000	1	1	1	1	1	1	1
9	1001	1	1	0	1	1	1	1

Example :

To Display 0 →use BCD

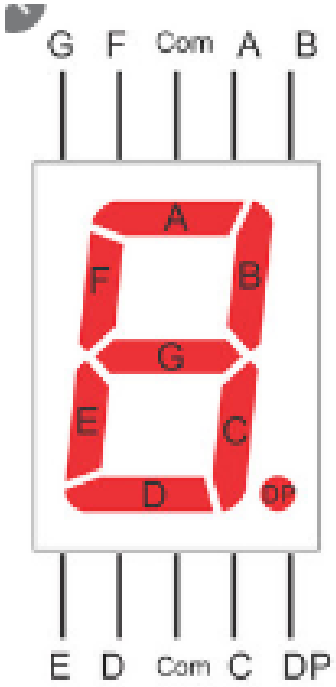
X DB 0

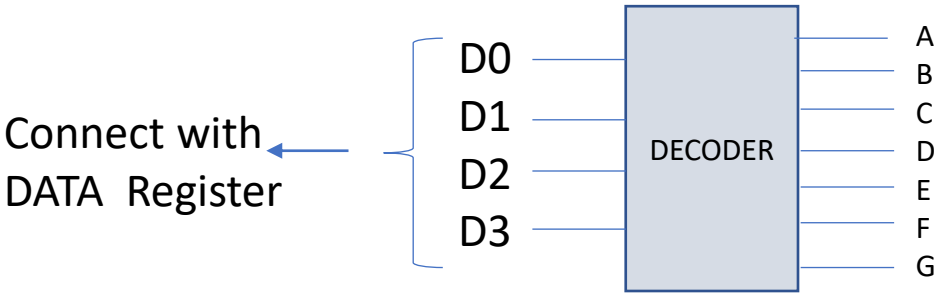
Example: display the numbers from 0-9 using the 7-segment display

```
ORG 100H
ST:MOV SI,0
MOV CX,10
MOV  DX,378H
L1:MOV AL,DATA[SI]
OUT DX,AL
CALL DELAY
INC SI
Loop L
JMP ST
DATA  DB   0,1,2,3,4,5,6,7,8,9
```

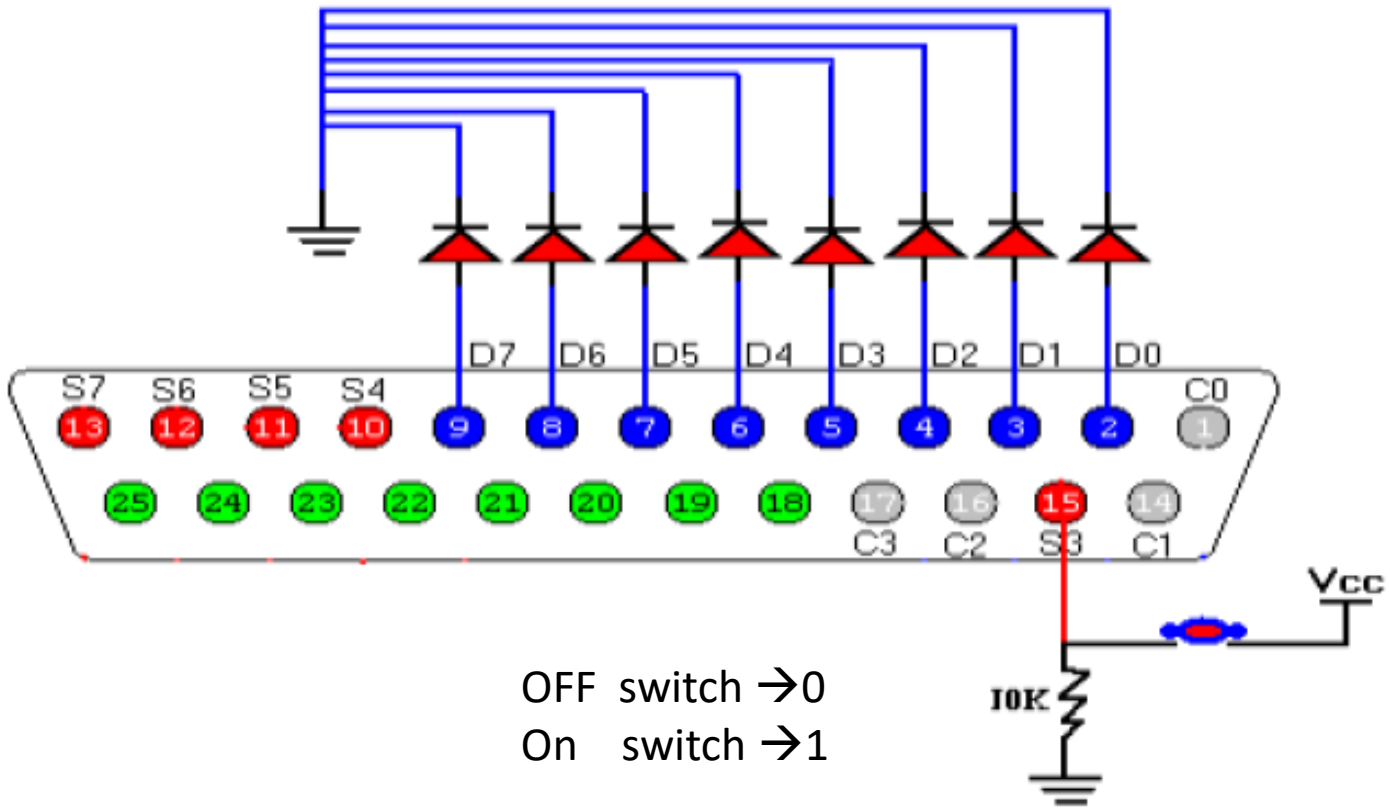
$D_7D_6D_5\,D_4D_3\,D_2D_1D_0$

s	BCD	G	F	E	D	C	B	A
0	0000	0	1	1	1	1	1	1
1	0001	0	0	0	0	1	1	0
2	0010	1	0	1	1	0	1	1
3	0011	1	0	0	1	1	1	1
4	0100	1	1	0	0	1	1	0
5	0101	1	1	0	1	1	0	1
6	0110	1	1	1	1	1	0	1
7	0111	0	0	0	0	1	1	1
8	1000	1	1	1	1	1	1	1
9	1001	1	1	0	1	1	1	1





- Connect 8 LEDs as in part 1 and connect a switch as shown in the following figure
- Write an assembly language program that does the following
    a. When switch is ON LEDs lights ON
    b. When switch is OFF LEDs lights OFF



OFF switch →0
On switch →1

```
L1: MOV DX,379H
IN AL,DX
TEST AL,00001000B
JNZ LIGHT
MOV AL,00000000B
JMP END1
LIGHT: MOV AL,11111111B
END1: MOV DX,378H
OUT DX,AL
END: JMP L1
```

Microprocessor System

Procedures

Unconditional transfer instruction

JMP instruction:
- Perform unconditional jump.

INT instruction:
- Perform interrupt operations.

RET instruction:
- Return from subroutines (procedure).

CALL instruction:
Call a procedure.

Procedures

Definition of procedure

- A procedure is a collection of instructions to which we can direct the flow of our program, and once the execution of these instructions is over control is given back to the next line to process of the code which called on the procedure.

- At the time of invoking a procedure the address of the next instruction of the program is kept on the stack so that, once the flow of the program has been transferred and the procedure is done, one can return to the next line of the original program, the one which called the procedure.

-With procedure it possible to break a large program down into smaller pieces so that each piece can be shown to work correctly without reference to any others.

- When a subroutine is called up, the address of the instruction following the call is saved on a stack until the RET instruction in the body of the subroutine is encountered. Then execution continues from the instruction following the call by retrieving the return address from that stack.

Syntax of a Procedure

- There are two types of procedures, the intra segments, which are found on the same segment of instructions, and the inter-segments which can be stored on different memory segments.

- When the intra-segment procedures are used, the value of IP is stored on the stack and when the intra-segments are used the value of CS:IP is stored.

CALL instruction:

The general format for CALL instruction is:

CALL procedure_name

- Subroutines (procedures) are defined after the end of code segment.
- There are two types of procedure: near and far

Procedure

- The part which make a procedure are:

1. Declaration of the procedure
2. Code of the procedure
3. Return directive
4. Termination of the procedure

Calling

- The calling of this procedure is done the following way:

Call Adding

- Note: Procedure is located at some specific address in memory, and if you use the same procedure 100 times, the CPU will transfer control to this part of the memory. The control will be returned back to the program by RET instruction. The stack is used to keep the return address.

```
org 100h


Call name

HLT
Name PROC                ;  Declaration Of The Procedure
..
..                        ; Content Of The Procedure
..
RET                       ; Return Directive
NAME ENDP                 ; End Of Procedure Declaration
```

There are two types of procedure: near and far

Name PROC NEAR

..

..

..

RET

NAME ENDP

Name PROC FAR

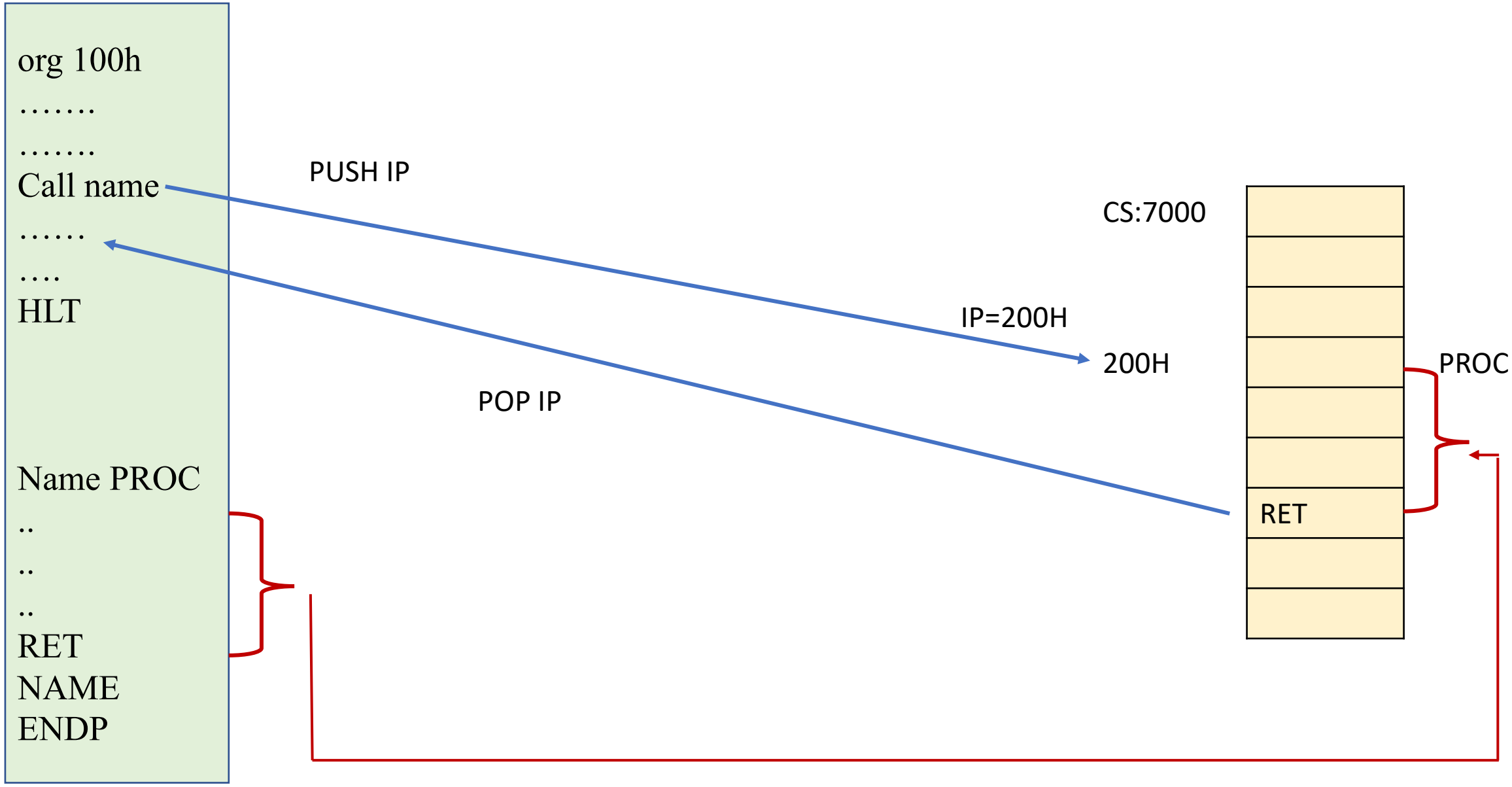
..

..

..

RET

NAME ENDP


org 100h
…….
…….
Call name
……
….
HLT
Name PROC
..
..
..
RET
NAME
ENDP
PUSH IP
POP IP
IP=200H
200H
CS:7000
RET
PROC

```
org 100h

 call r
 mov bl,al

Hlt

  x db 5
r PROC
mov al,x
RET
r ENDP
```

Example

- if we want a routine which adds two bytes stored in AH and AL each one, and keep the addition in the BX register:

```
ADDING PROC
MOV BX, 0
ADD AL, AH
MOV AH, 00
MOV BX, AX
RET
ADDING ENDP
```

```
ORG  100H
MOV AX,-3
MOV BX,4
CALL M1
CALL M1
HLT
M1 PROC
ADD BX,AX
INC AX
RET
M1 ENDP
```

AX	BX
FFFD	0004
FFF6	0001
FFFF	FFFF

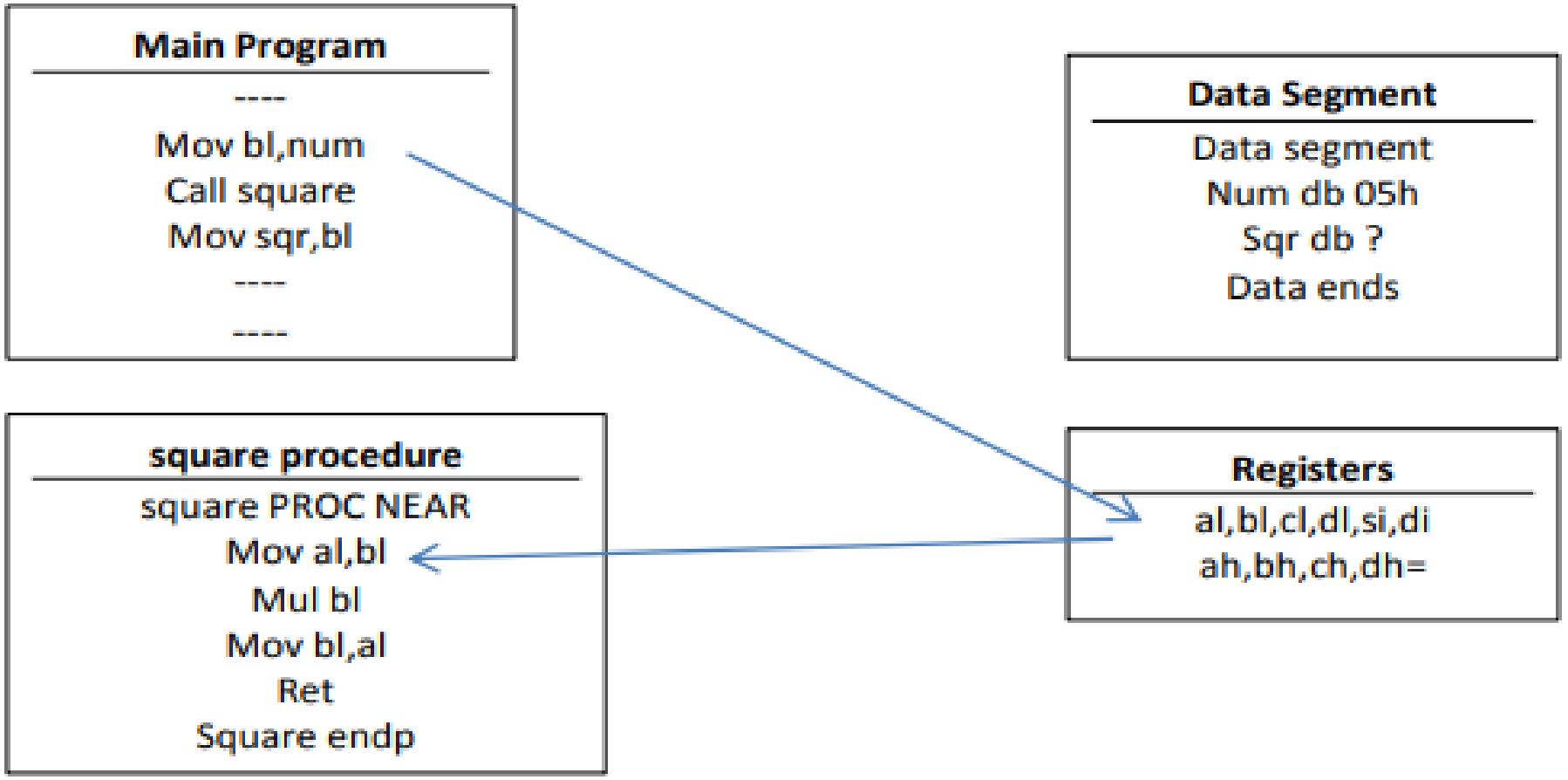
Passing Parameter

- Passing Parameter to procedure and from procedure :

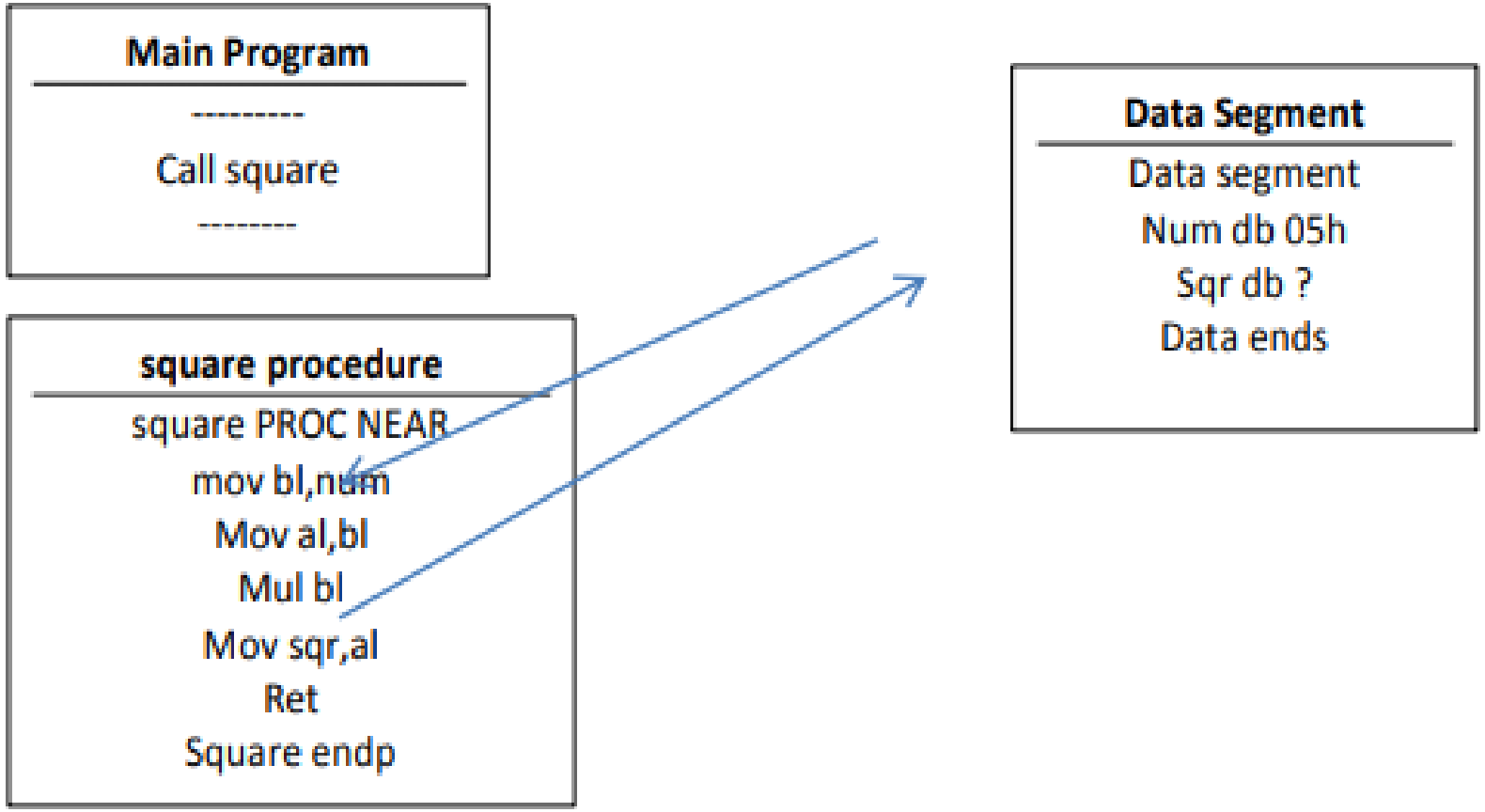
There are four major ways of passing parameters to and from a procedure.

1) In register
2) In dedicated memory locations accessed by name
3) With pointer passed in register.
4) With the stack

1) Passing parameters in registers : The main program can pass upto 6 parameters to the procedure through the registers AX,BX,CX,DX,SI & DI before executing the call instruction.
e.g. consider the program to calculate a square of given number.

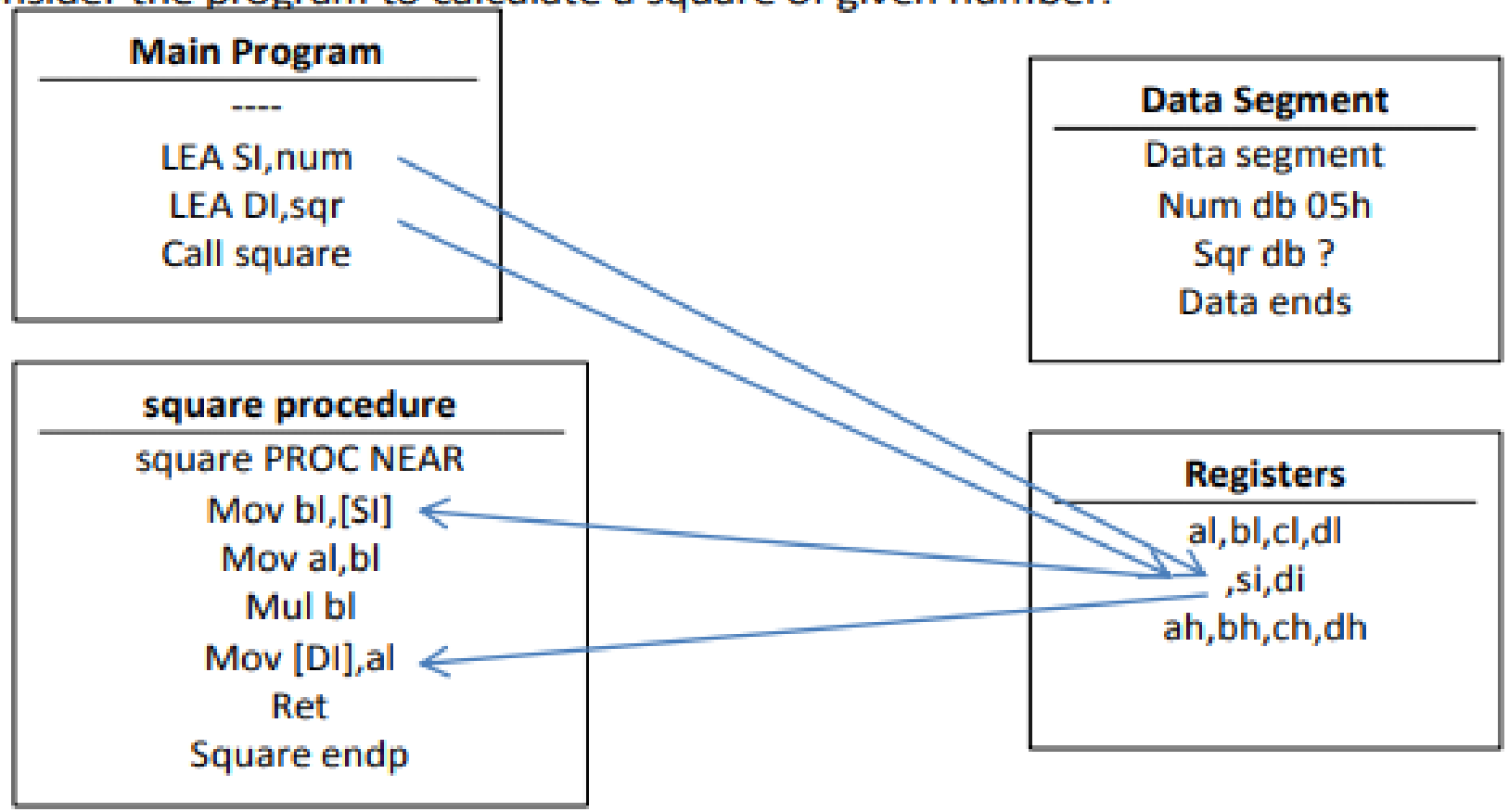


2) **Passing parameters in dedicated memory locations accessed by name :** when large number of parameters is to be passed to the procedure, then these parameters can be placed in an argument list as dedicated memory locations in one of the data segment from memory.
e.g. consider the program to calculate a square of given number.

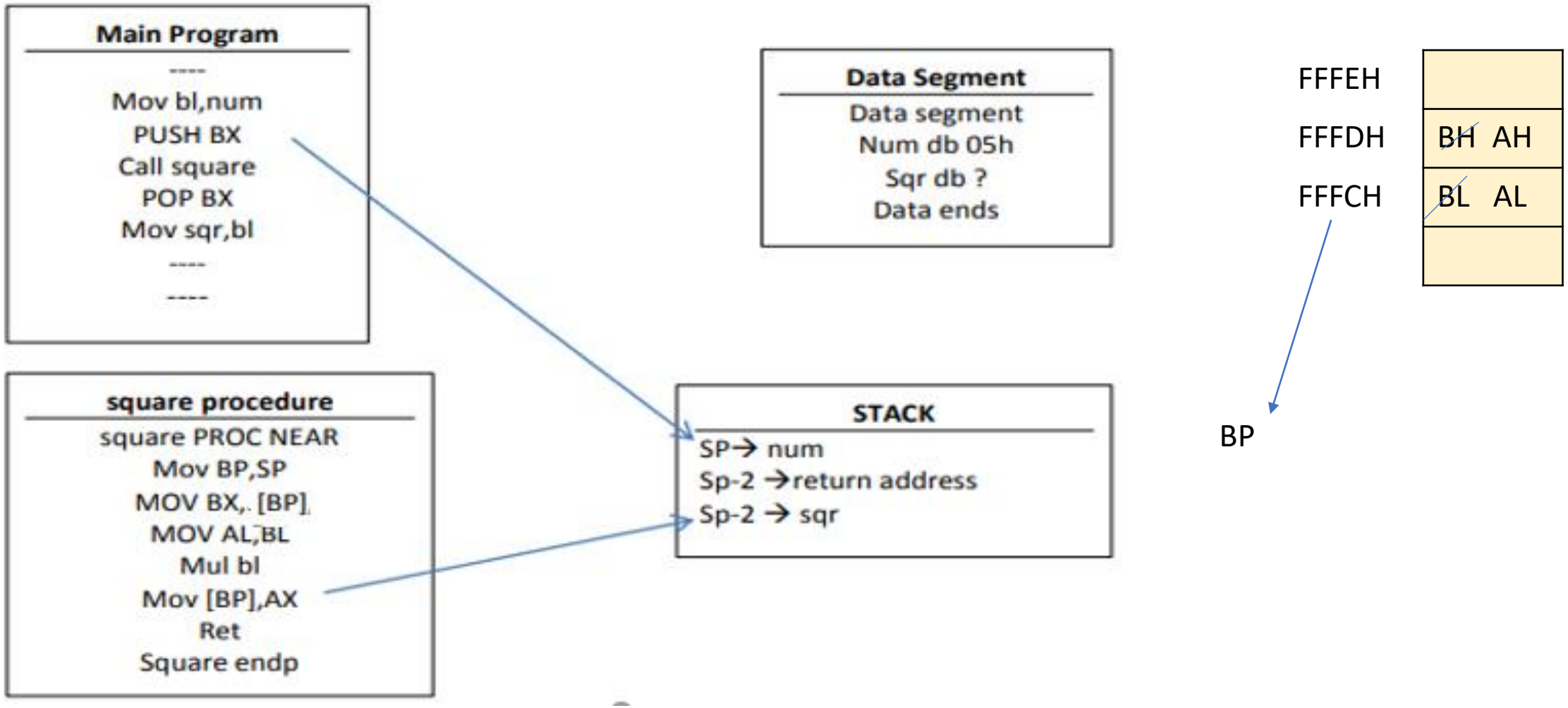


3) **Passing parameters with Pointer :** In the main program, before we call the procedure we can set up SI as a pointer to pass values to procedures and set DI as pointer to receive values from procedures.

e.g. consider the program to calculate a square of given number.



4) Passing parameters with stack : Alternate method of passing large number of parameters is to push the parameters on the stack in the main program before we call the procedure.
e.g. consider the program to calculate a square of given number.



Advantages and Disadvantages of using procedure : Advantages :

1) Allows to save memory space.
2) Program development becomes easier.
3) Debugging of errors in program become easy.
4) Reduced size of program
5) Reusability of procedure.

Disadvantages :

1) CALL and RET instructions are always required to integrate with procedures.
2) Requires the extra time to link procedure and return from it.
3) For small group of instructions, linking and returning back time more than the execution time, hence for small group of instructions procedures cannot be preffered.

Examples

```
ORG 100H
MOV CX, 2
L:    CALL SUM
LOOP L
MOV Y, AL
HLT
X DB 3
Y DB ?                ;OUTPUT : Y=6
SUM PROC NEAR
                      MOV AH, X
                      ADD AL, AH
                       RET
 SUM ENDP
```

```
ORG 100H
MOV CX, 2
L:    CALL SUM
LOOP L
MOV Y, AL             ;OUTPUT : Y=6

HLT
SUM PROC NEAR
                      MOV AH, X
                      ADD AL, AH
                       RET
 SUM ENDP
X DB 3
Y DB ?
```

```
ORG 100H
MOV AH, 2
LEA SI, STR + 19
MOV CX, 20
L1: MOV AL,[SI]
DEC  SI
CALL MAC
      LOOP L1
HLT

MAC PROC
      MOV DL, AL
      INT 21H
      RET
MAC ENDP

STR DB 'ASSEMBLY PROGRAMMING'
```



Microprocessor System

Processor Control Instructions:

- Processor control instruction can be divided into three classes:

1- Flag operations.

2- No operations.

3- Processor status operations.

Flag operations:

STC instruction:

- Set Carry flag (CF=1).
- The general format for STC is:

STC (no operands)

CLC instruction:

- Clear Carry flag (CF=0).
- The general format for CLC is:

CLC (no operands)

CMC instruction:

- Complement Carry flag. Inverts value of CF.
- The general format for CMC is:

CMC (no operands)

CLD instruction:

- Clear Direction flag (DF=0). SI and DI will be incremented by string instructions (process strings from left to right).
- The general format for CLD is:

CLD (no operands)

STD instruction:

- Set Direction flag (DF=1). SI and DI will be decremented by string instructions (process strings from right to left).
- The general format for STD is:

STD (no operands)

STI instruction:

- Set Interrupt flag (IF=1). This enables hardware interrupts.
- The general format for STI is:

STI (no operands)

CLI instruction:

- Clear Interrupt flag IF=0). This disables hardware interrupts (microprocessor will ignore any external request).
- Used before instruction that control time critical process.
- The general format for CLI is:

CLI (no operands)

No operations:

No Operation. (Do nothing).

- Used as delay operation.
- The general format for NOP is:

NOP (no operands)

Example:

```
    ; do nothing, 3 times:
NOP
NOP
NOP
RET
```

HLT instruction:

- Halt the System. (make the microprocessor idle)
- The general format for HLT is:

HLT (no operands)

Example:

```
MOV AX, 5
HLT
```

WAIT instruction:

- Halt the System (make the microprocessor idle) and check the IF every 5-clock pluses.
- Used to make compatibility between the microprocessor and the peripheral devices. (for example to resume the microprocessor work after receiving a signal from the printer of completed printed job)
- The general format for WAIT is:

WAIT (no operands)

Example:

```
MOV AX, 5
WAIT
```

Microprocessor system

Serial Port

Serial Port

- Serial port considered to be one of the most basic external connections to a computer, the **serial port** has been an integral part of most computers

- most modems still use the serial port, as do some printers, PDAs and digital cameras. Few computers have more than two serial ports.

- Essentially, serial ports provide a standard connector and protocol to let you attach devices, such as modems, to your computer.



- The name "serial" comes from the fact that a serial port "serializes" data. That is, it takes a byte of data and transmits the 8 bits in the byte one at a time.

- The advantage is that a serial port needs only one wire to transmit the 8 bits (while a parallel port needs 8).

- The disadvantage is that it takes 8 times longer to transmit the data than it would if there were 8 wires. Serial ports lower cable costs and make cables smaller.

- Before each byte of data, a serial port sends a **start bit,** which is a single bit with a value of 0. After each byte of data, it sends a **stop bit** to signal that the byte is complete. It may also send a **parity bit.**

- Serial ports, also called **communication (COM) ports**, are **bi-directional**. Bidirectional communication allows each device to receive data as well as transmit it.

- Serial devices use different pins to receive and transmit data -- using the same pins would limit communication to **half-duplex**, meaning that information could only travel in one direction at a time.

- Using different pins allows for **full-duplex** communication, in which information can travel in both directions at once.

UART Needed

- Serial ports rely on a special controller chip, the **Universal Asynchronous Receiver/Transmitter (UART)**, to function properly.

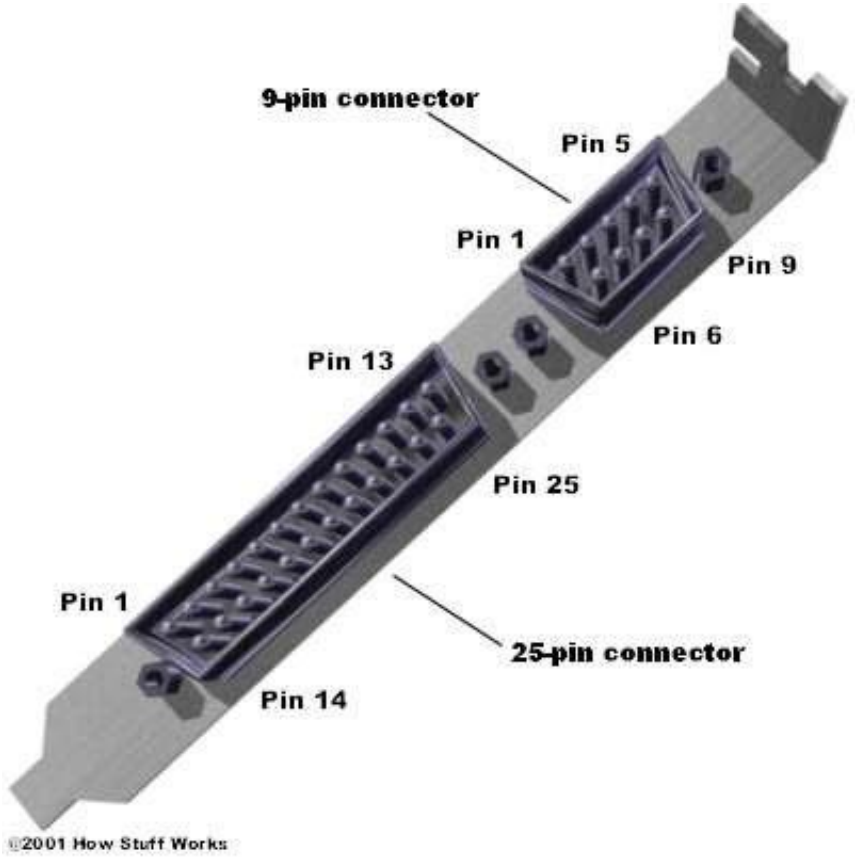
- The UART chip takes the parallel output of the computer's system bus and transforms it into serial form for transmission through the serial port.

- In order to function faster, most UART chips have a built-in buffer of anywhere from 16 to 64 kilobytes. This buffer allows the chip to cache data coming in from the system bus while it is processing data going out to the serial port.

- While most standard serial ports have a maximum transfer rate of 115 Kbps (kilobits per second), high speed serial ports, such as **Enhanced Serial Port (ESP)** and **Super Enhanced Serial Port (Super ESP)**, can reach data transfer rates of 460 Kbps.

The Serial Connection

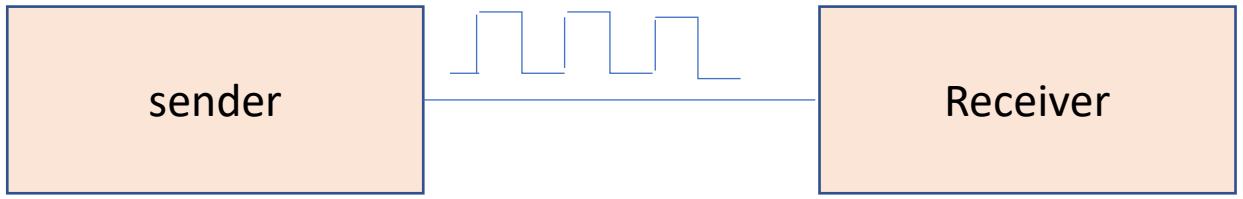
- The external connector for a serial port can be either 9 pins or 25 pins. Originally, the primary use of a serial port was to connect a modem to your computer. The pin assignments reflect that.



Pins functions when a modem is connected:

9-pin connector:

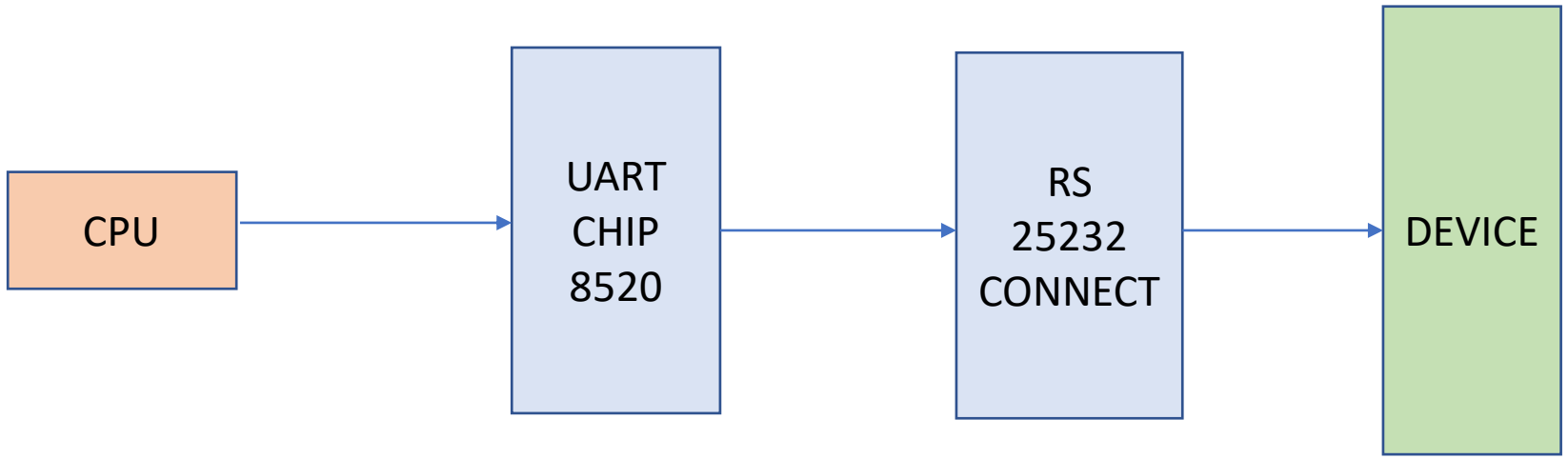
1. **Carrier Detect** - Determines if the modem is connected to a working phone line.
2. **Receive Data** - Computer receives information sent from the modem.
3. **Transmit Data** - Computer sends information to the modem.
4. **Data Terminal Ready** - Computer tells the modem that it is ready to talk.
5. **Signal Ground** - Pin is grounded.
6. **Data Set Ready** - Modem tells the computer that it is ready to talk.
7. **Request To Send** - Computer asks the modem if it can send information.
8. **Clear To Send** - Modem tells the computer that it can send information.
9. **Ring Indicator** - Once a call has been placed, computer acknowledges signal (sent from modem) that a ring is detected.



- Serial transition: synchronous

  asynchronous

Universal asynchronies receiver transmitter(UART)



Data format

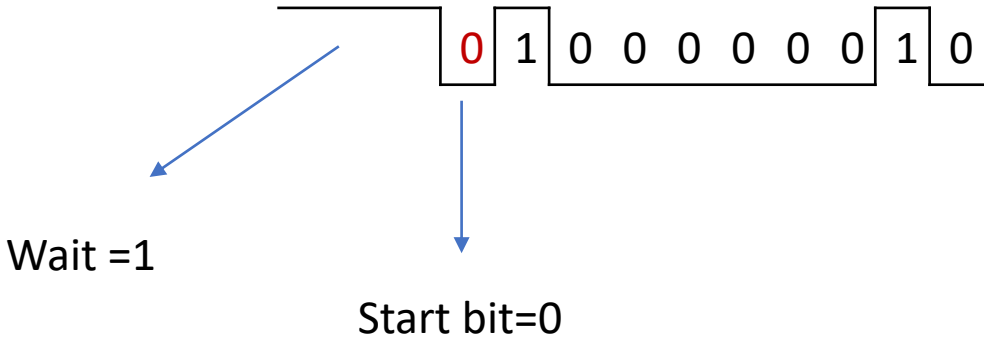
- How the data is packed

1. Start bit
2. Parity bit
3. Stop bit
4. # of data bits
5. Speed of transmission

Transmitter/receiver protocol

- Send char ‘A’=41h=0100 0001

Transition of char :Begin from right to left



- Parity system for check error in transition
- two types of parity system:

1- even parity : number of ones is even

2-odd parity : number of ones is odd

Example :

Send ‘B’=0100 0010

Use odd parity :

Transmit : 1 0100 0010

Receive : 1 0100 0011 → error

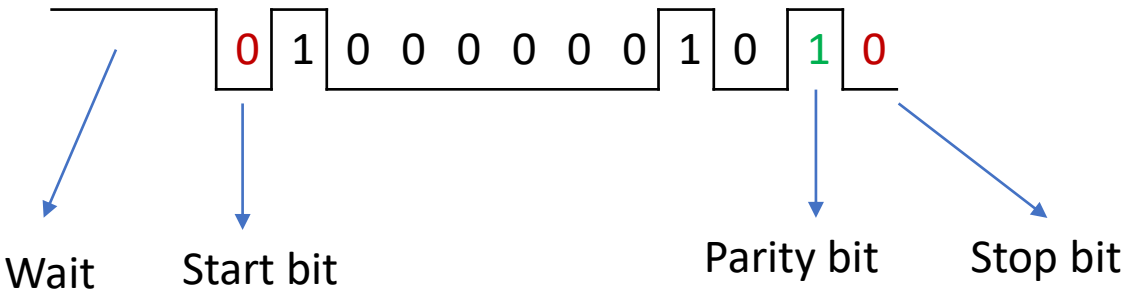
a) Data format:

1) 1 start bit
2) 8 data bit
3) Odd parity
4) 1 stop bit

Example:

‘A’=0100 0001

0 1 01 00 00010



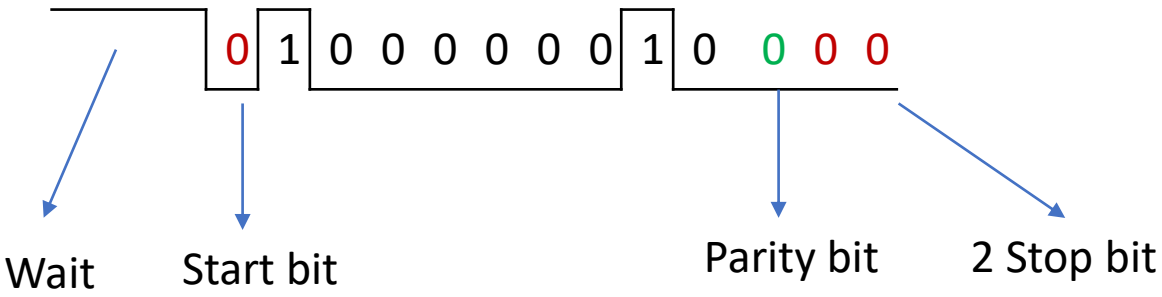
b) Data format:

1) 1 start bit
2) 8 data bit
3) even parity
4) 2 stop bit

Example:

'A'=0100 0001

0 0 0 01 00 00010



Transmitting speed

Baud rate =bit per second

Example: if the device can transmit 10 char per second find the baud rate

Data format:

1 start

7 data

1 parity

2 stop

Sum: 11 bit

Baud rate =10*11=110 bps

-Speed : 300 bps → find the number of chars that transmitted per second

Data format:

1 start

7 data

1 parity

1 stop

Sum: 10 bit

300/10=30 char/sec

Programming asynchronous serial interface

UART I/O port address	Register	Bit 7 line control register	Input/output
3F8h	Transmitter holding register Receiver data register Baud rate divisor (LSB)	0 0 1	Out In Out
3F9h	Baud rate divisor (MSB)	1	Out
3FBh	Line control register	-	Out
3FDh	Line status register	-	In

communications measured in bits per second.

The UART incorporates a divide by 16 counter which simply divides the incoming clock signal by 16 , therefore ;

$$\frac{1.8432\ \text{MHZ}}{16} = 115200\ \text{Hz}$$

This making the UART capable of transmitting and receiving at 115200 Bits Per Second (BPS).

For example we only wanted to communicate at 2400 BPS (Baud rate), then:

$$\text{Baud rate divisor} = \frac{115200}{\text{Baud rate}}$$

$$\text{Baud rate divisor} = \frac{115200}{2400} = 48\ (\text{decimal}) \qquad \text{---------- (1)}$$

Table 2

Speed (BPS)	Divisor (Decimal)	Divisor Latch High Byte DLM	Divisor Latch Low Byte DLL
2400	48	00_H	30_H

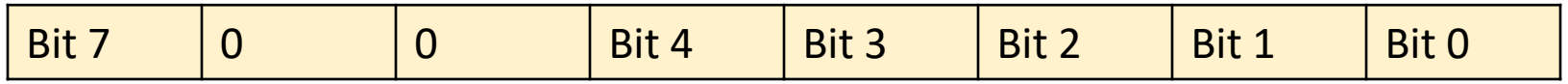
- Max speed =115200 bps (baud)

- Example: speed =300 baud

115200/300=384= 180h →MSB:LSB

- →3F9h:3F8h

Line control register



of data bit 00 5bit
01 6bit
10 7bit
11 8bit

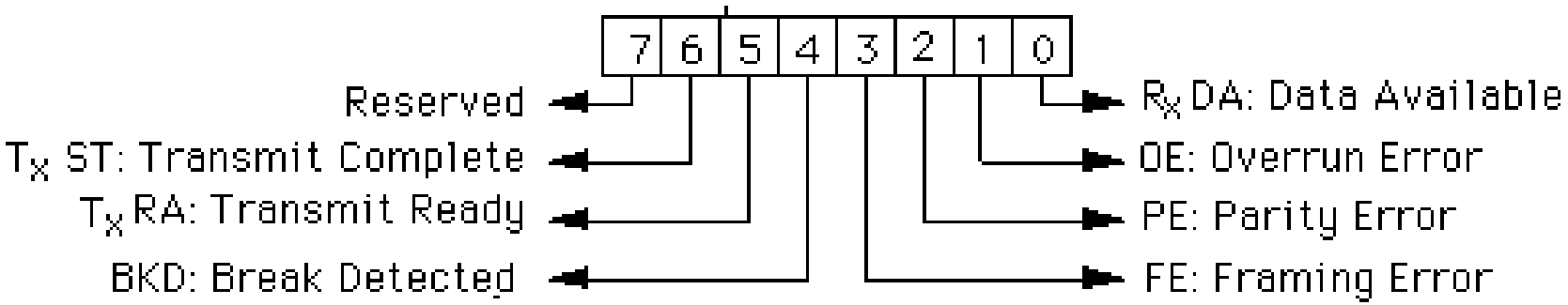
stop bit → 0:one stop bit
1: two stop bit

parity bit → 0: no parity
1: parity

parity Type → 0:odd
1:even

0: normal accessing transmitter and receiver
1: access baud rate divisor

Line status Register



LSR - Line Status Register (Address 5)

String Instructions

MOVSB

COPY byte at DS:[SI] to ES:[DI]. Update SI and DI.

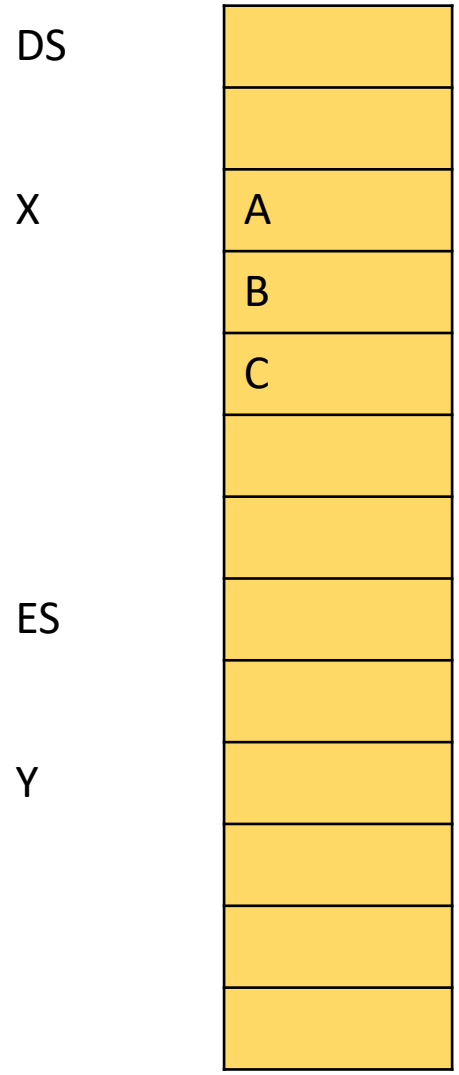
flags unchanged

Algorithm:

```
ES:[DI] = DS:[SI]
if DF = 0 then
SI = SI + 1
DI = DI + 1
else
SI = SI - 1
DI = DI - 1
```

```
X DB ‘ABC’
Y DB 3 DUP(?)
CLD
MOV SI,OFFSET X
MOV DI,OFFSET Y
MOV CX,3
L1: MOVSB
LOOP L1
```

CX	SI	DI
3	X	Y
2	X+1	Y+1
1	X+2	Y+2
0	X+3	Y+3



REP

Repeat following MOVSB, MOVSW, LODSB, LODSW, STOSB, STOSW instructions CX times.

Effect flag

Algorithm:

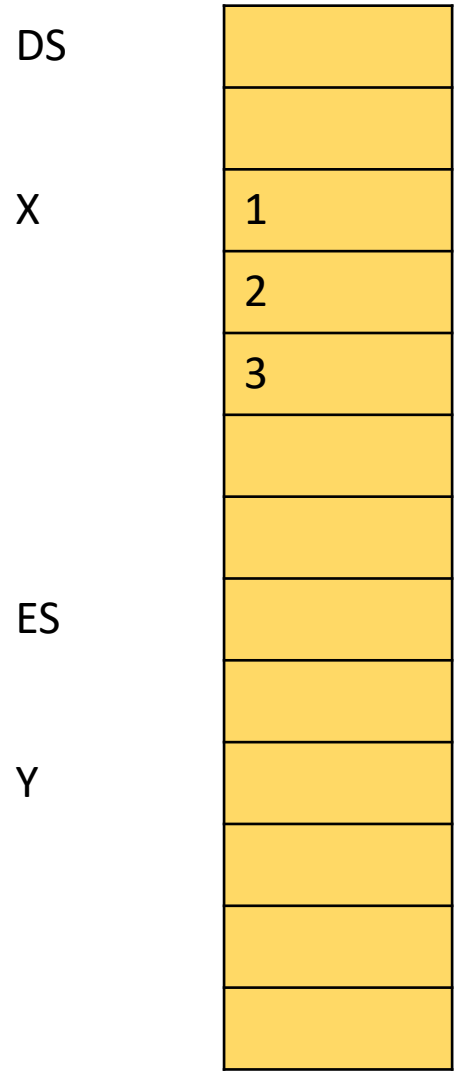
```
check_cx:

if CX <> 0 then
do following chain instruction
CX = CX - 1
go back to check_cx
Else exit from REP cycle
```

Example:

```
X DB 1,2,3
Y DB 3 DUP(0)
CLD
LEA SI, X
LEA DI, Y
MOV CX, 3
REP MOVSB
```

CX	SI	DI
3	X	Y
2	X+1	Y+1
1	X+2	Y+2
0	X+3	Y+3



```
X DB ‘ABC’
Y DB 3 DUP(?)
CLD
LEA  SI, X
LAE DI, Y
MOV CX,3
REP MOVSB
```

MOVSW

Copy word at DS:[SI] to ES:[DI]. Update SI and DI.

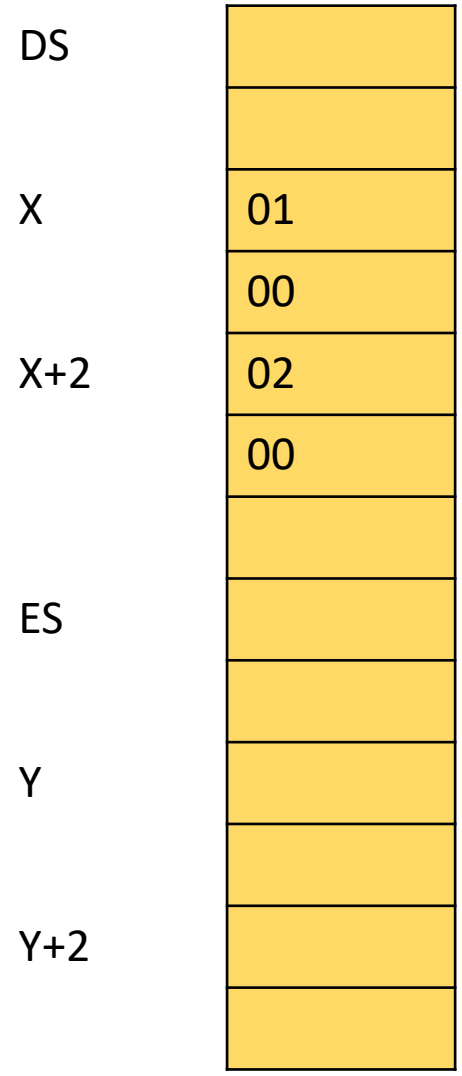
flags unchanged

Algorithm:

```
ES:[DI] = DS:[SI]
if DF = 0 then
SI = SI + 2
DI = DI + 2
else
SI = SI - 2
DI = DI - 2
```

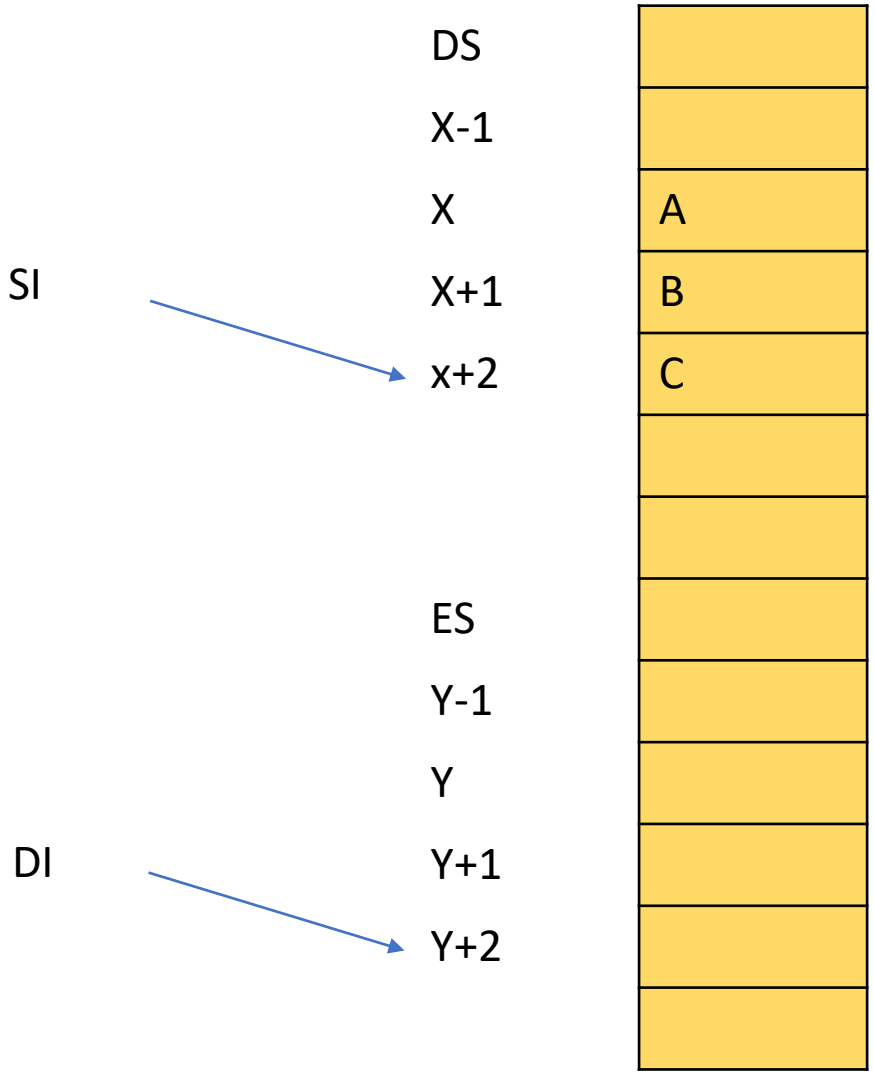
```
X DW 1,2
Y DW 2 DUP(0)
CLD
LEA SI, X
LEA DI, Y
MOV CX, 2
REP MOVSW
```

CXC	SI	DI
2	X	Y
1	X+2	Y+2
0	X+4	Y+4



```
X DB ‘ABC’
Y DB 3 DUP(?)
STD
LEA  SI, X+2
LAE DI, Y+2
MOV CX,3
REP MOVSB
```

CX	SI	DI
3	X+2	Y+2
2	X+1	Y+1
1	X	Y
0	X-1	Y-1



STOSB

Store byte in AL into ES:[DI]. Update DI.

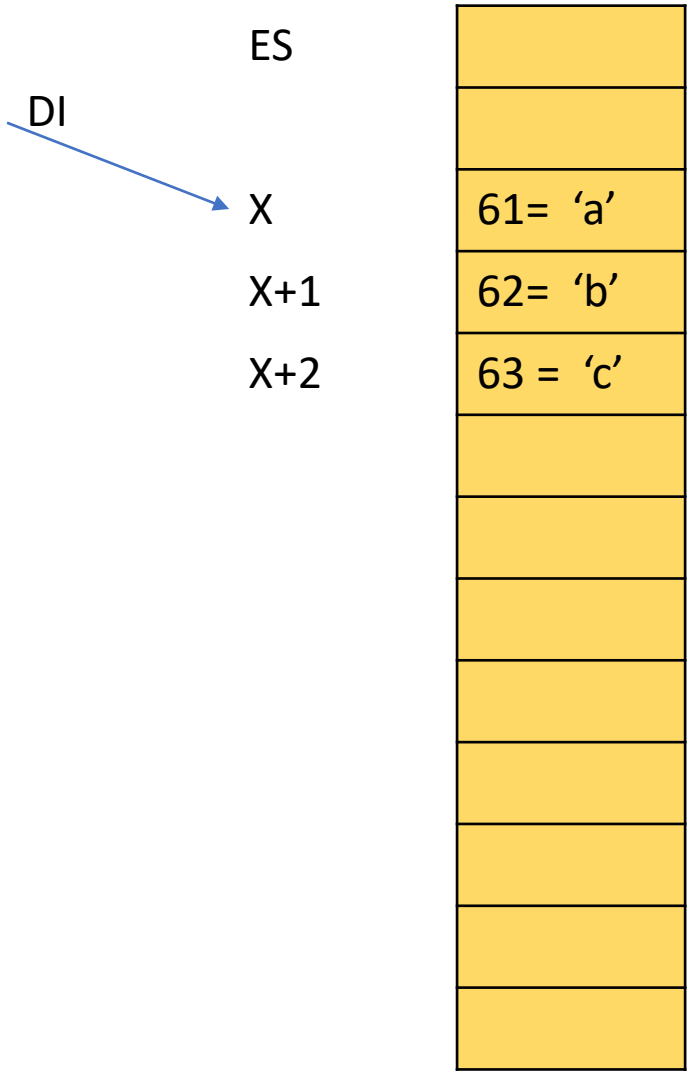
flags unchanged

Algorithm:

```
ES:[DI] = AL
if DF = 0 then
DI = DI + 1
else
DI = DI - 1
```

```
X DB  3 DUP(‘ ‘)
MOV AL,’a’
MOV CX,3
MOV DI,OFFSET X
L1: STOSB
INC AL
LOOP L1
```

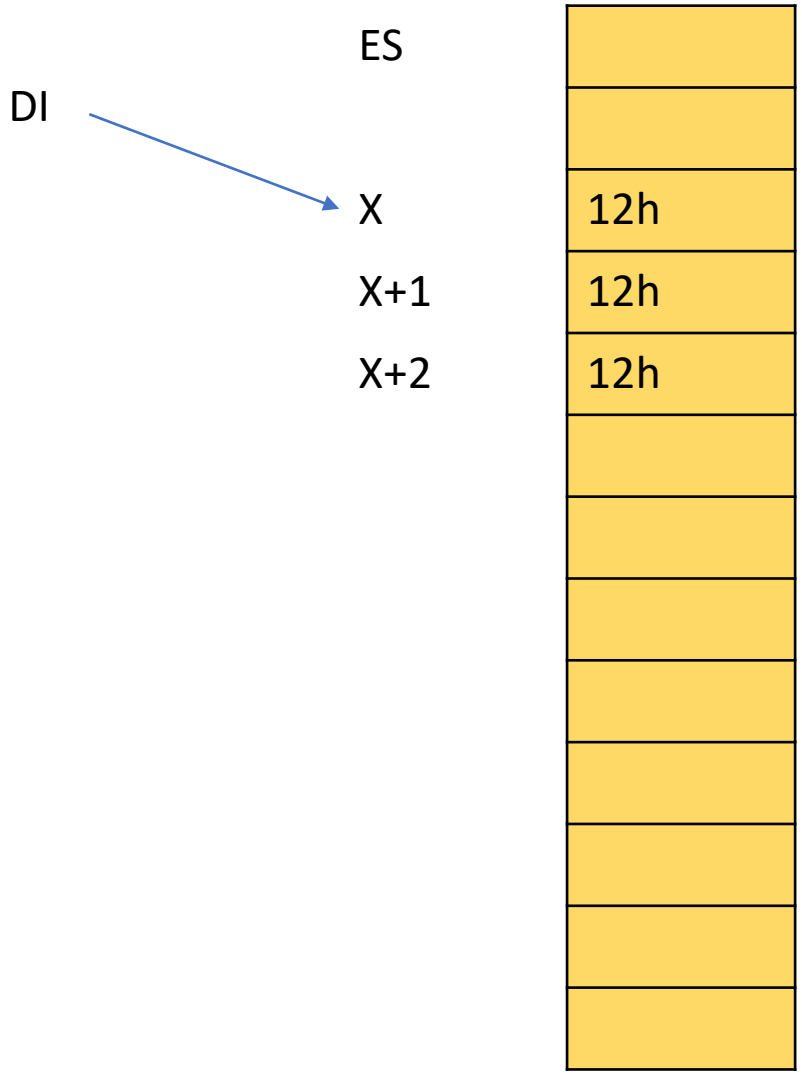
CX	DI	AL
3	X	‘a’
2	X+1	‘b’
1	X+2	‘c’
0	X+3	‘d’



```
X DB 3 dup(0)

LEA DI, X
MOV AL, 12h
MOV CX, 3
REP STOSB
```

CX	DI
3	X
2	X+1
1	X+2
0	X+3



STOSW

Store word in AX into ES:[DI]. Update DI.

flags Unchanged

Algorithm:

```
ES:[DI] = AX
if DF = 0 then
DI = DI + 2
else
DI = DI - 2
```

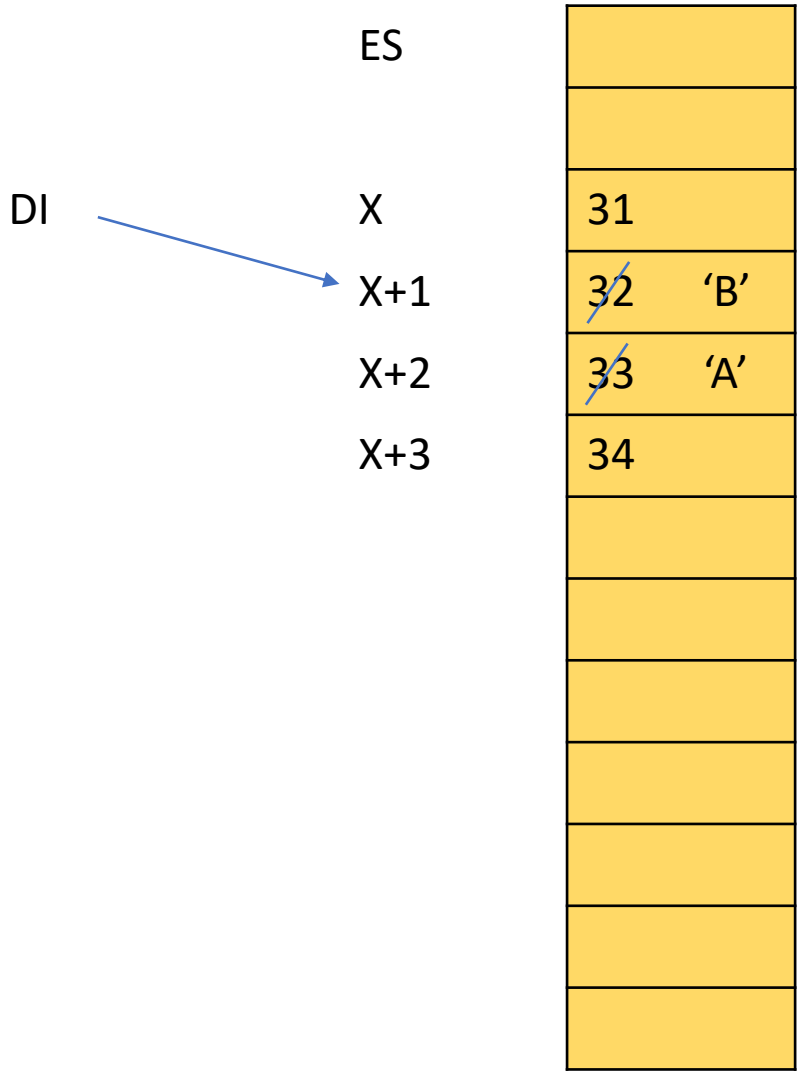
X DB ‘1234’

MOV AX,’AB’ AX=4142H

MOV DI,OFFSET X+1

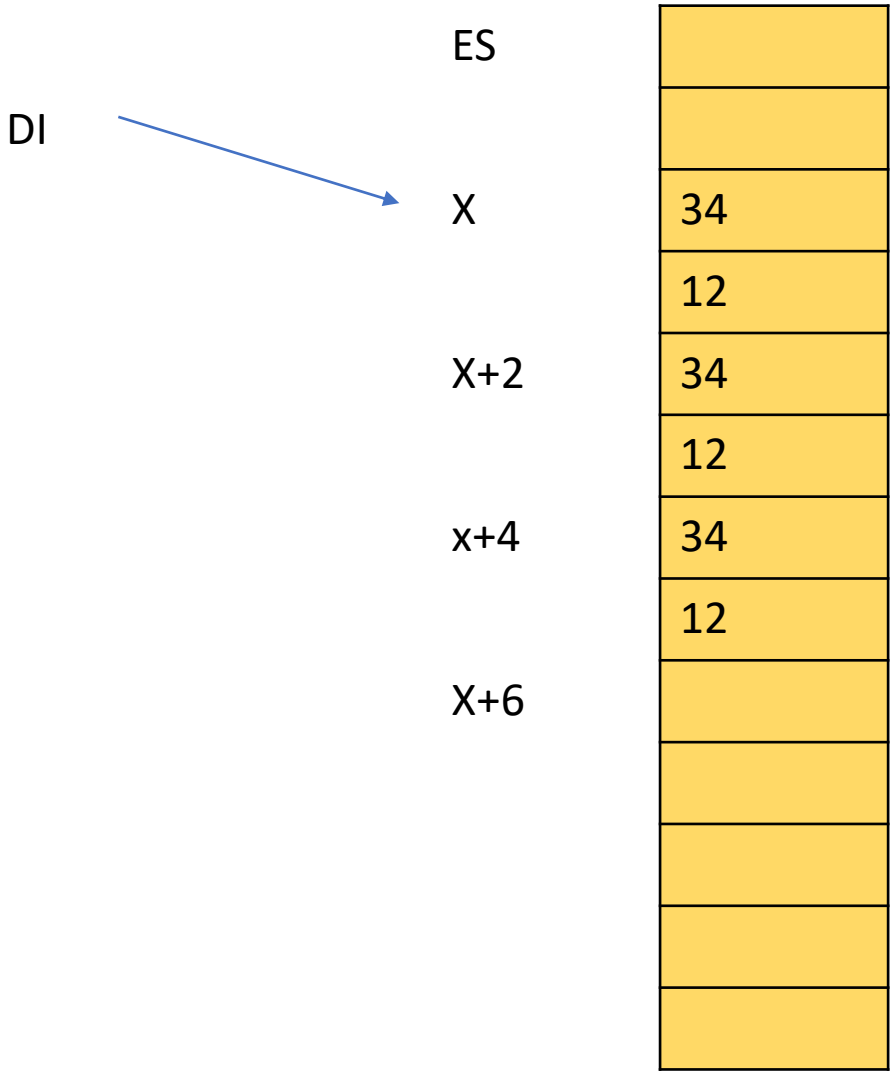
STOSW

MOV BL,[DI] BL=34H



```
X DW 3 dup(0)
LEA DI, X
MOV AX, 1234h
MOV CX, 3
REP STOSW
```

CX	DI
3	X
2	X+2
1	X+4
0	X+6



LODSB

Load byte at DS:[SI] into AL. Update SI.

flags Unchanged

Algorithm:

```
AL = DS:[SI]
if DF = 0 then
SI = SI + 1
else
SI = SI - 1
```

```
ORG 100h

LEA SI, a1
MOV CX, 5
MOV AH, 0Eh

m: LODSB
INT 10h
LOOP m

RET
a1 DB 'H', 'e', 'l', 'l', 'o'
```

LODSW

Load word at DS:[SI] into AX. Update SI.

flags Unchanged

Algorithm:

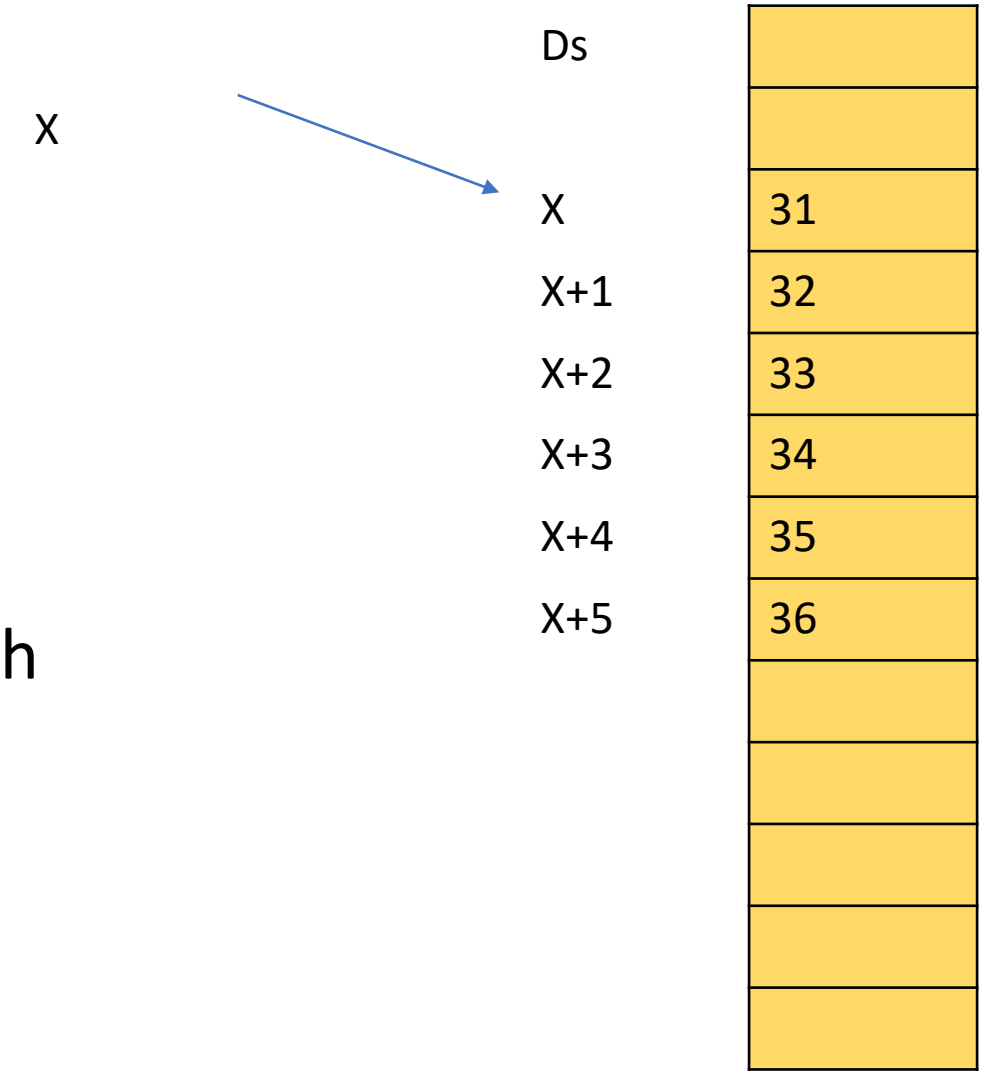
```
AX = DS:[SI]
if DF = 0 then
SI = SI + 2
else
SI = SI - 2
```

X DB ‘123456’

MOV SI,OFFSET X

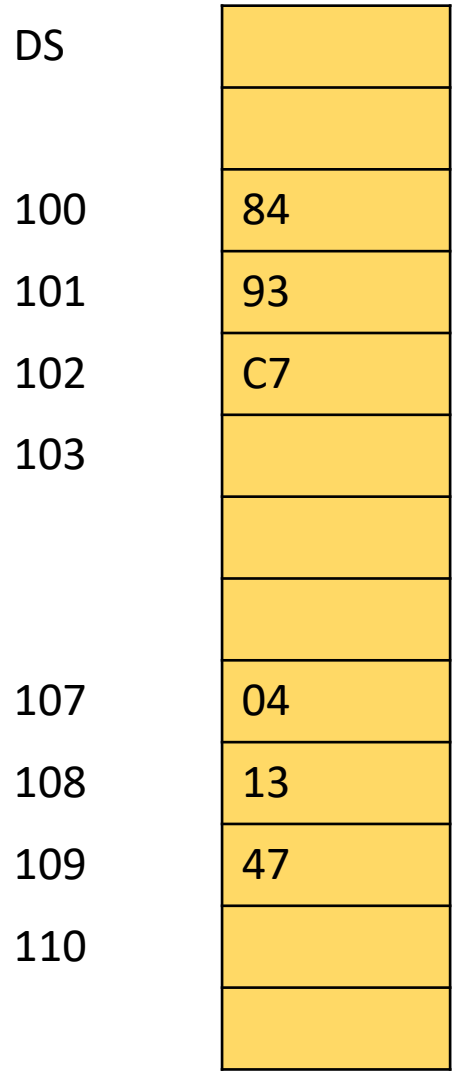
LODSB al=‘1’=31h

LODSW ax= ‘32’=3332h

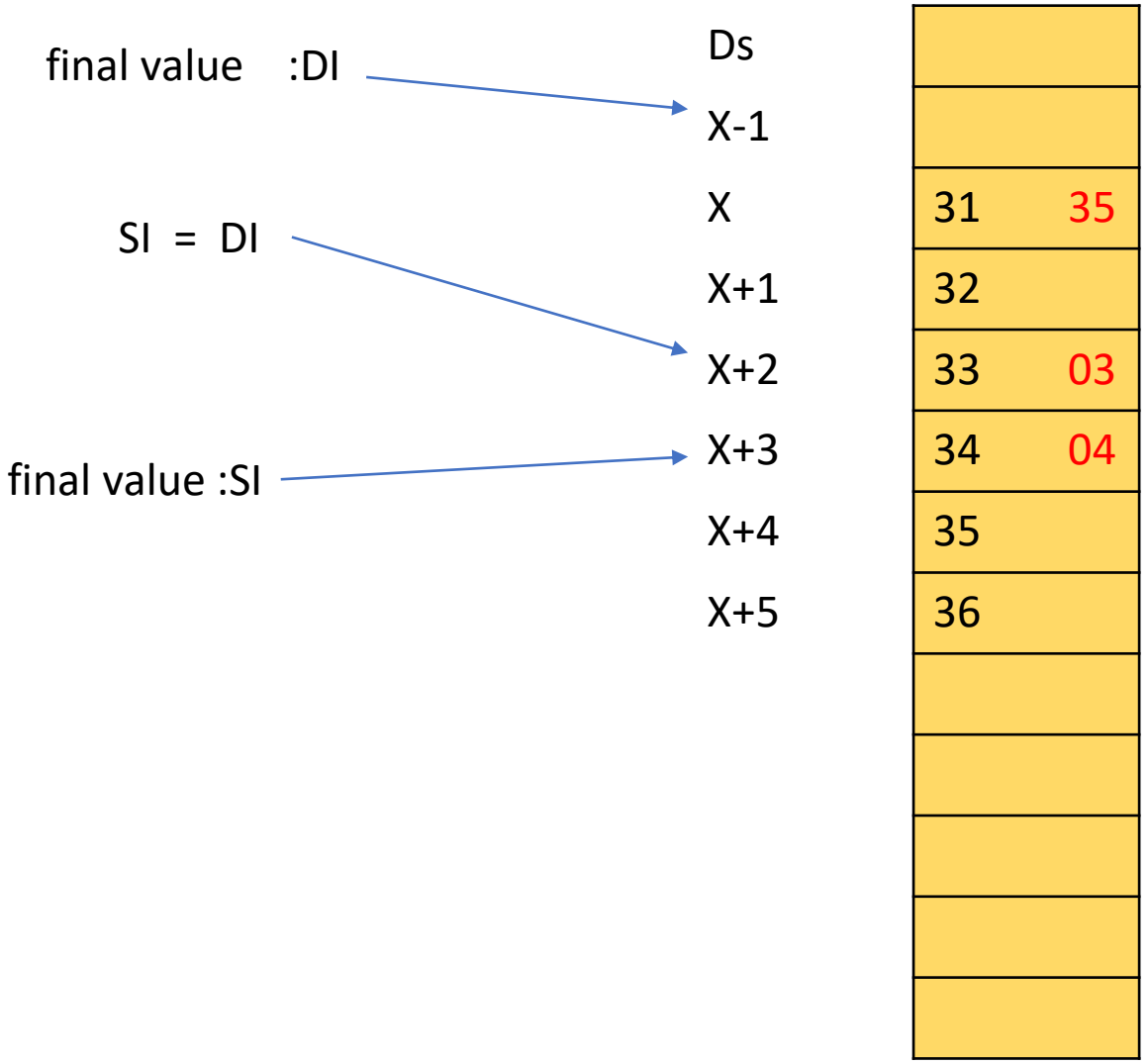


```
MOV SI,100H
MOV DI,107H
MOV CX,3
L: LODSB
AND AL,7FH
STOSB
LOOP L
```

CX	DI	SI	AL	AL
3	107	100	84	04
2	108	101	93	13
1	109	102	C7	47
0	110	103		



```
X DB ‘123456’
MOV SI,OFFSET X+2
MOV DI,SI
LODSW                AX=3433H
SUB AX,3030H         AX=0403H
STD
STOSW
LODSB                AL=35H
STOSB
```



```
X dw 111h, 222h, 333h

LEA SI, X
MOV CX, 3
REP LODSW                    AX= 333h.
```

SCASB

Compare bytes: AL from ES:[DI].

Effects on flags

Algorithm:

```
AL - ES:[DI]

set flags according to result: OF, SF, ZF, AF, PF, CF

if DF = 0 then
DI = DI + 1
else
DI = DI - 1
```

SCASW

Compare words: AX from ES:[DI].

Effects on flags

Algorithm:

```
AX - ES:[DI]

set flags according to result: OF, SF, ZF, AF, PF, CF

if DF = 0 then
   DI = DI + 2
else
   DI = DI - 2
```

CMPSB

Compare bytes: ES:[DI] from DS:[SI].

Algorithm:

```
DS:[SI] - ES:[DI]
set flags according to result: OF, SF, ZF, AF, PF, CF

if DF = 0 then
SI = SI + 1
DI = DI + 1
else
SI = SI - 1
DI = DI - 1
```

CMPSW

Compare words: ES:[DI] from DS:[SI].

Algorithm:

```
DS:[SI] - ES:[DI]

set flags according to result: OF, SF, ZF, AF, PF, CF

if DF = 0 then
   SI = SI + 2
   DI = DI + 2
else
   SI = SI - 2
   DI = DI - 2
```

```
X DB 10
Y DB 5
MOV SI,OFFSET Y
MOV DI,OFFSET X
CMPSB                  →[SI]-[DI]
JG L1
MOV AL,1
JMP END
L1:MOV AL,2
```

```
X DB ‘AB’
Y DB ‘BB’
MOV BL,2
MOV SI,OFFSET X
MOV DI,OFFSET Y
CMPSW                →[SI]-[DI]
JB L1
ADD BL,2
LI: ADD BL,2
```

REP

Repeat following MOVSB, MOVSW, LODSB, LODSW, STOSB, STOSW instructions CX times. Effect flag

Algorithm:

```
check_cx:

if CX <> 0 then do following chain instruction
CX = CX - 1
go back to check_cx
Else exit from REP cycle
```

Repeat instructions

- `REP`
- `REPNE == REPNZ`
- `REPE == REPZ`

REPNE

Repeat following CMPSB, CMPSW, SCASB, SCASW instructions while ZF = 0 (result is Not Equal), maximum CX times.

Algorithm:

```
check_cx:

if CX <> 0 then
do following chain instruction
CX = CX - 1
if ZF = 0 then:
   go back to check_cx
else
   exit from REPNE cycle
Else exit from REPNE cycle
```

REPNZ

Repeat following CMPSB, CMPSW, SCASB, SCASW instructions while ZF = 0 (result is Not Zero), maximum CX times.

Algorithm:

```
check_cx:

if CX <> 0 then do following chain instruction
CX = CX - 1
if ZF = 0 then:
   go back to check_cx
else
   exit from REPNZ cycle
Else exit from REPNZ cycle
```

REPZ

Repeat following CMPSB, CMPSW, SCASB, SCASW instructions while ZF = 1 (result is Zero), maximum CX times.

Algorithm:

```
check_cx:

if CX <> 0 then
do following chain instruction
CX = CX - 1
if ZF = 1 then:
   go back to check_cx
else
   exit from REPZ cycle
Else exit from REPZ cycle
```

REPE

Repeat following CMPSB, CMPSW, SCASB, SCASW instructions while ZF = 1 (result is Equal), maximum CX times.

Algorithm:

```
check_cx:

if CX <> 0 then do following chain instruction
CX = CX - 1
if ZF = 1 then:
   go back to check_cx
else
   exit from REPE cycle
Else exit from REPE cycle
```

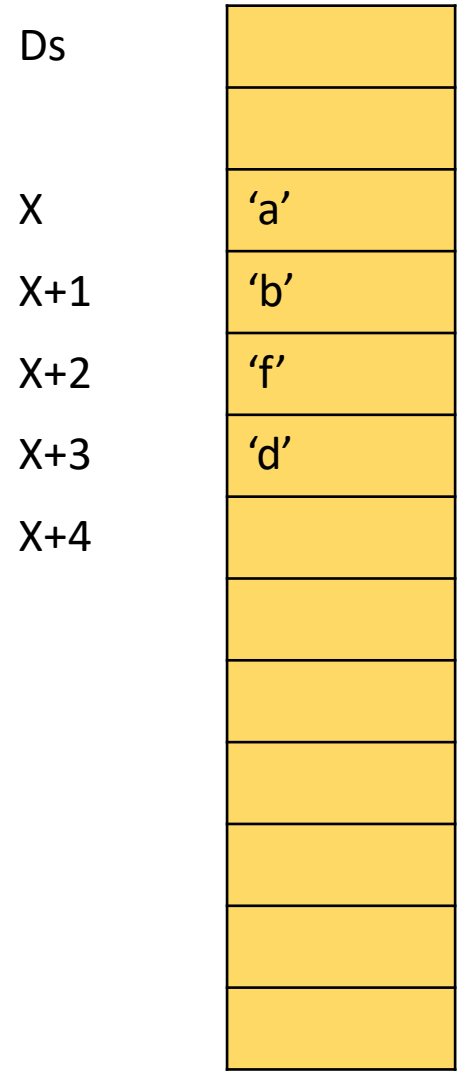
```
X DW -1,2,-4,5
Y DW -1,2,3,5
CLD
MOV AL,0
LEA SI,X
LEA DI,Y
MOV CX,4
REPE CMPSW
JB L1
INC AL
JMP END
L1:DEC AL
END:
```

CX	DI	SI	ZF	AL
4	Y	X	1	0
3	Y+2	X+2	1	0
2	Y+4	X+4	0	0
1	Y+6	X+6	0	1

X	FFH
	FFH
X+2	02
	00
X+4	FCH
	FFH
X+6	05
	00
Y	FFH
	FFH
Y+2	02
	00
Y+4	03
	00
Y+6	05
	00

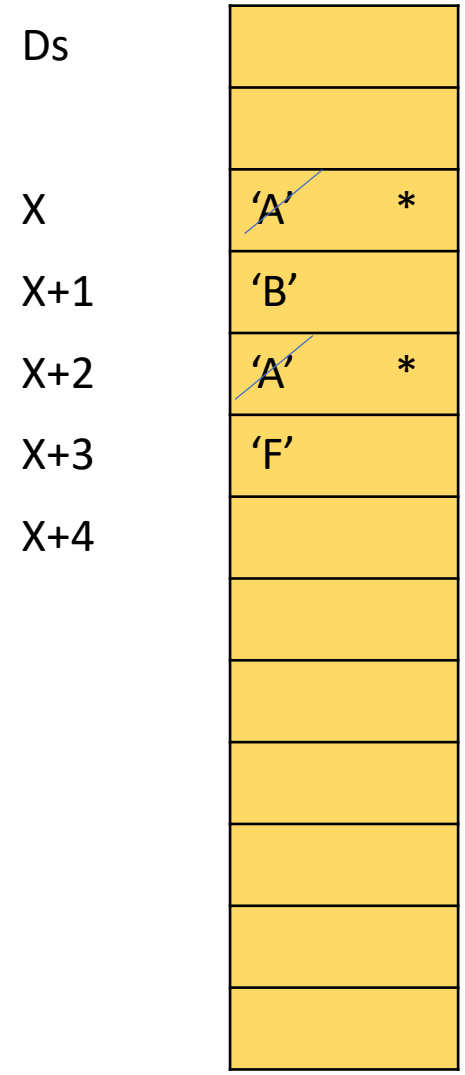
```
X DB ‘abfd’
MOV DI,OFFSET X
MOV CX,4
MOV AL,’f’
REPNE SCASB
JE L1
MOV DL,0
JMP END
L1:MOV DL,1
END:
```

CX	DI	ZF	DL
4	X	0	
3	X+1	0	
2	X+2	1	1
1	X+3		



```
X DB ‘ABAF’
MOV CX,4
MOV DI,OFFSET X
MOV AL,’A’
MOV BL,’*’
L:REPNE SCASB
JNZ EXIT
MOV [DI-1],BL
JCXZ EXIT
JMP L
EXIT:
```

CX	DI	ZF
4	X	1
3	X+1	0
2	X+2	1
1	X+3	0
0	X+4	



.

EXAMPLE : Copy 100 bytes from Array1 to Array2, using MOVSB instruction with the REP prefix

```
ORG 100H
LEA DI, X
LEA SI, Y
MOV CX, 100
CLD
REP MOVSB
HLT
X  DB 100 DUP('A')
Y  DB 100 DUP(?)
```

EXAMPLE : write an assembly code to decide if there is a letter ‘m‘ in string Z , if so store 1 in BL register

```
ORG 100H
    MOV BL,0
    CLD
    LEA DI, Z
    MOV CX,7
    MOV AL, 'm'
    L:REPNE SCASB
    JCXZ E
    MOV BL,1
    JMP L
    E:
HLT
Z DB "Maximum"
```

EXAMPLE : Write an assembly code to count number of a's in the following string (Note: using SCASB instruction):

```
STR  DB   'aabcadxyaaAa'
MOV CX, L
MOV DI, OFFSET STR
MOV AL, 'a'
MOV DL, 0
L1:
REPNE SCASB
JNE END
INC DL
JCXZ END
JMP L1
END:
```

EXAMPLE : find number of ones in each element in array X and store the count in another array.

```
X DB   0F7H,0,0FH,0CFH
Y DB ?,?,?,?
MOV SI,OFFSET X
MOV DI,OFFSET Y
MOV CX,4
L3: LODSB
MOV BX,CX
MOV CX,8
MOV DL,0
L: SHR AL,1
JNC L2
INC DL
L2: LOOP L
MOV CX,BX
MOV AL,DL
STOSB
LOOP L3
```

Homeworks

- Question1 : copy the following string to another string starting from first space .

X db ‘This is my program’

Y db 20 dup(?)

- Question 2: replace the character ‘a’ in the following string to ‘*’

String db ‘abcdefaghiak’